C++ Programming Language

Prepared by

Eng. Randa Aziz Al-Dallah Eng. Nawal Ameen Al-Zabin

الطبعة الأولى

2018م – 1439هـ

مكتبة المجتمع العربي

مكتبة المجتمع العربي للنشر والتوزيع

Scanned by CamScanner

The Hashemite Kingdom of Jordan.
The Deposit Number at The National Library
(2017/8/4341)

006

Al-Dallah, Randa Aziz

Programming Language/ Randa Aziz Al-Dallah, Nawal Ameen Al-Zabin.- Amman: Arab Community Library for publishing and Distribution, 2017.

( ) p.

Deposit No: 2017/8/4341

Descriptors: /Program//Language/

• يتحمل المؤلف كامل المسؤولية القانونية عن محتوى مصنفه ولا يعبر هذا المصنف عن رأي دائرة المكتبة الوطنية أو أي جهة حكومية أخرى.

جميع حقوق الطبع محفوظة

لا يسمح بإعادة إصدار هذا الكتاب أو أي جزء منه أو تخزينه في نطاق استعادة المعلومات أو نقله بأي شكل من الأشكال، دون إذن خطي مسبق من الناشر

عمان – الأردن

All rights reserved. No part of this book may be reproduced, stored in a retrieval system or transmitted in any form or by any means without prior permission in writing of the publisher.

الطبعة الانجليزية الأولى

2018م – 1439هـ

مكتبة المجتمع العربي للنشر والتوزيع

عمان – وسط البلد – ش. السلط – مجمع الفحيص التجاري
تلفاكس 96264632739- خلوي 962795651920. ص.ب. 8244 عمان 11121 الأردن
ش. الملكة رانيا العبد الله – مقابل كلية الزراعة – مجمع سمارة التجاري
Email: Moj_pub@yahoo.com - Info@ muj-arabi-pub.com
www.muj-arabi-pub.com
مكتبة المجتمع العربي للنشر والتوزيع

ISBN 978-9957-83-635-1 (ردمك)

Scanned by CamScanner

Contents

Contents *page*

LESSON 1

Introduction to C++ Programming Language..... 11

LESSON 2

Basic Elements of C++ Part1..... 37

LESSON 3

Basic Elements of C++ Part2..... 65

LESSON 4

Control Structure: Conditional Statements..... 85

LESSON 5

Control Structure: Looping_Part1..... 113

LESSON 6

Control Structure: Looping_Part2..... 133

LESSON 7

Functions_Part1..... 153

LESSON 8

Functions_Part2..... 175

Scanned by CamScanner

LESSON 9

Arrays- one dimensional arrays 211

LESSON 10

Arrays- two dimensional arrays 237

Scanned by CamScanner

Introduction

This book is a vital activity for helping you gain valuable programming skills, by giving step-by-step examples of varying length and complexity under the guidance of your instructor to become familiar with programming concepts, design, and coding.

It covers introductory and intermediate C++ programming concepts. C++ is a powerful language based on C, it improves on many of C's features and provides object-oriented programming (OOP) capabilities that hold great promise for increasing software productivity, quality and reusability. The programming language C++ (originally named "C with Classes") was devised by Bjarne Stroustrup also an employee from Bell Labs (AT&T). Stroustrup started working on C with Classes in 1979 and states that the purpose of C++ is to make writing good programs easier and more pleasant for the individual programmer. When he designed C++, he added OOP (Object Oriented Programming) features to C without significantly changing the C component. Thus C++ is a "relative" (called a superset) of C, meaning that any valid C program is also a valid C++ program. In 1983, the name of the language was changed from C with Classes to C++.

This book is divided into chapters called "lesson". Each lesson consists of three parts, the *overview*, *pre-lab writing assignments* as a prerequisite to attempting the last part which is the *lab exercises.*

Electronic files for all necessary programs and data files are included in this material.

Scanned by CamScanner

LESSON 1

Introduction to C++ Programming Language

Outline:

- *Overview*
  - ✓ *Computer Organization*
  - ✓ *Introduction to Programming*
  - ✓ *Programming Languages Levels*
  - ✓ *C++ Compiler*
  - ✓ *Integrated Development Environment IDE*
  - ✓ *Error Types*
- *PRE-LAB WRITING ASSIGNMENT*
- *Lesson 1 Exercises*

Scanned by CamScanner

LESSON (1)
Introduction to C++ Programming Language

Overview

Computer Organization

A computer consists of both hardware and software. The hardware refers to the components used to build a computer. Breaking down the components into categories, you will find three main groups:

1. Input/Output
2. Memory
3. CPU

Input/Output category refers to the components a computer uses that receive data and send information. Input devices do the receiving and the output devices do the sending. Some examples of input devices are a keyboard, mouse, and a gaming controller. Examples of output devices are a printer, monitor, and speakers.

The memory within a computer can be broken down into two categories: volatile memory (main memory) and non-volatile memory (secondary memory). Volatile memory is the random access memory (RAM) while the non-volatile is either your hard disk drive (HDD) or compact disk drive (CDD). RAM can be tapped into immediately by programs on a computer allowing it to compute faster, but if the user needs to save information for later use, using the HDD or CDD is required.

The central processing unit (CPU) is used to calculate the commands sent to it by the programs used on the system. It performs all the arithmetic and logical operations. The CPU consists of three main components: Control Unit (CU), Arithmetic and Logic Unit (ALU), and Registers.

The computer software is used to communicate with the computer processor to direct certain operations to be performed. This is done through computer programming languages. Software can be broken into two parts: System Software and Application Software.

Scanned by CamScanner

Introduction to Programming

A computer program is a collection of instructions that performs a specific task when executed by a computer. A computer requires programs to function, and typically executes the program's instructions in a central processing unit (CPU). A computer program is usually written by a computer programmer in a programming language. Each instruction in a program is a command that tells the CPU to perform a specific operation. Here's an example of an instruction that might appear in a program:

10110000

To you and me, this is only a series of 0s and 1s. To a CPU, however, this is an instruction to perform an operation. It is written in 0s and 1s because CPUs only understand instructions that are written in machine language, and machine language instructions always have an underlying binary structure. A machine language instruction exists for each operation that a CPU is capable of performing. For example, there is an instruction for adding numbers, there is an instruction for subtracting one number from another, and so forth. The entire set of instructions that a CPU can execute is known as the CPU's instruction set.

Although a program can be stored on a secondary storage device such as a disk drive, it has to be copied into main memory, or RAM, each time the CPU executes it. For example, suppose you have a word processing program on your computer's disk. To execute the program you use the mouse to double-click the program's icon. This causes the program to be copied from the disk into main memory. Then, the computer's CPU executes the copy of the program that is in main memory.

Programming Languages Levels

Computers can only execute programs that are written in machine language. Programming in machine language would also be very difficult because putting a 0 or a 1 in the wrong place will cause an error, and it is impractical for people to write programs in machine language. For this reason, assembly language was created in the early days of computing as an alternative to machine language. Instead of using binary numbers for

Scanned by CamScanner

instructions, assembly language uses short words that are known as mnemonics. For example, in assembly language, the mnemonic add typically means to add numbers, mul typically means to multiply numbers, and mov typically means to move a value to a location in memory. When a programmer uses assembly language to write a program, he or she can write short mnemonics instead of binary numbers.

Assembly language programs cannot be executed by the CPU, however. The CPU only understands machine language, so a special program known as an assembler is used to translate an assembly language program to a machine language program. This process is shown in Figure 1-1.

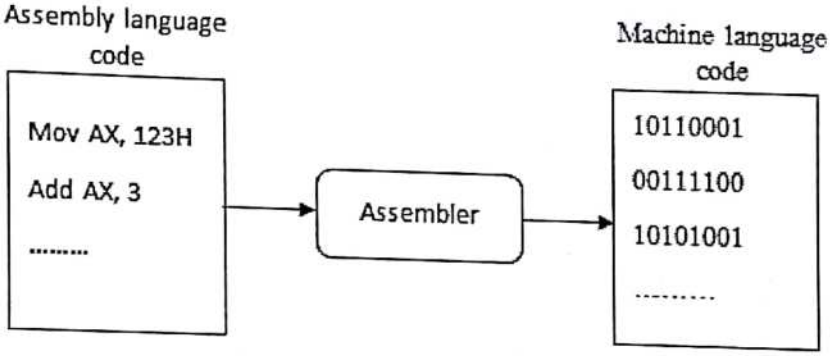


Figure 1-1: An assembler translates an assembly code to machine code.

Assembly language is primarily a direct substitute for machine language, and like machine language, it requires that you know a lot about the CPU. Assembly language also requires that you write a large number of instructions for even the simplest program. Because assembly language is so close in nature to machine language, it is referred to as a low-level language.

In the 1950s, a new generation of programming languages known as high-level languages began to appear. A high-level language allows you to create powerful and complex programs without knowing how the CPU works, and without writing large numbers of low-level instructions. In addition, most high-level languages use words that are easy to understand.

Each high-level language has its own set of predefined words that the programmer must use to write a program. The words that make up a high-level programming language are known as key words or reserved words. Each key word has a specific meaning, and cannot be used for any other purpose. For example, in C++ the *int* keyword is used for defining a

Scanned by CamScanner

variable of integer type. In addition to key words, programming languages have operators that perform various operations on data. For example, all programming languages have math operators that perform arithmetic. In addition to key words and operators, each language also has its own syntax, which is a set of rules that must be strictly followed when writing a program. The syntax rules dictate how key words, operators, and various punctuation characters must be used in a program. When you are learning a programming language, you must learn the syntax rules for that particular language. The individual instructions that you use to write a program in a high-level programming language are called statements. A programming statement can consist of key words, operators, punctuation, and other allowable programming elements, arranged in the proper sequence to perform an operation.

Because the CPU understands only machine language instructions, programs that are written in a high-level language must be translated into machine language. Depending on the language that a program has been written in, the programmer will use either a compiler or an interpreter to make the translation. A compiler is a program that translates a high-level language program into a separate machine language program as illustrated in figure 1-2. But the interpreter reads each individual instruction in the program, it converts it to machine language instructions and then immediately executes them. This process repeats for every instruction in the program as illustrated in figure 1-3.

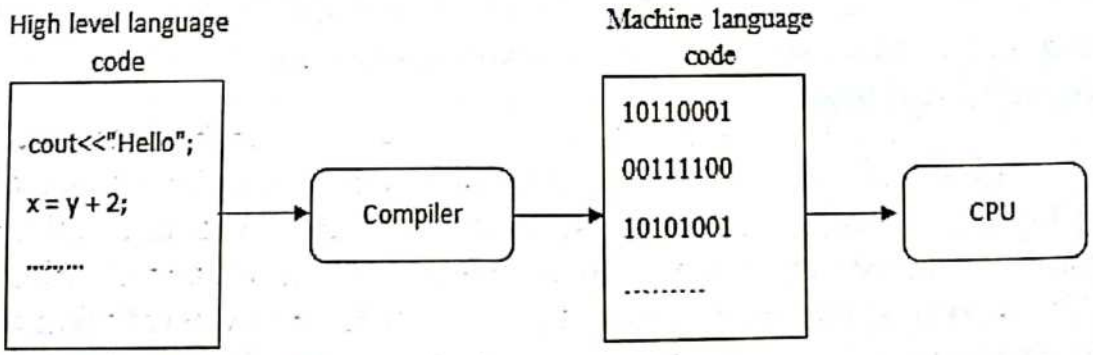


Figure 1-2: Compiling a high level code and executing it

Scanned by CamScanner

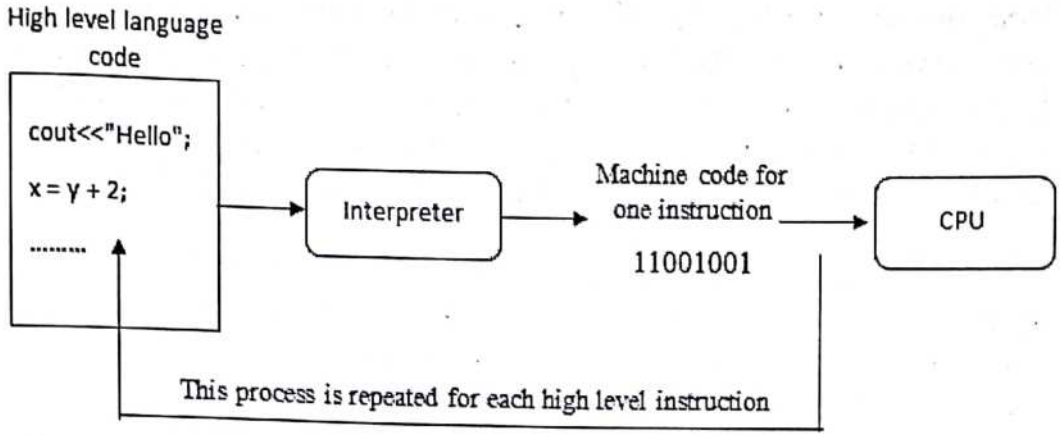


Figure 1-3: Executing a high level code with an interpreter

The statements that a programmer writes in a high-level language are called source code, or simply code. Typically, the programmer types a program's code into a text editor and then saves the code in a file on the computer's disk. Next, the programmer uses a compiler to translate the code into a machine language program, or an interpreter to translate and execute the code. If the code contains a syntax error, however, it cannot be translated. A syntax error is a mistake such as a misspelled key word, a missing punctuation character, or the incorrect use of an operator. When this happens the compiler or interpreter displays an error message indicating that the program contains a syntax error. The programmer corrects the error and then attempts once again to translate the program.

C++ Compiler

You need two programs to create your own C++ programs. First, you need a text editor that you can use to enter your C++ instructions. However, an editor that knows something about the syntax of C++ is preferable; it can save you a lot of typing, and sometimes highlight any mistakes you might make as you type, in much the same way that a spell checker highlights misspelled words in a word processor. Such as Microsoft Visual C++ 2010.

When you write a C++ program, the second program you need is a compiler that converts your C++ source statements into machine language that the computer can understand and interpret. This process of converting from source-code C++ statements to machine code is called building. There are many stages involved in creating an executable file

Scanned by CamScanner

from the source file. The main stages include Preprocessing, Compiling and Linking in C++. This means that even if the program gets compiled, it may result in not running as errors may arise during the linking phase. Hence most IDE (Integrated Development Environment) consider the term build for transforming source code file to an executable file.

Now, how you compile a certain program that depends upon operating system and compiler. These days' compilers are run from IDE (Integrated Development Environment) that contains building and analysis tools. You just have to click the menu to compile the program and then execute it. But actually the whole process occurs in different phases.

The compilation of a C++ program involves three phases (See figure 1-4):

1. ***Preprocessing:*** the preprocessor takes a C++ source code file and deals with the #includes, #defines and other preprocessor directives (that starts with # sign). The output of this step is a "pure" C++ file without pre-processor directives. For example, in c++ program #include<iostream.h> will tell the preprocessor to read all the contents of the iostream header file and include the contents into the program and generate the separate C++ program file. C++ supports many preprocessor directives like #include, #define, #if, #else etc.

2. ***Compilation:*** the compiler takes the pre-processor's output and produces an object file from it. But you still can't run these object files until to convert them into executable file, now here linker comes into play, which links all the object files to generate single executable file.

3. ***Linking:*** the linker takes the object files produced by the compiler and produces either a library or an executable file. The file created after linking is ready to be loaded into memory and executed by the system. There is difference in linking and compilation when it comes to understanding errors. Compiler shows errors in syntax, for example semi-colon not mentioned, data type not defined etc but if there is an error that function has been defined multiple times, then this error is from linker as it's indicating that two or more source code files have the same meaning and that is leading to an error.

Scanned by CamScanner

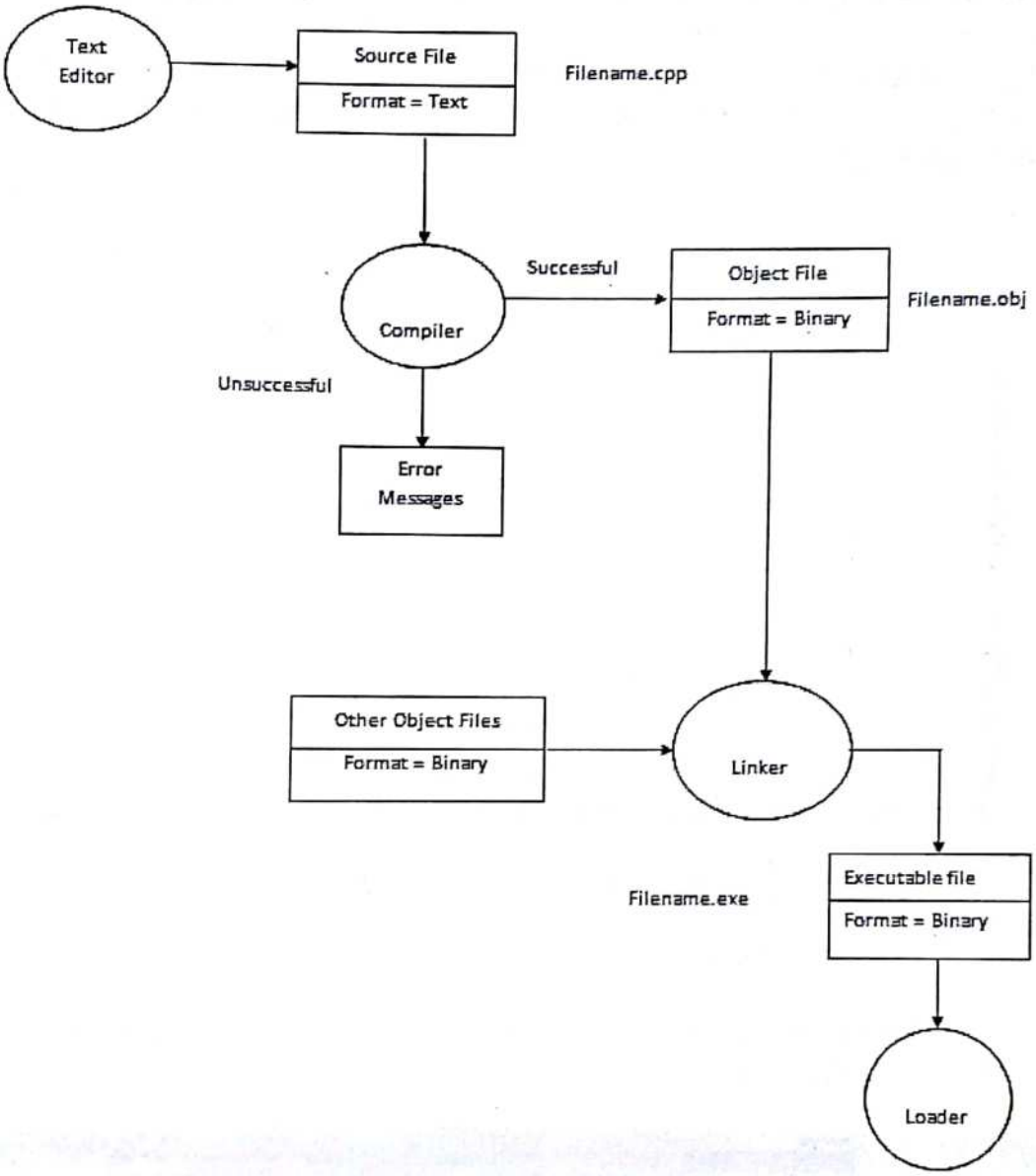


Figure 1-4: C++ Translation Process

Integrated Development Environment IDE

The Integrated Development Environment (IDE) is a software package that contains an editor (used to write programs), a compiler (that translates programs) and run time component in one system. Such that IDEs is the Microsoft Visual Studio 2010 Express.

Scanned by CamScanner

To create a new project, you must follow the following steps:

Step1: Launch the MS Visual C++ 2010 Express software from task bar. The main window of Visual Studio 2010 should be similar to the figure 1-5:

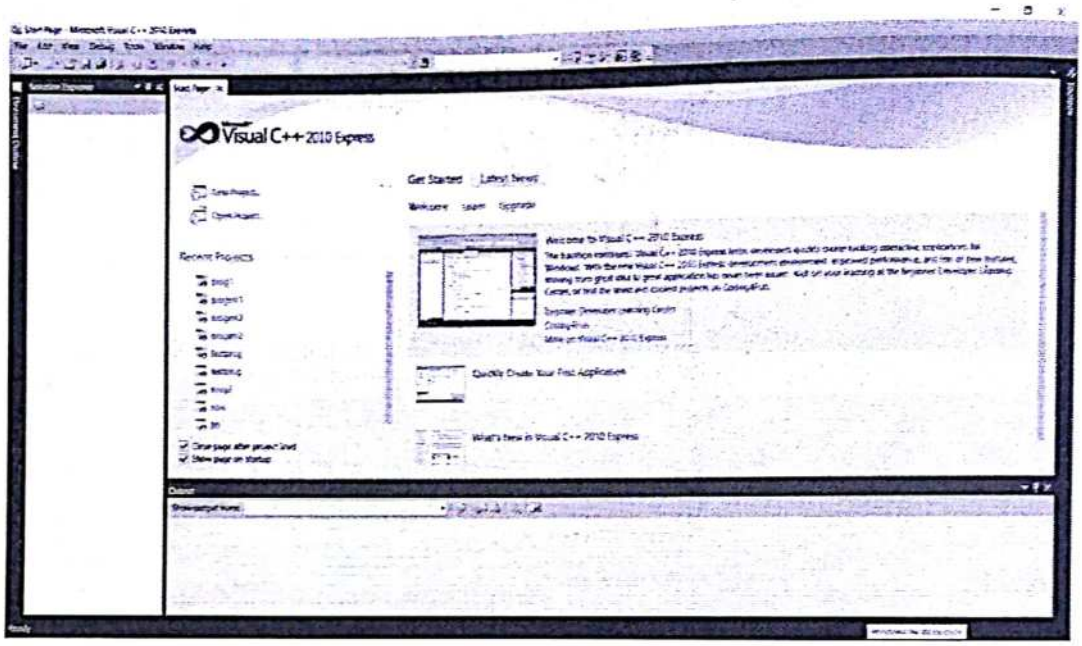


Figure 1-5: Start Page

Step 2: In the menu bar, click File >> New >> Projects, to display the New Project dialog box shown in figure 1-6:

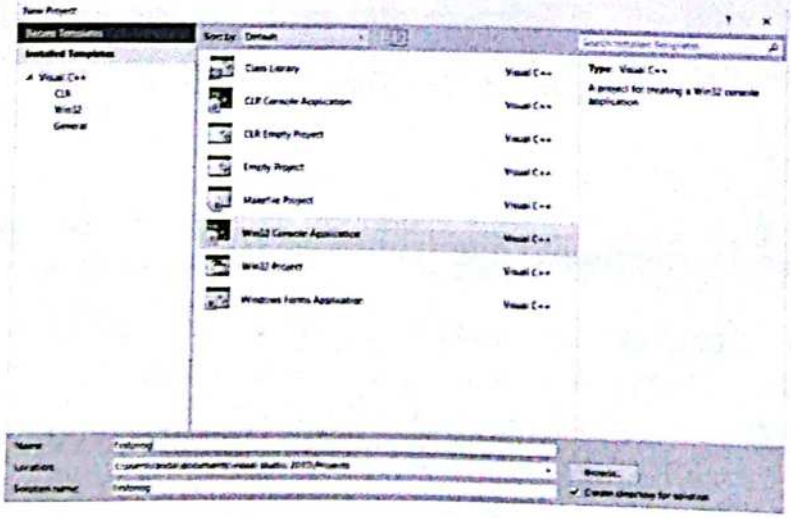


Figure 1-6: New Project

Scanned by CamScanner

In the New dialog box, select by clicking Visual C++ in the Installed Template pane and Win32 Console Application in the middle pane. You then enter a name of your choice for the project (e.g., firstprog as shown) in the Name box, select the location or folder in which you'd like to store project files by clicking the Browse. Note that there is no need to enter a name in the Solution name box; the system fills the project name in it by default.

Click on the OK button to display the Win32 Application Wizard – firstprog window shown in figure 1-7:

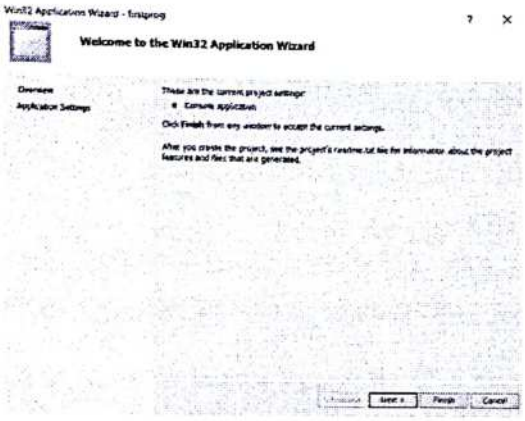


Figure 1-7: Win32 Application Wizard

Click the Next button to display the dialog box as shown in figure 1-8:

Figure 1-8: Application Settings

Check the Empty project box as shown above and click on the Finish button to proceed to the next step.

Scanned by CamScanner

Step3: Now the system displays the window as in figure 1-9.

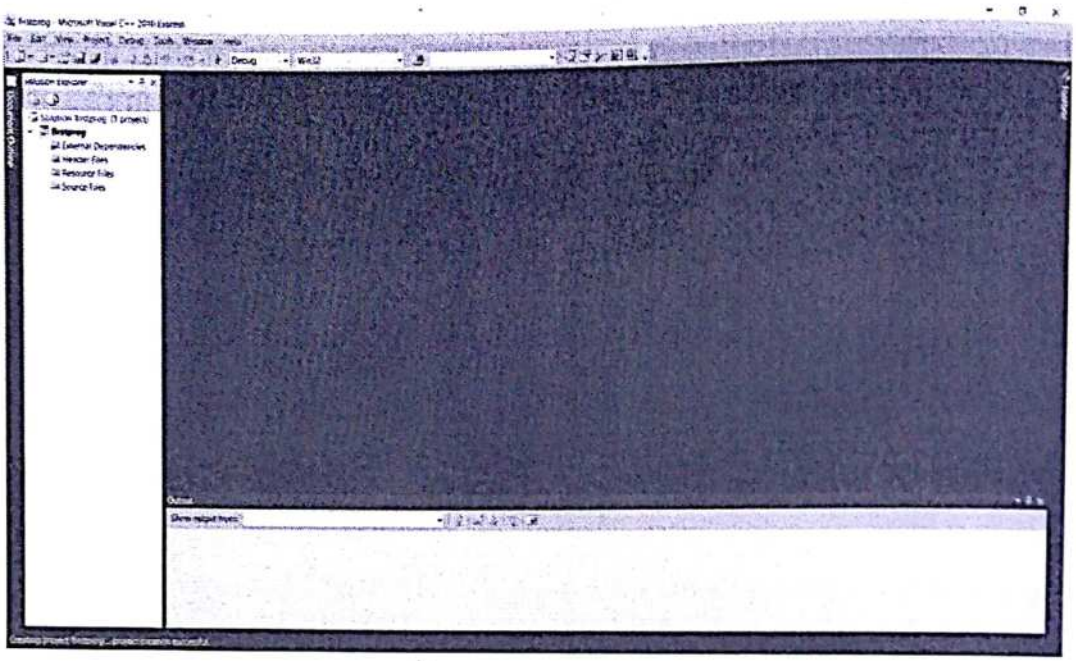


Figure 1-9: Microsoft Visual C++ 2010 Express

Right click on the Source Files folder in the Solution Explorer pane. As shown in figure 1-10:

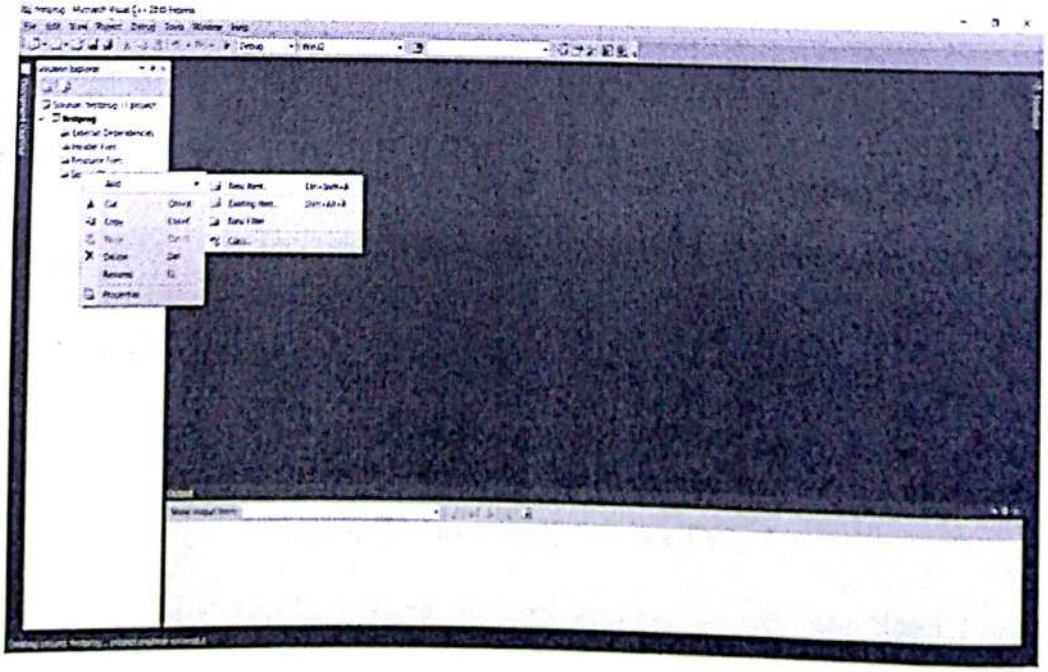


Figure 1-10: Solution Explorer Pane

Scanned by CamScanner

In the popup menu, click Add then New Item, to display the Add New Item – firstprog dialog box as shown in figure 1-11:

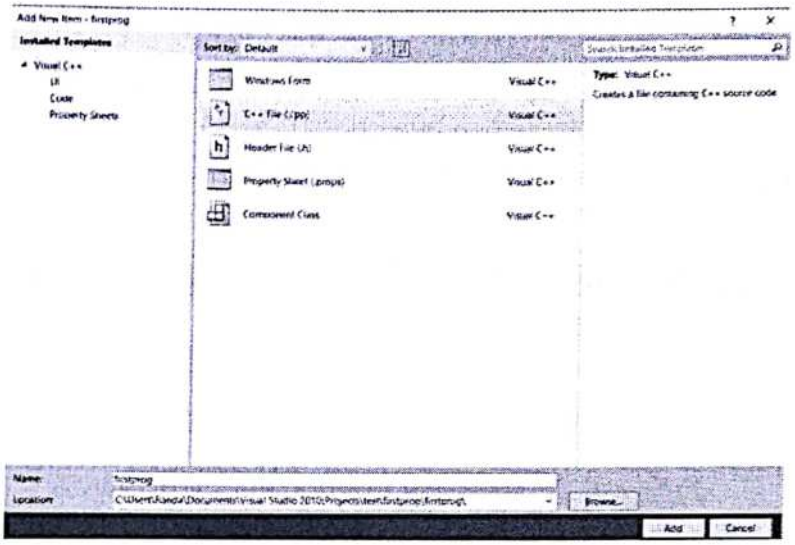


Figure 1-11: Add New Item

Select C++ Files (.cpp) by clicking on it in the middle pane and enter an arbitrary file name (e.g., firstprog, the same name we used for the project is chosen here). Click Add to proceed to the next step.

Step4: The system displays the window as in figure 1-12. Notice that

1. The Source Folder in Solution Explorer pane (make sure the Solution Explorer instead of Class View tab is selected) contains the firstprog.cpp file that we just added and

2. The blank editing area/board is displayed with a firstprog.cpp tab for you to enter your C++ source code.

Scanned by CamScanner

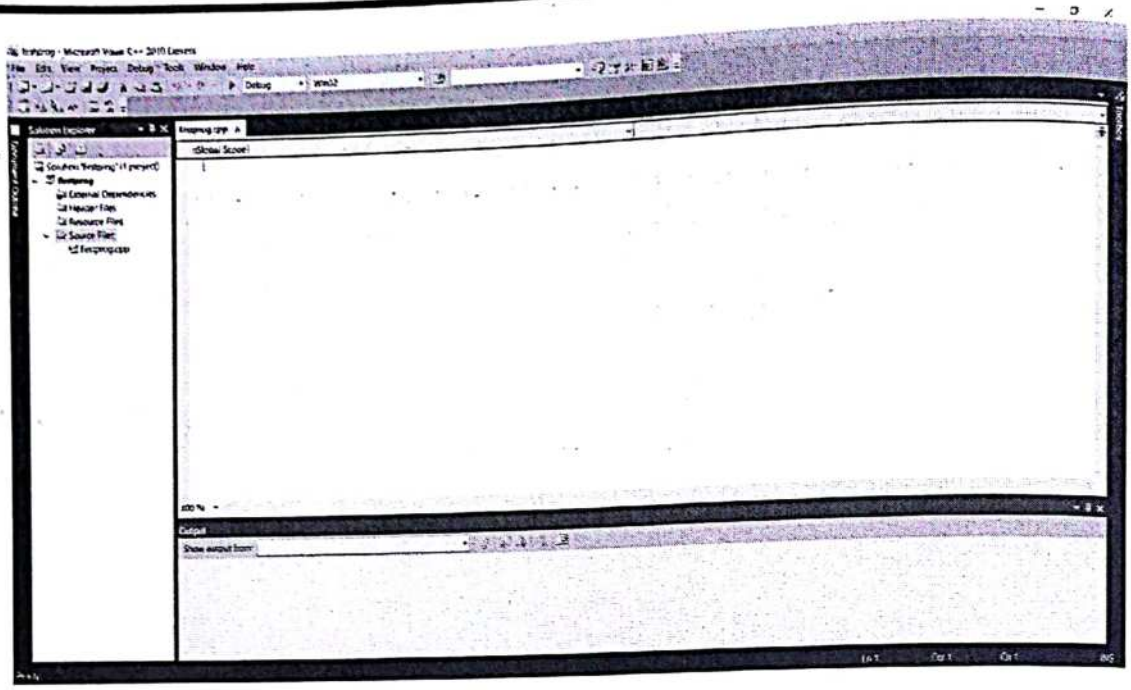


Figure 1-12: Microsoft Visual C++ 2010 Express

After writing your program, to compile, link, load, and execute a program in a single step, click Debug in the menu bar and the click Start Debugging.

Error Types

There are at least three kinds of errors that you can encounter when writing computer programs:

1. Syntax errors
2. Runtime errors
3. Logic errors

Syntax errors

Syntax or format errors are a major problem for most beginning programmers using languages such as C++, Java, and C#. Just like English, all computer languages have their own set of grammar rules that have to be obeyed. A syntax error occurs when the programmer fails to obey one of the grammar rules of the language. Typically this involves things like using the wrong case, putting punctuation where it is not supposed to be, failing to put punctuation where it is supposed to be, etc. This kind of error is the easiest to figure out. If we have something that

Scanned by CamScanner

violates the grammatical rules of the language, the compiler will give us error messages. These have to be corrected and a grammar error free program must be submitted to the compiler before it translates the source code into machine language. In C++, for example, instructions end with a semicolon. The following would indicate a syntax error:

```
cout<< "Welcome to C++"
```

Since there is no semicolon at the end, the compiler would indicate an error, which must be corrected as follows:

```
cout<< "Welcome to C++";
```

Runtime errors

In computer science, run time or execution time is the time during which a program is running (executing), in contrast to other program lifecycle phases such as compile time, link time and load time.

Once we have the executable code, the program is ready to be run. Hopefully it will run correctly; however that is not always the case. During "run time", we may encounter a second kind of error called a run time error. This error occurs when we ask the computer to do something it cannot do. The run time errors are not detected by the compiler, because the code is syntactically correct.

An example of a run time error is a statement that causes division by zero. In the following statement, the syntax is correct, but the statement may cause a division by zero error if the variable *n2* is assigned a value of 0:

```
result = n1 / n2;
```

Any value divided by 0 produces an infinitely large result, which is too large to be accommodated by the computer. The common trait of these errors is that the code works correctly in many situations, but fails in others. The risk is that since there is nothing syntactically wrong with the code, the error could occur when the program is already in use.

Logic errors

A logic error is a bug in a program that causes it to operate incorrectly, but not to terminate abnormally (or crash). A logic error

Scanned by CamScanner

produces unintended or undesired output or other behavior. Unlike a program with a syntax error, a program with a logic error is a valid program in the language, though it does not behave as intended. The only clue to the existence of logic errors is the production of wrong solutions. Logic errors are usually the most difficult and worst kind of errors to find and fix, because there frequently is no obvious indication of the error. Usually the program runs successfully. It simply doesn't behave as it should. In other words, it simply doesn't produce the correct answers.

This example is to calculate the average of two numbers contains a logic error. It is missing parentheses in the calculation, so it compiles and runs but does not give the expected answer due to operator precedence (division is evaluated before addition).

```
c = a + b / 2;   /* should be c = (a + b) / 2*/
```


Scanned by CamScanner

PRE-LAB WRITING ASSIGNMENT

Multiple Choice

1. A(n) _____ is a set of instructions that a computer follows to perform a task.

   a. compiler **b.** program

   c. interpreter **d.** programming language

2. The physical devices that a computer is made of are referred to as _____.

   a. hardware **b.** software

   c. the operating system **d.** tools

3. The part of a computer that runs programs is called _____.

   a. RAM **b.** secondary storage

   c. main memory **d.** the CPU

4. This is a volatile type of memory that is used only for temporary storage while a program is running.

   a. RAM **b.** secondary storage

   c. the disk drive **d.** the USB drive

5. Computers can only execute programs that are written in _____.

   a. Java **b.** assembly language

   c. machine language **d.** Python

6. The _____ translates an assembly language program to a machine language program.

   a. assembler **b.** compiler

   c. translator **d.** interpreter

Scanned by CamScanner

7. The rules that must be followed when writing a program are called _____.

a. syntax
b. punctuation
c. key words
d. operators

8. A(n) _____ program translates a high-level language program into a separate machine language program.

a. assembler
b. compiler
c. translator
d. utility

9. Compilers detect _____ errors

a. syntax
b. logical
c. run time
d. all of them

10. Usually the most difficult errors to correct are the _____ errors, since they are not detected in the compilation process.

a. syntax
b. logical
c. run time
d. syntax, logical and run time

Scanned by CamScanner

Lesson 1 Exercises

Assignment 1.1: Opening, Compiling and Running your First Program

Step1: Logon to your eLearning system.

Step2: Bring in the *firstprog.cpp* program from Lab1 folder.

Step3: Compile and run the program using visual C++ IDE.

Step4: What is printed on the screen?

The code of *firstprog.cpp* is as follows:

```
// This is the first program that just writes out a simple message

#include<iostream> // needed to perform Input and Output
#include<conio.h> // provide console input/output
using namespace std;

void main(){

    cout<<"***** This is My First Program *****"<<endl;
    cout<<"WELCOME TO C++"<<endl;
    getch(); // reads a character from the console without echo.

}
```

Assignment 1.2: Compiling a Program with a Syntax Error

Step1: Bring in the progerr1.cpp program from the Lab1 folder.

Step2: Compile the program.

Here we have many syntax errors. The error message you receive may be different depending on the system you are using. The error messages are:

Scanned by CamScanner

progerr1.exe - 2 error(s), 0 warning(s)

progerr1.cpp(11) : error C2146: syntax error : missing ';' before identifier 'cin'

progerr1.cpp(17) : error C2146: syntax error : missing ';' before identifier 'cout'

The compiler insists that a semicolon is missing somewhere.

Step3: To correct the errors place a semicolon after the line *cout<<"Enter the first number:"* and after the line *cout<<"***** The Summation = "<<x+y*

Step4: Recompile the program and when you have no syntax errors, run the program.

Step5: input a number 4 and a number 5 when asked. Record the output.

Step6: Try running the program with different numbers. Record your output.

Scanned by CamScanner

The code of *progerr1.cpp* is as follows:

```
// This program contains a syntax error

#include<iostream>
#include<conio.h>

using namespace std;

void main(){

    int x, y;

    cout<<"Enter the first number:"
    cin>>x;

    cout<<"Enter the second number:";
    cin>>y;

    cout<<"***** The Summation = "<<x+y
    cout<<" *****"<<endl;

    getch();

}
```

Assignment 1.3: Compiling and Running a Program with a Run Time Error

Step1: Bring in the *progerr2.cpp* program from the Lab1 folder.

Step2: Compile the program. You should get no syntax errors.

Step3: Run the program and input 5 when asked. You should now see the run time error. Because the program is asking the computer to break a law of math by dividing by zero. it cannot be done. On some installations, you may see this as output that looks very strange.

Scanned by CamScanner

Step4: Correct the program to divide by 3 instead of 0. Change the statement *d = 0;* to *d = 3;*

Step5: Recompile and run the program. Input 5 when asked. Record your output.

Step6: Try running the program with different numbers. Record your output.

The code of *progerr2.cpp* is as follows:

Scanned by CamScanner

```
// This program contains a run time error (division by 0)

#include<iostream>
#include<conio.h>
using namespace std;

void main(){

    int d;
    float n;
    float result;

    cout<<"Enter a number:";
    cin>>n;

    d = 0;
    result = n / d;

    cout<<"***** The Result = "<<result<<endl;

    getch();

}
```

Assignment 1.4: Compiling and Running a Program with a Logical Error

Step1: Bring in the *progerr3.cpp* program from the Lab1 folder. This program compute the average value of two numbers entered from keyboard.

Step2: Compile the program. You should get no syntax errors.

Step3: Run the program and input the number 90 and the number 80 when asked. Record the output.

Scanned by CamScanner

Note that the average result is incorrect. This is due to operator precedence, division is evaluated before addition.

Step4: this program has no syntax or run time errors, but it has a logical error. Correct the program by putting parentheses to the average equation as follows:

average = (n1+n2) / 2

Step5: Recompile and run the program. Input 90 and 80 when asked. Record the output.

Note the average result is correct.

Step6: Try running the program with different numbers. Record your output.

Scanned by CamScanner

The code of *progerr3.cpp* is as follows:

```
// This program contains a logical error (Incorrect result)

#include<iostream>
#include<conio.h>
using namespace std;

void main(){

    int n1, n2;
    float average;

    cout<<"Enter a first number (n1):";
    cin>>n1;

    cout<<"Enter a second number (n2):";
    cin>>n2;

    average = n1 + n2 / 2;

    cout<<"***** The Average = "<<average;
    cout<<" ***** "<<endl;

    getch();
}
```


Scanned by CamScanner

LESSON 2

Basic Elements of C++

Part1

Outline:

- *Overview*
  - ✓ *Structure of a C++ Program*
  - ✓ *Standard output (cout)*
  - ✓ *Memory Concepts*
  - ✓ *Variables and Constants*
  - ✓ *Assignment Statement*
  - ✓ *Standard input (cin)*
  - ✓ *Arithmetic Operators*
- *PRE-LAB WRITING ASSIGNMENT*
- *Lesson 2Exercises*

Scanned by CamScanner

LESSON (2)
Basic Elements of C++: Part1

Overview

Structure of a C++ Program

Probably the best way to start learning a programming language is by writing a program. For now, we can think of a C++ program as consisting of two general divisions: header and main. The header contains an information on which library will be used by the program. Every C++ program has a main function which indicates the start of the executable instructions.

Therefore, here is our first program:

```
// Program Name: prog1.cpp

#include<iostream>
#include<conio.h>
using namespace std;

int main() // main() is where program execution begins.
{

    cout<<"Hello, World!"<<endl;

    getch();

}// End main.
```

We are going to look line by line at the code we have just written:

1) Comments

// Program Name: prog 1.cpp

This is a comment line. All lines beginning with two slash signs (//) are considered comments and do not have any effect on the behavior of the program. The programmer can use them to include short explanations or observations within the source code itself to make it more readable. In this case, the line is a brief description of what our program is.

Scanned by CamScanner

C++ supports two ways to insert comments:

```
// line comment
/* block comment */
```

The first of them, known as line comment, discards everything from where the pair of slash signs (//) is found up to the end of that same line. The second one, known as block comment, discards everything between the /*characters and the first appearance of the */ characters, with the possibility of including more than one line.

2) Header files

```
#include <iostream>
#include<conio.h>
```

Lines beginning with a hash sign (#) are directives for the preprocessor. They are not regular code lines with expressions but indications for the compiler's preprocessor. In this case the directive *#include <iostream>* tells the preprocessor to include the iostream.h standard file. This specific file (iostream.h) includes the declarations of the basic standard input-output library in C++, and it is included because its functionality is going to be used later in the program. The directive *#include<conio.h>*, include the conio.h header file which used mostly by MS-DOS compilers to provide console input/output. In these directives no semicolon at the end.

3) using namespace std

Namespaces in C++ are used to define a scope and allows us to group global classes, functions and/or objects into one group. When you use *using namespace std*; you are instructing C++ compiler to use the standard C++ library. If you don't give this instruction, then you will have to use *std::*, each time you use a standard C++ function/entity. Namespaces are like drawers in a filing cabinet. All of the standard library is in one drawer, and the only way to get to the stuff in that drawer is to tell C++ where to find it. You do that by prefixing the name with *std::*,

for example, *std::cout<<"Hello, World!"<<std::endl;*

std::cout is basically like saying "Hey C++! Look in std and get me cout". If you don't tell C++ which drawer to look in, it won't find what

Scanned by CamScanner

you want. That's why this gives you an error, C++ is complaining that it can't find cout.

4) int main ( )

This line corresponds to the beginning of the definition of the main function. Functions will be discussed in details in a later lesson, but essentially, their definition is introduced with a succession of a data type *int*, *int* tells the compiler that main function returns integer value. Optionally, you can write the statement *return 0;* at the end of the main function. In case a return value is not defined by the programmer, an implicit *return 0;* at the end of the main function is inserted by the compiler. Therefore, there are two ways to write the main function as illustrated below:

```
int main()
{
    //Write your statements here
}
```

Note: with implicit *return 0;* statement. (Inserted by the compiler)

Or

```
int main()
{
    //Write your statements here

    return 0;
}
```

Note: with explicit *return 0;* statement. (Inserted by the programmer)

Some compilers, allow the programmer to write the main function in another way. The main function is declared as a *void* type. *void* tells the compiler that the main function does not return any value, no need for *return 0;* statement. as illustrated below:

Scanned by CamScanner

```
void main()
{
    //Write your statements here

}
```

The main function is the point by where all C++ programs start their execution, independently of its location within the source code. It does not matter whether there are other functions with other names defined before or after it - the instructions contained within this function's definition will always be the first ones to be executed in any C++ program. For that, all C++ programs have a main function.

The word main is followed in the code by a pair of parentheses (()). That is because it is a function declaration: In C++, what differentiates a function declaration from other types of expressions are these parentheses that follow its name. Optionally, these parentheses may enclose a list of parameters within them.

Right after these parentheses we can find the body of the main function enclosed in braces ({}). What is contained within these braces is what the function does when it is executed.

5) *cout line statement*

```
cout<<"Hello, World!"<<endl;
```

This line is a C++ statement. A statement is a simple or compound expression that can actually produce some effect. In fact, this statement performs the only action that generates a visible effect in our first program.

cout is the name of the standard output stream in C++, and the meaning of the entire statement is to insert a sequence of characters (in this case the *Hello World* sequence of characters) into the standard output stream (*cout*, which usually corresponds to the screen). *cout* is declared in the iostream standard file.

Notice that the statement ends with a semicolon character (;). This character is used to mark the end of the statement and in fact it must be included at the end of all expression statements in all C++ programs (one

Scanned by CamScanner

of the most common syntax errors is indeed to forget to include some semicolon after a statement).

The program has been structured in different lines in order to be more readable, but in C++, we do not have strict rules on how to separate instructions in different lines. For example, instead of

```
int main() {
        cout<<"Hello, World!"<<endl;
}
```

We could have written:

```
int main() {cout<<"Hello, World!"<<endl;}
```

All in just one line and this would have had exactly the same meaning as the previous code.

In C++, the separation between statements is specified with an ending semicolon (;) at the end of each one, so the separation in different code lines does not matter at all for this purpose. We can write many statements per line or write a single statement that takes many code lines. The division of code in different lines serves only to make it more legible and schematic for the humans that may read it.

The C++ language is a "case sensitive" language. That means that an identifier written in capital letters is not equivalent to another one with the same name but written in small letters. Thus, for example, the function with the name *MAIN()* is not the same as the function with name *main()*.

Standard output (cout)

C++ uses *cout* to print output to the terminal (screen) in conjunction with the insertion operator <<. The insertion operator << may be used more than once in a single statement. To use the *cout*, you need to #include <iostream.h>

The characters print exactly as they appear between the double quotes. For example, the statement *cout<<"1+1";* print *1+1* on the screen, while the statement *cout<<1+1;* print *2* on the screen.

Scanned by CamScanner

Escape sequences are used to define certain special characters within string literals. The following escape sequences are available:

Escape sequence	Description
\n	new line (takes the cursor to the beginning of the next line)
\t	horizontal tab (takes the cursor to the next tab stop)
\r	carriage return (causes a carriage return)
\a	audible bell (makes a sound from the computer)
\b	Backspace (takes the cursor back)
\\	Backslash (displays a backslash \)
\'	single quote (displays an apostrophe ')
\"	double quote (displays a quotation mark ")

When a backslash is encountered in a string of characters, the next character is combined with the backslash to form an escape sequence.

Let's begin with a simple C++ program that display a message on screen without using escape sequence character.

```
#include<iostream>
using namespace std;
int main()
{
   cout<<"Escape Sequence ";
   cout<<"in C++ Programming.";
}
```

Program output is:

```
Escape Sequence in C++ Programming.
```


Scanned by CamScanner

The above program prints the complete string message in just one line instead of two. The reason is that you didn't gave any instruction to print next cout string message in the next line. Take a look at the following program to know how to print anything in a new line using C++ Escape Sequence Character.

```
#include<iostream>
using namespace std;
int main()
{
    cout<<"C++\nProgramming.";
}
```

Program output is:

```
C++
Programming.
```

The *"\n"* used to display the output string message in a new line. Alternatively, the *endl* manipulator can also be used to print a new line. For example:

```
cout<<"C++"<<endl<<"Programming.";
```

Will produce the same output as the previous statement.

The output is:

```
C++
Programming.
```

Consider the following program. So are you able to guess the output of the program?

Scanned by CamScanner

```
#include<iostream>
using namespace std;
int main()
{
    cout<<"Randa\nAl-Dallah\nLoves C++";
}
```

Program output is:

```
Randa
Al-Dallah
Loves C++
```

Now think about the situation when you wish to display a double quotation (") mark on the screen? No, you can't put it in cout statement as it will just terminate the cout message and nothing else happen other than getting an error on the screen by a compiler. If you will simply put a quotation mark in between cout quotes then the compiler will get confused. Your quotation mark between quotes will be seen as an end of cout message by the compiler and the actual quotation mark (") will be seen as an extra.

For example: *cout<<"Quotation Symbol is inserted " here"; // Invalid*

To display a quotation mark ("), use escape sequence character. As follows:

cout<<"Quotation Symbol is inserted \" here"; //Valid

In the following program we use multiple escape sequence characters

Scanned by CamScanner

```
#include<iostream>
using namespace std;
int main()
{
   cout<< "123456789"<<endl;
   cout<<"\tMy name is Nawal\r";
   cout<<"hi\n";
   cout<<"Programming Language \"C++\"\n";
   cout<<"Programming Language \\C++\\\n";
   cout<<"Programming Language\b\b\b\bC++\n\a";
}
```

The program output is:

```
123456789
Hi          My name is Nawal
Programming Language "C++"
Programming Language \C++\
Programming LangC++e
```

Memory Concepts

The program shown in the previous sections, such as prog1.cpp, is just to obtain the result of a simple sentence written on the screen. However, programming is not limited only to printing simple texts on the screen. In order to go a little further on and to become able to write programs that perform useful tasks that really save us work, we need to introduce the concept of variables.

Let's imagine that I ask you to remember the number 5, and then I ask you to also memorize the number 2 at the same time. You have just stored two different values in your memory (5 and 2). Now, if I ask you to add 1 to the first number I said, you should be retaining the numbers 6 (that is 5+1) and 2 in your memory. Then we could, for example, subtract these values and obtain 4 as result.

Scanned by CamScanner

The whole process described above is a similar of what a computer can do with two variables. The same process can be expressed in C++ with the following set of statements:

```
a = 5;
b = 2;
a = a + 1;
result = a – b;
```

Obviously, this is a very simple example, since we have only used two small integer values, but consider that your computer can store millions of numbers like these at the same time and conduct sophisticated mathematical operations with them.

We can now define *variable* as a portion of memory to store a value. Each variable needs a name that identifies it and distinguishes it from the others. For example, in the previous code the variable names were a, b, and result, but we could have called the variables any names we could have come up with, as long as they were valid C++ identifiers.

Variables and Constants

In this section will introduce the concept of variable and constant so that our program can perform calculation on data. The identifiers are used to name variables, constants and many other components of a program.

The variable declaration syntax is:

```
datatype identifier [=value];
```

The square brackets [] in the syntax, indicates that the part in the square bracket is optional. As shown in the variable declaration syntax, *[=value]*, is an optional part. We can declare a variable with initial value or without initial value as discussed later in this section.

A *valid identifier* is a sequence of one or more letters (A-Z or a-z), digits (0-9), or underscore characters (_). Spaces, punctuation marks, and symbols cannot be part of an identifier. In addition, identifiers shall always begin with a letter. They can also begin with an underline

Scanned by CamScanner

character (_). C++ uses a number of keywords to identify operations and data descriptions (such as int, if and so on); therefore, identifiers created by a programmer cannot match these keywords.

Note: C++ is case sensitive. Therefore, the identifiers myvar, MyVar, and MYVAR are three different identifiers.

The values of variables are stored somewhere in an unspecified location in the computer memory as zeros and ones. Our program does not need to know the exact location where a variable is stored; it can simply refer to it by its name. What the program needs to be aware of is the kind of data stored in the variable. It's not the same to store a simple integer as it is to store a letter or a large floating-point number; even though they are all represented using zeros and ones, they are not interpreted in the same way, and in many cases, they don't occupy the same amount of memory.

Fundamental data types are basic types implemented directly by the language that represent the basic storage units supported natively by most systems. They can mainly be classified into:

1. Character types: They can represent a single character, letters of the alphabet (upper and lower cases), the digits 0-9 and special symbols such as ? ! @ , . and so on. Each character data is enclosed with single quotes to distinguish it from other data types. Thus '1' is different than 1. The first is a character while the second is an integer number. The most basic type is char, which is a one-byte character. The following statement defines a character variable initialized to 'A'.

   ```
   char var1= 'A';
   ```

2. Numerical integer types: Integers are real numbers that do not contain any fractional component. They can store a whole number value, such as 7 or 1024. They exist in a variety of sizes, and can either be signed or unsigned, depending on whether they support negative values or not. C++ has three data types that are integers: short, int and long. The following table illustrate various integer data types in C++.

Scanned by CamScanner

Data Type (keywords)	Description	Size	Typical Range
int / signed int	Signed integer, values may be negative, positive or zero	4 bytes	-2^{31} to $2^{31}-1$
unsigned int	Unsigned integer. Values are always positive or zero. Never negative.	4 bytes	0 to $2^{32}-1$
short / signed short	Signed short integer. Values may be negative, positive or zero	2 bytes	-2^{15} to $2^{15}-1$
unsigned short	Unsigned short integer. Values are always positive or zero. Never negative.	2 bytes	0 to $2^{15}-1$
long / signed long	Signed long integer, values may be negative, positive or zero	4 bytes	-2^{31} to $2^{31}-1$
unsigned long	Unsigned long integer. Values are always positive or zero. Never negative.	4 bytes	0 to $2^{32}-1$

The following three statements define integer variables in C++:

```
short num;
int sum;
long total;
```

3. Floating-point types: In computer science 5 = 5.0 is not a true statement. The number on the left is an integer and the number on the right is a real, or floating point number (a number that has a fractional component). Although mathematically the two are equal, the computer stores them as different data types. C++ uses both float and double to indicate floating point numbers, with different levels of precision. The table below illustrate the two types of the real number data type.

Scanned by CamScanner

Data Type (keywords)	Description	Size	Typical Range
float	Floating point number. There is no fixed number of digits before or after decimal point	4 bytes	+/- 3.4e +/- 38 (~7 digits)
double	Double precision floating point number. More accurate compared to float	4 bytes	+/- 1.7e +/- 308 (~15 digits)

The following two statements define floating-point variables in C++:

```
float average;
double factorial;
```

4. Boolean type: The boolean type, known in C++ as bool, can only represent one of two states, true or false, which are internally represented as 0 and non-zero, respectively. The boolean variable size is 1 byte. The following statements define a boolean variables initialized to false.

```
bool var1= false;
bool var2= 0;
```

The following statements define a boolean variables initialized to true:

```
bool var3= true;
bool var4= 1; // any non-zero value is true
bool var5= 30; // any non-zero value is true
bool var6= - 10; // any non-zero value is true
```

Initialization of variable

The variable declaration *int x;* they have an undetermined value (rubbish value) until they are assigned a value for the first time. But it is possible for a variable to have a specific value from the moment it is declared. This is called the initialization of the variable. Such as *int x= 10;*

Scanned by CamScanner

The following example, declare the variables with and without initial value:

```
#include<iostream>
using namespace std;
int main() {
    int x;
    int y=10;
    int z, w=30;
    cout<<"x="<<x<<endl;
    cout<<"y="<<y<<endl;
    cout<<"z="<<z<<endl;
    cout<<"w="<<w<<endl;
    return 0;
}
```

The output is:

```
x=-858993460
y=10
z=-858993460
w=30
```

Note that the variables x and z are uninitialized, they contains rubbish data. But the variables y and w, are initialized with y = 10 and w = 30.

Constants

A constant is a value or an identifier whose value cannot be altered in a program. The *const* keyword specifies that a variable's value is constant and tells the compiler to prevent the programmer from modifying it. The constant is a read only variable, any attempt to change the value of a constant will result in an error message. A constant in C++ can be of any of the basic data types. The constant declaration syntax is as follows:

const datatype identifier = value;

Scanned by CamScanner

Examples of valid constant declarations are:

```
const float PI = 3.1415;
const int STDNO = 100;
const char CH = '#';
```

Note that it is a good programming practice to define constants in CAPITALS.

Assignment Statement

The = symbol in C++ is called the assignment operator and is read "is assigned the value of." It assigns the variable on its left the value on its right. Although this symbol looks like an equal sign, do not confuse it with equality. The left hand side must always be a variable.

```
Variable_name = expression;
```

For example, number = 5, however 5 = number; generates a syntax error.

The *expression* maybe a literal, variable, constant, or math/logical expression. As shown below:

```
int x, y, z, r;
const int W=2;
x = 10; // literal example: 10is an integer literal.
y = x; // variable example: store the x value in y variable.
z = W; // constant example:
       //store the W constant value in z variable.
r = x + 12* 2; // math expression example:
               // store the equation result in r variable.
```

Standard input (cin)

cin can be used to input a value entered by the user from the keyboard. However, the extraction operator >> is required to get the typed value from cin and store it in the memory location (variable).

Scanned by CamScanner

```
int age;
cin >> age;
```

The first statement declares a variable of type *int* called *age*, and the second extracts from *cin* a value to be stored in it. This operation makes the program wait for input from *cin*; generally, this means that the program will wait for the user to enter some sequence with the keyboard. In this case, note that the characters introduced using the keyboard are only transmitted to the program when the ENTER (or RETURN) key is pressed. Once the statement with the extraction operation on *cin* is reached, the program will wait for as long as needed until some input is introduced.

The extraction operation on *cin* uses the type of the variable after the >> operator to determine how it interprets the characters read from the input; if it is an integer, the format expected is a series of digits, if a string a sequence of characters, etc.

Extractions on *cin* can also be chained to request more than one datum in a single statement:

```
cin >>a>>b;
```

This is equivalent to:

```
cin>>a;
cin>>b;
```

In both cases, the user is expected to introduce two values, one for variable a, and another for variable b. Any kind of space is used to separate two consecutive input operations; this may either be a space, a tab, or a new-line character.

Arithmetic Operators

Programming has the traditional arithmetic operators: addition (+), subtraction (-), multiplication (*), division (/) and modulus (%). Integer division occurs when both the numerator and denominator of a divide operation are integers (or numbers stored in variables defined to be

Scanned by CamScanner

integers). The result is always an integer because the decimal part is truncated or "chopped" from the number. For example:

9/2 will give the answer 4 not 4.5!

For this reason there are two division operations for integer numbers. The modulus operator, (%) used **only with integers,** gives the remainder of a division operation. 9/2 gives 4 while 9 % 2 gives 1 (the remainder of the division). These arithmetic operators will be discussed in details in the next lesson.

Scanned by CamScanner

PRE-LAB WRITING ASSIGNMENT

Fill in the blank

1. C++ program is consisting of two general divisions: _____ and _____.
2. // or /* in C++ indicates the start of a _____.
3. _____ header file, includes the declarations of the basic standard input-output library in C++.
4. The _____ function is the point by where all C++ programs start their execution.
5. The _____ escape character, takes the cursor to the next tab stop.
6. _____ is a data type that only holds numbers with no fractional component.
7. A _____ is a value or an identifier whose value cannot be altered in a program.
8. The int data type memory size is _____.

Scanned by CamScanner

Lesson 2 Exercises

Assignment 2.1: Working with the standard output (cout)_1

Step1: Logon to your eLearning system.

Step2: Bring in the *stdinfo.cpp* program from Lab2 folder.

Step3: Fill in the code as mentioned in the comments.

Step4: Compile and run the program using visual C++ IDE.

Step5: What is printed on the screen?

The code of *stdinfo.cpp* is as follows:

```
// This program will print the name, address and telephone number of the programmer

#include<iostream>
#include<conio.h>

using namespace std;

int main(){

    // Fill in this space to print your first and last name
    // Fill in this space to print your address (on new line)
    // Fill in this space to print your telephone number (on new line)

    getch();
}
```

Step5: modify the program so that three blank lines separate the telephone number from the address. Compile and run the program. Record the output.

Scanned by CamScanner

Step6: modify the program so that all information printed at the same line separated with horizontal tab. Compile and run the program. Record the output.

Assignment 2.2: Working with the standard output (cout)_2

Step1: Bring in the *out_prog_err.cpp* program from Lab2 folder.

Step2: Compile the program. Here we have many syntax errors. List the errors you found in the following table.

Line Number	Error Description	Correction

Step3: Correct the errors and recompile and run the program. Record the output.

Scanned by CamScanner

The code of *out_prog_err.cpp* is as follows:

```
#include iostream
#include <conio.h>
using namespace std;

void Main()
{
cout<<"\n\new Programming Language\n"<<endl<<endl;
cout<<"*******\r####\n"
cout<<"Faculty of Engineering \tTechnology"<<"endl"<<"\a"<<'FET';
//cout<<"\nDone";
getch();
return 0;
}
```

Assignment 2.3: Write program: Working with the **cout**

Create a new project named writeprog1. After that, create a new source-code file named writeprog1.cpp. Then write a program that displays the following message.

```
***                    ***
***                    ***
***                    ***
***                    ***
***                    ***
 ***Programming Language***
```


Scanned by CamScanner

Write the code here:

Assignment 2.4: Working with variables, constants and arithmetic operators

Step1: Bring in the *circle.cpp* program from Lab2 folder.

Step2: Complete the program and fill in the code for cout as mentioned in the comments. So that the output will produce the following:

```
The circumference of the circle is 33.912
The area of the circle is 91.5624
```


Scanned by CamScanner

Write the code here:

Step3: Change the data type of circumference from float to int. Compile and run the program. Record the output and explain what happened to get this result.

The code of *circle.cpp* is as follows:

Scanned by CamScanner

```
// This program will output the circumference and area of the circle with a given radius.

#include <iostream>
#include<conio.h>

using namespace std;

const  double PI = 3.14;
const  double RADIUS = 5.4;

int main()
{
    _____ area; // definition of area of circle
    float  circumference; // definition of circumference
    circumference = 2 * PI * RADIUS; // computes circumference
    area = _____;       // computes area
    // Fill in the code for the cout statement that will output (with description) the circumference
    // Fill in the code for the cout statement that will output (with description) the area of the circle

    getch();
}
```

Assignment 2.5: Write program: Working with variables, standard input (cin) and arithmetic operators

Step1: Create a new project named writeprog2. After that, create a new source-code file named writeprog2.cpp.

Step2: Write a program that prints the sum, difference and product of two integers. Initialize the integers with the values 14 and 8.

Write the code here:

Scanned by CamScanner

Step3: Modify the previous code so that the values of the two integer variables are inputs interactively (from keyboard).

Write the code here:

Scanned by CamScanner

LESSON 3

Basic Elements of C++

Part2

Jamal 12/10

Outline:

- *Overview*
  - ✓ Data Type Conversion
  - ✓ Operators
  - ✓ Precedence of operators
  - ✓ Converting Algebraic Expressions to C++
- *PRE-LAB WRITING ASSIGNMENT*
- *Lesson 3 Exercises*

Scanned by CamScanner

LESSON (3)
Basic Elements of C++: Part 2

Overview

Data Type Conversion

The process in which one pre-defined type of expression is converted into another type is called conversion. There are two types of conversion in C++:

1. Implicit Conversion.
2. Explicit Conversion.

Implicit Conversion:

This conversion happens whenever the compiler expects data of a particular type, but the data is given as a different type, leading to an automatic conversion by the compiler without an explicit indication by the programmer. This automatic conversion is called implicit conversion.

Consider the following:

```
int x = 3.8; // initialize integer variable with floating point 3.8
float y = 9; // initialize floating point variable with integer 9
```

In the first case an expression of type float is given and automatically interpreted as an integer, by truncate the fractional component, and thus we have loss of information. The variable x = 3 after truncation. In the second case, an integer is given and automatically interpreted as a float, y = 9.0 after conversion.

In general, when the right-hand-side of the assignment statement has different type than left-hand-side of the assignment statement, the implicit conversion will be made as the following table:

Scanned by CamScanner

Right / Left	Integer	Floating Point	Character
Integer	-	Truncation	Ascii Code
Floating Point	No Change	-	Ascii Code
Character	Ascii Character	Truncation + Ascii Character	-

Consider the following examples:

```
int var1 = 'A'; // var1 = 65 (65 is the asci code for the character A
float var2 = 'a'; // var2 = 97.0
                  // (97 is the asci code for the character a
char var3 = 66; // var3 = 'B'
                  // ('B' is the asci character for the code 66
char var4 = 98.5; // var4 = 'b'
                  // ('b' is the asci character for the code 98
```

Data type can be mixed in the expression. For example

```
double a;
int b = 6;
float c =2.5;
a = b * c;
```

When two operands of different type are encountered in the same expression, the lower type variable is converted to the higher type variable. The order of data types are:

double (highest) >> float >> long >> int >> char (lowest).

In the previous example, the int value of b is converted to type float and stored in a temporary variable before being multiplied by the float variable c. the result is then converted to double so that it can be assigned to the double variable a.

Scanned by CamScanner

Explicit Conversion:

Type conversions can be made explicit (by the programmer). It is also called type casting. It temporarily changes a variable data type from its declared data type to a new one. It may be noted here that type casting can only be done on the right-hand-side the assignment statement. The syntax for type casting is: (type) expression

Consider the following example:

```
int num1 = 65 ;
char ch1, ch2;
ch1 = (char) num1; // Explicit conversion (casting)
                   // from integer to character
ch2 = num1 + 1; // Implicit conversion
cout << ch1 << "\t" << ch2 << "\t" << num1 << "\t"
cout << (char) num1 ;

/* OUTPUT: A       B       65      A */
```

In this example, the statement ch1 = (char) num1 cast an integer variable num1 having the value 65 to a character variable ch1, and in the last statement cout << (char) num1, the integer variable num1 is first converted to character and then printed on the screen.

Operators

Operators are special symbols used for specific purpose, which tells the compiler to perform specific mathematical or logical manipulations. C++ is rich in built-in operators and provides the following types of operators: Assignment operator, Arithmetical operators, Compound assignment operators, Increment/Decrement operators, Relational and equality operators, Logical operators, and Conditional ternary operator.

Scanned by CamScanner

Assignment operator

Operates '=' is used for assignment, it takes the right-hand-side and copy it into the left-hand-side as discussed in lesson2.

Arithmetic operators

There are operators used to perform basic mathematical operations. Addition (+), subtraction (-), diversion (/), multiplication (*) and modulus (%) are the basic mathematical operators. The addition, subtraction, and multiplication operators work just like they do in real life, with no caveats. Division and modulus (remainder) need some additional explanation.

It is easiest to think of the division operator as having two different "modes". If both of the operands are integers, the division operator performs integer division. Integer division truncates any fractions and returns an integer value. For example, 7 / 4 = 1 because the fraction is truncated. Note that integer division does not round. If either or both of the operands are floating point values, the division operator performs floating point division. Floating point division returns a floating point value, and the fraction is kept. For example, 7.0 / 3 = 2.333, 7 / 3.0 = 2.333, and 7.0 / 3.0 = 2.333.

We can use cast (type) to convert an integer to a floating point number so that we can do floating point division instead of integer division. Consider the following code:

```
int x = 6, y = 4;
cout<< x/y << "\t";
cout<< (float) x / y << "\t" << x / (float) y
      << "\t" << (float) x / (float) y;
```

This produces the result:

```
1       1.5       1.5       1.5
```


Scanned by CamScanner

The modulus operator is also informally known as the remainder operator. The modulus operator only works on integer operands, and it returns the remainder after doing integer division. For example, 7 / 4 = 1 remainder 3, thus 7 % 4 = 3. As another example, 4 / 7 = 0 remainder 4, thus 4 % 7 = 4. If either of the operands are negative, the modulus result is always as the sign of the first operand. For example, -5 % 3 = -2, 5 % -3 =2, -5 % -3 = -2 and 5 % 3 = 2.

Modulus is very useful for testing whether a number is evenly divisible by another number: if x % y == 0, then we know that y divides evenly into x.

Compound assignment operators

In addition to the simple assignment operator =, C++ provides several compound assignment operators, including: +=, -=, *=, /=, %=

Compound assignments can be used when you want to modify the value of a variable by performing an operation on the value currently stored in that variable. (For example: A = A + 1).

For example, the syntax of using += operator is:

variableName += expression;

The left-side of the assignment operator must be a variable. The compound assignment operators (+=, -=, *=, /=, %=) are always applied *last* in an expression. When an assignment (=) is evaluated, the expression to the right of the operator is always evaluated first, then the value is assigned to the *variable* on the left. When a compound assignment is evaluated, the appropriate operator is applied to the *variable*'s original value and the value to the operator's right, then the resulting value is assigned to the *variable* on the left. Consider the following code:

```
int a = 5, b = 2;
a += 10; // it is equivalent to a =a + 10;        >>> a = 15
a *= b + 2; // it is equivalent to a = a * ( b + 2);      >>> a = 60
```


Scanned by CamScanner

Increment/Decrement operators

There are some operations that occur so frequently in writing assignment statements that C++ has shorthand methods for writing them. One common situation is that of incrementing or decrementing an integer variable. C++ has an increment operator ++ and a decrement operator --. They are equivalent to +=1 and to -=1, respectively. Thus:

```
++i;
i+=1;
i=i+1;
```

Are all equivalent in its functionality; the three of them increase by one the value of i. Notice that, the Increment and decrement operators cannot be applied on a constant.

There are two version of this operator, pre and post. That means that it can be written either (pre) before the variable name (++i) or (post) after it (i++). Although in simple expressions like i++ or ++i, both have exactly the same meaning; in other expressions in which the result of the increment or decrement operation is evaluated, they may have an important difference in their meaning: In the case that the increase operator is used as a prefix (++i) of the value, the expression evaluates to the final value of x, once it is already increased. On the other hand, in case that it is used as a postfix (i++), the value is also increased, but the expression evaluates to the value that x had before being increased. Notice the difference:

Example 1	*Example 2*

`i = 7;`
`j = ++i; // i contains 8,`
`// j contains 8`
	`i = 7;`
	`j = i++; // i contains 8,`
	`// j contains 7`

In Example 1, the value assigned to j is the value of i after being increased. While in Example 2, it is the value i had before being increased.

Scanned by CamScanner

Consider the following example:

```
int x, a, b, c;
a = 2;
b = 4;
c = 5;
x = a-- + b++ - ++c;
cout<<"x: "<<x;
```

The output is

```
x: 0
```

In the statement $x = a-- + b++ - ++c;$ the expression is evaluated as $x = 2 + 4 - 6$, thus, *2* is the original value of the variable a, because in post decrement (a--), first use the value of variable a in the expression then decrement the variable by 1. *4* is the original value of the variable b, because in post increment (b++), first use the value of variable b in the expression then increment the variable by 1. Finally, *6* is the modified value of variable c, because in pre increment (++c), first increase the value of the variable by 1 then use it in the expression.

The value of the variables a, b and c after execution is 1, 5 and 6 respectively.

Relational and equality operators

With the relation and equal operators it is possible to make a comparison between two expressions. The result is a Boolean value that can be true or false. The relational and equality operators are:

- less than (<)
- greater than (>)
- less than or equal to (<=)
- greater than equal to (>=)
- equivalent (equal) (==)
- not equivalent (not equal) (!=).

Scanned by CamScanner

You must notice that assignment operator is (=) and there is an equality operator, for equivalent (==). These two are different from each other, the assignment operator assigns the value to any variable, whereas equality operator is used to compare values.

Here there are some examples:

```
(3 == 5) // evaluates to false

(9 < 9) // evaluates to false

(2 != 7) // evaluates to true

(8 >= 8) // evaluates to true

(7 > 5)  // evaluates to true
```

Logical operators

The logical operators are AND (&&), OR (||) and NOT (!). The AND and OR operators are used to combine two different expressions together.

If two statement are connected using AND operator, the validity of both statements will be considered, but if they are connected using OR operator, then either one of them must be valid. The NOT operator has only one operand, to its right, and inverts it, producing false if its operand is true, and true if its operand is false. Basically, it returns the opposite Boolean value of evaluating its operand. For example: *!false* **is evaluated to true**, and the expression !(4 > 2) is evaluated to false.

Logical operators are mainly used to control program flow. Usually, you will find them as part of an if, while, or some other control statement.

Logical operands allow a program to make decisions based on multiple conditions. Each operand is considered a condition that can be evaluated to a true or false value. Then the value of the conditions is used to determine the overall value of the statement.

Scanned by CamScanner

Conditional ternary operator (?:)

Will be discussed in the next lesson.

Precedence of operators

A single expression may have multiple operators. For example:

```
x = y + 2 / 6;
```

In C++, the above expression always assigns y to variable x, because the / operator has a higher precedence than the + operator, and is always evaluated before. The result of (*2 / 6*) is equal 0, then the expression is *x = y + 0;*

Parts of the expressions can be enclosed in parenthesis to override this precedence order, or to make explicitly clear the intended effect. Notice the difference:

```
x = y + (2 / 6); // x = y (same as without parenthesis)
x = (y + 2) / 6; // assume y = 10, then x = 2
```

The following table shows the order of precedence.

Order	Operators
1	()
2	++, -- (post increment/decrement)
3	++, -- (pre increment/decrement), !, -(unary), +(unary), (type)
4	*, /, %
5	+, -
6	<, <=, >, >=
7	==, !=
8	&&
9	||
10	=, +=, -=, *=, /=, %=

Scanned by CamScanner

Try the following example to understand operators precedence concept available in C++.

Check the simple difference with and without parenthesis. This will produce different results because (), /, * and + have different precedence. Higher precedence operators will be evaluated first:

```
#include <iostream>
#include <conio.h>

using namespace std;

int main() {
   int a = 20;
   int b = 10;
   int c = 15;
   int d = 5;
   int e;

   e = (a + b) * c / d;       // ( 30 * 15 ) / 5
   cout << "Value of (a + b) * c / d is :" << e << endl ;

   e = ((a + b) * c) / d;     // (30 * 15 ) / 5
   cout << "Value of ((a + b) * c) / d is  :" << e << endl ;

   e = (a + b) * (c / d);    // (30) * (15/5)
   cout << "Value of (a + b) * (c / d) is  :" << e << endl ;

   e = a + (b * c) / d;      //  20 + (150/5)
   cout << "Value of a + (b * c) / d is  :" << e << endl ;

   getch();
}
```


Scanned by CamScanner

When the above code is compiled and executed, it produces the following result:

```
Value of (a+b) * c / d is  :90
Value of ((a+b) * c )/ d is    :90
Value of (a+b) * (c / d) is    :90
Value of a+(b * c) / d is      :50
```

Converting Algebraic Expressions to C++

One of the challenges of learning a new computer language is the task of changing algebraic expressions to their equivalent computer expressions.

For example, consider the algebraic expression:

4y (3-2) y + 7

How would this expression be implemented in C++?

4 * y * (3 - 2) * y + 7

Other expressions are a bit more challenging. Consider the quadratic formula:

$$\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

We need to know how C++ handles functions such as the square root and squaring functions. There are several predefined math library routines that are contained in the math library (header file), which will be discussed later.

Scanned by CamScanner

PRE-LAB WRITING ASSIGNMENT

Fill in the blank

1. _____ conversion happens whenever the compiler expects data of a particular type, but the data is given as a different type, leading to an automatic conversion by the compiler.
2. Assume the variable x is defined of type int and initialized with value -20.6. what it is the stored in the variable x _____.
3. _____ it is a conversion that can be made by the programmer and it is also called type casting.
4. Assume the variable ch declared as char ch = 'A'; the output of the statement cout<< (int) ch; is _____
5. What is the final value of the following expression? $(5-16/2*3+(3+2/2)-5)$ _____
6. How would the following expressions be written in C++?

   A.

   $z = 2x + y^2$

   _____

   B.

   $w = \frac{4}{3x} 3.1416r^3$

   _____

Scanned by CamScanner

Lesson 3 Exercises

Assignment 3.1: Simple calculation

Step1: Logon to your eLearning system.

Step2: Bring in the *calc.cpp* program from Lab3 folder.

Step3: Fill in the code as mentioned in the comments.

Step4: Compile and run the program using visual C++ IDE. Input 22 for the number of items bought and 10.98 for the price of each item. Record the output:

```
int kitkat
float price;
float total ;
kitkat = 22;
price = 10.98;
total = kitkat * price;
cout << total;
```

The code of *calc.cpp* is as follows:

```
// This program will read in the quantity of a particular item and its price.
// It will then print out the total price.
// The input will come from the keyboard and the output will go to the screen.


#include <iostream>
#include <conio.h>
using namespace std;

int main()
{

int   quantity; // contains the amount of items purchased
float itemPrice; // contains the price of each item
float totalBill; // contains the total bill.

cout << "Please input the number of items bought" << endl;

// Fill in the input statement to bring in the quantity.

// Fill in the prompt to ask for the price.

// Fill in the input statement to bring in the price of each item.

// Fill in the assignment statement to determine the total bill.

// Fill in the output statement to print total bill, with a label to the screen.

getch();

}
```


Scanned by CamScanner

Assignment 3.2: Working with type casting

Step1: Bring in the *cast_test.cpp* program from Lab3 folder.

Step2: Compile and run the program. Input 90 for mark1 and 95 for mark2. Record the output.

```
~~int float stdAvg~~
90
95
The Studant avg is 92
```

Step3: there is a logical error in this program centering around data types. Modify the code to obtain the correct result. The correct result should be 92.5, if the inputs are 90 and 95.

Write the correct code here:

```
float stdAvg;
float m1, m2;
   /
   (
```


Scanned by CamScanner

The code of *cast_test.cpp* is as follows:

```
// This program will determine the student average of a two marks.

#include <iostream>
#include <conio.h>
using namespace std;

int main() {
    int stdAvg;
    int m1, m2;

    cout<<"Enter your first mark: ";
    cin>>m1;
    cout<<"Enter your second mark: ";
    cin>>m2;

    stdAvg = (m1+m2) / 2;    // compute the student average
    cout << "The student average is " << stdAvg << endl;    // output the result

    getch();
}
```

Assignment 3.3: Working with type casting and operators precedence

What is the output for the following codes?

The Code	The Output
int a=98, b=2; cout<<(char)a<<"\t"<<a%3*3;	b----6

double x = 2;
x = 10 / 4.0 + 9 * x;
cout<<x;
	20.5

int x = 10; cout<< (double) x % 10;	Error

double p = 3.2, x = 7.9;
p *= x++;
cout<<p<<' '<<x;
	3.2 8.9

int a, b, c;
a = 5;
b = 3;
c = ++a * b--;
cout<<a<<' '<<b<<' '<<c;
	6 2 1 8

Scanned by CamScanner

The Code	The Output

int x = 5;
float y = x + -11 / 2 + (x -= 3) + x;
cout<<y;
	1

double z = 3.4;
int a = 0.8;
float b;
b = z * a + z + 5 / (a + 10);
cout<< int (b);
	3

Assignment 3.4: Write Program: type casting

Step1: Create a new project named cast2. After that, create a new source-code file named cast2.cpp.

Step2: Write a program which accepts a character from keyboard and display its ASCII value at the screen.

Write the code here:

```
{char ascii;
int numeric;
cout<<"Give character:";
cin>>ascii;
cout<<"Its ascii value is :" <<(int) ascii<<endl;
cout <<"Give a num to convert to ascii: ";
cin >> numeric;
cout <<"The ascii value of "<<numeric <<" is"<<(char)
}
```


Scanned by CamScanner

Assignment 3.5: Write Program: Arithmetic Operators

Step1: Create a new project named split. After you've created the project, create a new source-code file named split.cpp.

Step2: Write a program which read an integer number of a fixed length (3 digits) from the standard input then print its digits separated by spaces. For example if 123 is the input then the output should be 1 2 3

Hint: Use division and modulus operators.

Write the code here:

Scanned by CamScanner

LESSON 4

Dawar 19/10

Control Structure:

Conditional Statements

Outline:

- *Overview*
  - ✓ *Simple if statement*
  - ✓ *The if..else statement*
  - ✓ *Conditional (ternary) Operator (?:)*
  - ✓ *Nested if Statement*
  - ✓ *The switch Statement*
- *PRE-LAB WRITING ASSIGNMENT*
- *Lesson 4 Exercises*

Scanned by CamScanner

LESSON (4)
Control Structure: Conditional Statements

Overview

Simple if statement

We need to control the flow of execution of a program; in particular, to decide if a certain instruction or block of instructions should be executed, depending on a given condition. The most fundamental statement related to these types of situations is the if statement. The syntax for the if statement is as follows:

```
if (condition)
   statement;
```

Here, condition is the expression that is being evaluated. The if statement evaluates the condition inside parenthesis. If the condition is evaluated to true, statements inside the body of *IF* is executed. If the condition is evaluated to false, statements inside the body of *IF* is skipped.

Condition is true

```
int x = 5;
if (x < 10){
//Code if condition is true
}
//Codes after if
```

Condition is false

```
int x = 5;
if (x > 10){
//Code if condition is true
}
//Codes after if
```

The figure 4-1 illustrates the flowchart for simple if statement.

Scanned by CamScanner

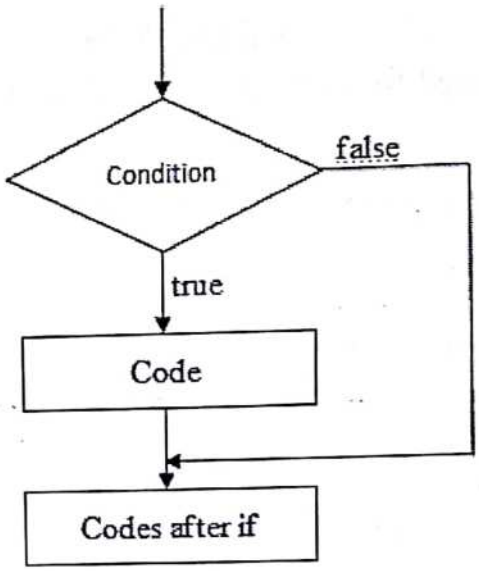


Figure 4-1: Simple if statement Flowchart.

Here is a simple example that shows the syntax:

```
If ( 5< 10)
cout<< "Five is less than ten, that's a big surprise! ";
```

This works as follows: the value 5 is compared against the value 10. If it is less than 10, then the cout statement is executed. If it is not less than 10, then the cout statement is skipped.

To have more than one statement execute after an if statement that evaluates to true, use braces, like we did with the body of a function. Anything inside braces is called a compound statement, or a block.

```
if (condition){
   statement(s);
}
```

For example:

Scanned by CamScanner

```
#include<iostream>
using namespace std;
int main() {
    int number;
    cout<<"Enter an integer: ";
    cin>>number;
    //checks if the number is positive
    if (number>0){
       cout<<"You entered a positive integer: "<<number<<endl;
       number++;
       cout<<"The number after modification is: "<<number<<endl;
       }
    cout<<"This statement is always executed. ";
}
```

Output 1:

```
Enter an integer: 8
You entered a positive integer: 8
The number after modification is: 9
This statement is always executed.
```

Output 2:

```
Enter an integer: -5
This statement is always executed.
```

Important: the if statement must NOT have a semicolon at the end of the line (i.e., after the closing bracket for the condition). This is one of the common mistakes that can give you a really hard time before identifying it and fixing it!

This is how it works. If you code the following:

```
If (number>0);
{
```


Scanned by CamScanner

```
cout<<"You entered a positive integer: "<<number<<endl;
}
```

The compiler sees that there is a statement after the closing bracket for the condition. It is a special case of statement: a null statement; one that has no effect. But it is a statement that finishes at the semicolon. Therefore, the entire block that follows is simply a normal block of code that executes after the if statement - it executes unconditionally and independently of the *if* statement.

The if..else statement

The if else executes the codes inside the body of if statement if the condition is true and skips the codes inside the body of else.

If the condition is false, it executes the codes inside the body of else statement and skips the codes inside the body of if.

Condition is true

```
int x = 5;
if (x < 10)
{
//Code if condition is true
}
else
{
//Code if condition is false
}
//Codes after if else
```

Condition is false

```
int x = 5;
if (x > 10)
{
//Code if condition is true
}
else
{
//Code if condition is false
}
//Codes after if else
```

The figure 4-2 illustrates the flowchart for if..else statement.

Scanned by CamScanner

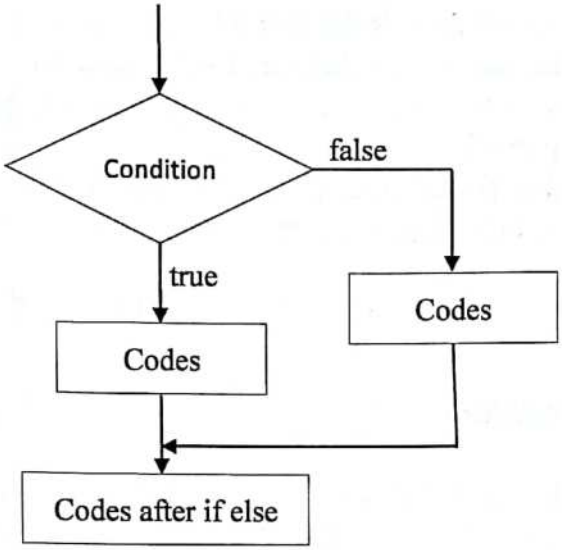


Figure 4-2: if..else statement Flowchart.

For example:

```
#include<iostream>
using namespace std;
int main() {
    int number;
    cout<<"Enter an integer: ";
    cin>>number;
    //checks if the number is positive or negative
    if (number>=0)
       cout<<"You entered a positive integer: "<<number<<endl;
    else
       cout<<"You entered a negative integer: "<<number<<endl;
    cout<<"This statement is always executed. ";
}
```

Output 1:

```
Enter an integer: 8
You entered a positive integer: 8
This statement is always executed.
```


Scanned by CamScanner

Output 2:

```
Enter an integer: -5
You entered a negative integer: -5
This statement is always executed.
```

Composite Conditions

Very often, we encounter situations where the condition cannot be expressed as the simple conditions, just by comparing two values. An example of such situations is testing if a number is within a given range of values; for instance, testing if a number is between 0 and 10.

This example involves testing two conditions, and verifying if the two conditions are simultaneously met. In other words, we want to test if the number is greater than or equal to 0 and also less than or equal to 10.

To do this, we use logical operators to combine two conditions. In the above example, we use the logical AND operator — &&, as shown below:

```
if (0<= number && number <= 10)
```

Notice that, even though the first part may look a little strange, there is nothing wrong with it — each operand around a comparison operator may be an arbitrary expression, so there is nothing that prevents us from writing a numeric constant on the left and a variable on the right. I wrote this condition this way to make it look like the equivalent mathematical notation, $0 \leq n \leq 10$.

However, notice that we cannot write *(0 <= number <= 10)*, because the comparison operators (in this case, <=) require two compatible expressions (in this case, two expressions that evaluate to a numeric value) on each side. If we tried to write the expression in this compact form, the second operator <= would not have a numeric expression on its left, but a Boolean expression, and we would be asking the program to compare if a true/false value is less than or equal than 10.

Scanned by CamScanner

The rule is simple: the logical AND operator requires two expressions that are valid conditions on each side. If we write *number >= 0 && <= 10*, the expression on the right of && is not a valid condition.

The same applies to the logical OR operator, which in C++ is written as two pipe characters, as shown in the example below:

```
if (answer == 'y' || answer == 'Y')
```

In this example, we use a composite condition to check, in a case-insensitive way, if the variable answer contains a y.

The NOT operator can also be used for arbitrary logical expressions; it is not restricted to be used with boolean variables. For example, the following if statement is perfectly valid:

```
if (! (answer == 'q' || answer == 'Q'))
```

The above statement checks if the user not select Q or q to Quit the application.

Short-circuit Evaluation

In C++, the logical operators && and || have an interesting property: short-circuit evaluation. The language rules require that when evaluating a composite condition, if the result from the first part (the expression on the left of the logical operator) is sufficient to determine the result of the condition, then the second part must not be evaluated.

This is a stronger requirement than simply an optimization issue; when evaluating the condition such as:

```
0<= number && number <= 10
```

it seems a reasonable optimization that if the number is negative, then as soon as the first part of the condition is evaluated to false, we do not need to evaluate the second part to know what is the final result — if the first part of the condition evaluates to false, then the requirement that both be true will also evaluate to false, and so, we do not need to make the processor waste time in evaluating something that will not make a difference in the result. The same applies to the logical OR operator (||) when the first part of the expression evaluates to true; at this point, the

Scanned by CamScanner

result of the whole condition is known with absolute certainty, regardless of the second part of the condition.

Conditional (ternary) Operator (?:)

The conditional operator (? :) is a ternary operator (it takes three operands). The conditional operator works as follows:

```
expression1? expression2: expression3;
```

The first operand is implicitly converted to bool. It is evaluated and all side effects are completed before continuing. If the first operand evaluates to true (1), the second operand is evaluated. Otherwise, if the first operand evaluates to false (0), the third operand is evaluated.

The result of the conditional operator is the result of whichever operand is evaluated — the second or the third. Only one of the last two operands is evaluated in a conditional expression.

The conditional operator is basically the same as:

```
if( expression 1)
        expression2;
else
        expression3;
```

Consider the following:

```
7==5? 4: 3   // evaluates to 3, since 7is not equal to 5
7==5+2? 4: 3 // evaluates to 4, since 7is equal to 5+2
5>3? a : b   // evaluates to the value of a,
             // since 5is greater than 3
a>b ? a : b  // evaluates to whichever is greater, a or b.
```

Now take the following example:

```
if(a > b)
        largest = a;
else
        largest = b;
```


Scanned by CamScanner

The above code can be written using (?:) in two ways. The first way is:

```
largest = a > b ? a : b;
```

The second way is

```
a > b ? largest = a : largest = b;
```

Nested if Statement

The if..else statement executes two different codes depending upon whether the test expression is true or false. Sometimes, a choice has to be made from more than 2 possibilities. In C++, the braces of an if or an else clause can contain another if statement. These are known as nested if statements. The nested if..else statement allows you to check for multiple conditions and execute different codes for more than two conditions.

Consider the following program, which check if the integer input is positive, negative or zero:

```
#include<iostream>
using namespace std;
int main() {
    int number;
    cout<<"Enter an integer: ";
    cin>>number;
    if (number > 0)
        cout<<"You entered a positive integer: " << number << endl;
    else if (number < 0)
        cout<<"You entered a negative integer: " << number << endl;
    else
        cout<<"You entered 0. " << endl;
    cout<<"This line is always printed. ";
}
```

Output 1:

```
Enter an integer: 0
You entered 0.
This line is always printed.
```


Scanned by CamScanner

Output 2:

```
Enter an integer: 6
You entered a positive integer: 6
This line is always printed.
```

Output 3:

```
Enter an integer: -7
You entered a negative integer:-7
This line is always printed.
```

The programmer must take care when using nested if statement, because some if statements do not use the optional else as shown in the following program:

```
if (x>y)
  if (c!=0)
      cout<<"x is greater than y and c non-zero";
  else
      cout<<"x is greater than y and c is zero";
```

the else is associated to the nearest if statement (*if (c!=0)*), the first if statement (*if(x>y)*) is a simple if (if without else). This association of else can be altered by using braces to force different association as in below:

```
if (x>y) {
  if (c!=0)
      cout<<"x is greater than y and c non-zero";
} else
  cout<<"x is greater than y and c is zero";
```


Scanned by CamScanner

In this case, the else is associated with the first if statement (*if(x>y)*), and the second if statement (*if (c!=0)*) is a simple if statement (if without else).

Before tracing the code, the programmer must determine the start and end for each if statement. In the following example, there are two if statements and 2 else statements, the first else is associated with the nearest if statement (the inner if statement), and the second else is associated to outer if statement. The following table illustrates which statement will be executed according to the conditions evaluations.

```
if (Condi on 1)
        if (Condi on2)
                statement(s)A;
        else
                statement(s)B;
else
        statement(s)C;
next_sequential_statement;
```

Condition1	Condition2	Executed statement(s)
True	True	statement(s)A
True	False	statement(s)B
False	Not Evaluated	statement(s)C

The switch Statement

The purpose of the switch statement is to check for a value among a number of possible constant expressions. It is something similar to concatenating if-else statements, but limited to constant expressions. Its most typical syntax is:

```
switch (expression)
{
  case value 1:
    group-of-statements-1;
    [break;] // Optional
```


Scanned by CamScanner

No Duplicate cases

```
  case value2
    group-of-statements-2
    [break;] // Optional
  .
  .
  .
  [default:
    default-group-of-statements;] // Optional
}
```

A typical example that illustrates the use of the switch statement is handling a menu selection. In a text-based interface, a program could present the user with a menu of options (say option 1 to option 3), and prompt the user to select an option. Then, depending on the value that the user entered, the program should execute the corresponding fragment of code, among the five possible choices.

In this case, a solution using a switch statement would look like the following:

```
#include<iostream>
using namespace std;
int main() {
    int choice;
    cin >> choice;
    switch (choice)
      {
        case 1:
          cout << "First item selected!" << endl;
          break;
        case 2:
          cout << "Second item selected!" << endl;
          break;
        case 3:
          cout << "Third item selected!" << endl;
          break;
        default:
          cout << "Invalid selection!" << endl;
          break;
    }
}
```


Scanned by CamScanner

The flow of execution goes as follows: the line switch (expression) causes execution to continue in the case that has the same value as the given expression. From that point, execution continues all the way through the rest of the statement block. For this reason, the break statements are necessary, to restrict the executed code; the break statement causes execution to continue after the body of the switch statement. The default clause is chosen if the value of the expression does not appear in any of the case clauses. Notice that the last block does not require a break statement, since execution will continue after the body of the switch statement anyway.

Some important points about switch statement:

- The expression of switch statement must be of type *integer*, *boolean* or *character* type.
- The default case need not be used at last case. It can be placed at any place.
- The case values need not be in specific order.
- The duplicate case values not allowed.
- The case value can be literal or constant of type integer, boolean or character type.
- Not every case needs to contain a break. If no break appears, the flow of control will fall through to subsequent cases until a break is reached.

Consider the following example:

```
#include<iostream>
using namespace std;
int main() {
    char grade = 'D';
    switch(grade){
        case 'A' :    cout << "Excellent!" << endl;      break;
        case 'B' :
        case 'C' :    cout << "Well done" << endl;     break;
        case 'D' :    cout << "You passed" << endl;     break;
        case 'F' :    cout << "Better try again" << endl;     break;
        default :     cout << "Invalid grade" << endl;
    }
    cout << "Your grade is " << grade << endl;
}
```


Scanned by CamScanner

Output:

```
You passed
Your grade is D
```

The equivalent code for the previous example (using nested if instead of switch) is:

```
#include<iostream>
using namespace std;
int main() {
    char grade = 'D';
    if (grade == 'A')
        cout << "Excellent!" << endl;
    else if (grade == 'B' || grade == 'C' )
        cout << "Well done" << endl;
    else if (grade == 'D' )
        cout << "You passed" << endl;
    else if (grade == 'F' )
        cout << "Better try again" << endl;
    else
        cout << "Invalid grade" << endl;
    cout << "Your grade is " << grade << endl;
}
```


Scanned by CamScanner

PRE-LAB WRITING ASSIGNMENT

Fill in the blank

1. The two possible values for a relational expression are _____ and _____.

2. C++ uses the _____ symbol to represent the AND operator.

3. In simple if statement, If the condition is evaluated to _____, statements inside the body of if is skipped.

4. Assume the following code:

```
int x = 3;
if ( x < 2);
        cout<<"***";
```

what is the output? _____

5. Assume the following code:

```
int x = 3;
if ( x = 2)
        cout<<"***";
```

what is the output? _____

6. Assume the following code:

```
int x = 3;
x>=3 || ++x < 4? cout<<"***" : cout<<"###";
cout<<x;
```

what is the output? _____

7. In a switch statement the _____ branch is followed if none of the case expressions match the given switch expression.

Scanned by CamScanner

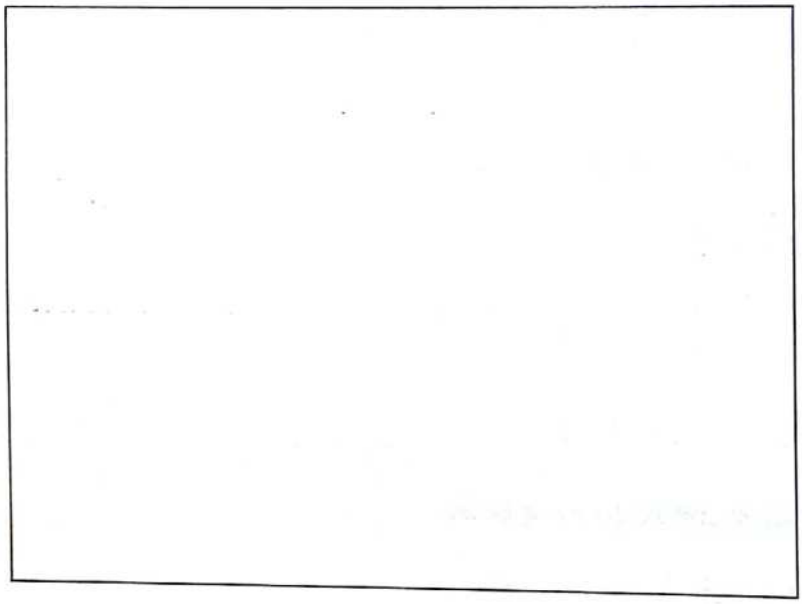
Lesson 4 Exercises

Assignment 4.1: Equality operator and if statement

Step1: Logon to your eLearning system.

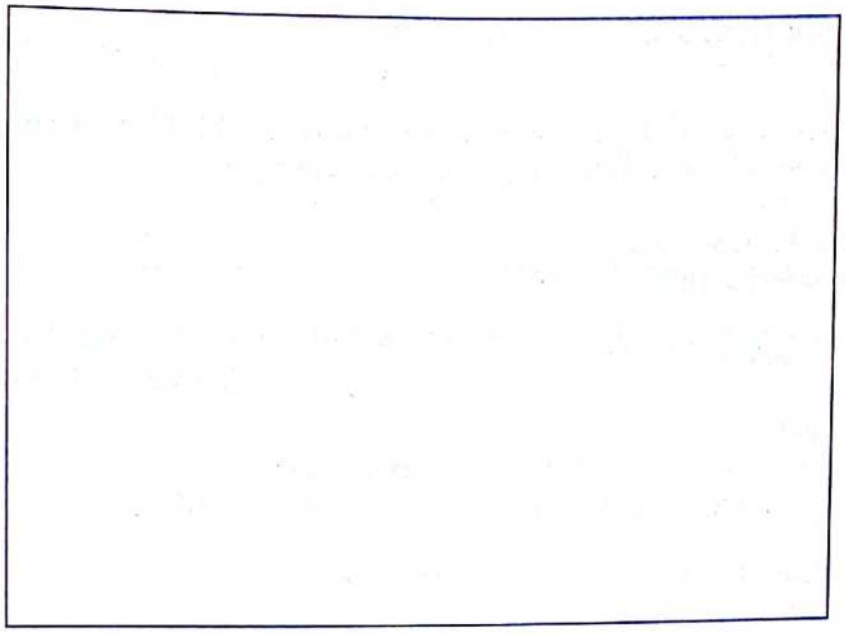
Step2: Bring in the *compare1.cpp* program from Lab4 folder.

Step3: Run the program several times using a different input each time. Does the program do what you expect? Is so, explain what it is doing. If not, locate the error and fix it.

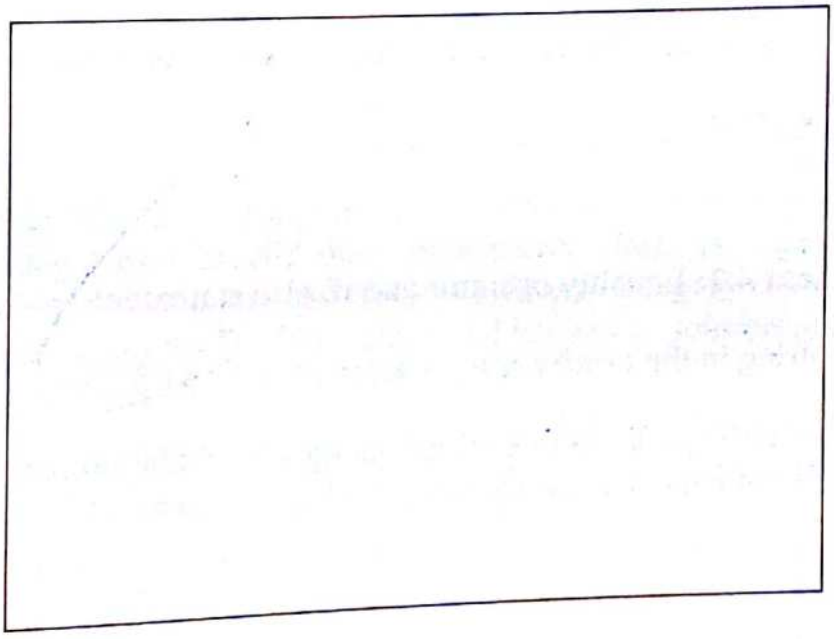


Step4: Modify the program so that the user inputs both values to be tested for equality. Make sure you have a prompt for each input. Test the program with pairs of values that are the same and that are different.

Scanned by CamScanner



Step5: Modify the revised step4 program by replacing the two if statements with a single if/else statement. Run the program again to test the results.



Scanned by CamScanner

The code of *compare1.cpp* is as follows:

```
// This program tests whether or not an initialized value
// is equal to a value input by the user

#include <iostream>
#include<conio.h>

using namespace std;

int main( ) {
    int num1, // num1 is not initialized
        num2 = 5; // num2 has been initialized to 5

    cout << "Please enter an integer: ";
    cin >> num1;
    cout << "num1 = " << num1 << " and num2 = " << num2 << endl;

    if (num1 = num2)
        cout << "The values are the same" << endl;

    if (num1 != num2)
        cout << "The values are not the same" << endl;

    getch();

}
```

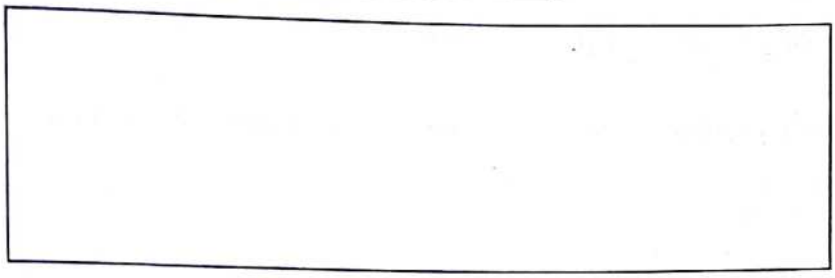
Assignment 4.2: Equality operator and if..else statement

Step1: Bring in the *grades.cpp* program from Lab4 folder.

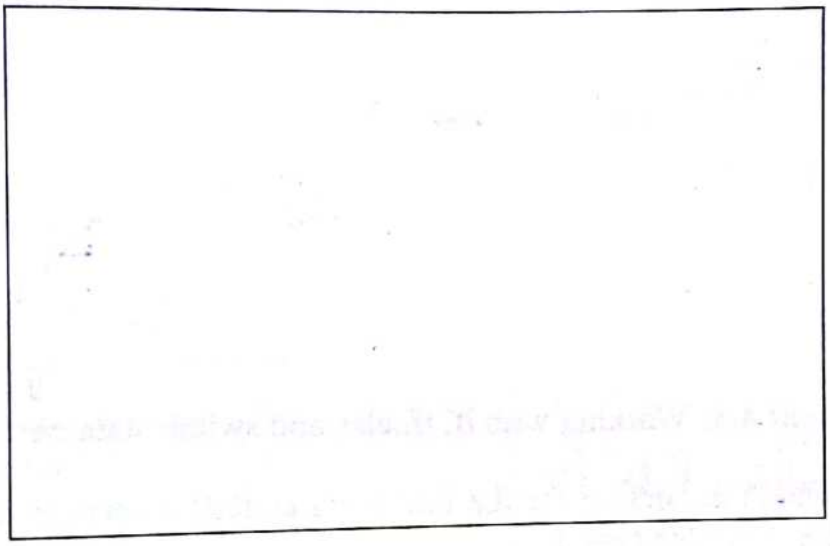
Step2: Run the program three times using 80, 55 and 60 for the average. What happens when you input 60 as the average?

Step3: Modify the first if statement so that the program will also print "You Pass" if the average equals 60.

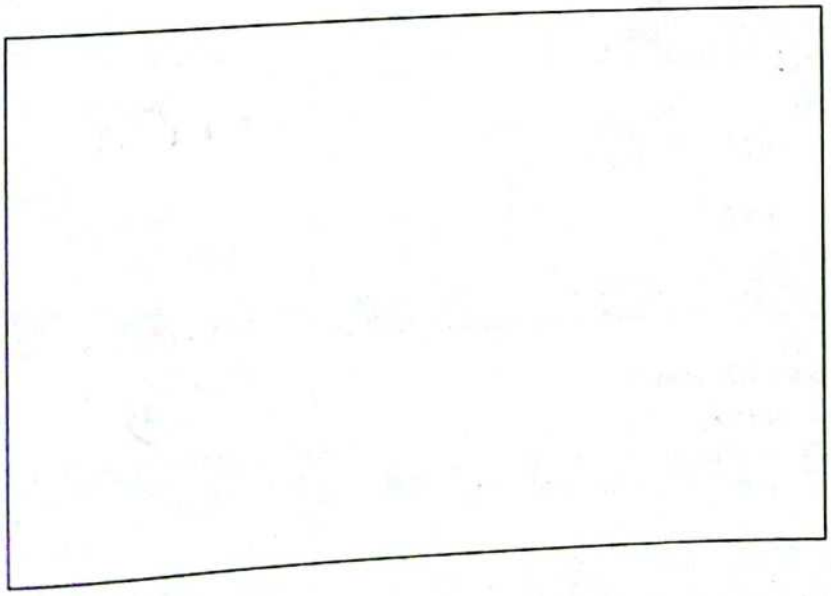
Scanned by CamScanner



Step4: Modify the program so that it uses an if/else statement rather than two if statements.



Step5: Modify the program from step4 to allow the following categories: Invalid data (data above 100), 'A' category (90–100), 'B' category (80–89), "You Pass" category (60–79), "You Fail" category (0–59). What will happen to your program if you enter a negative value such as -12?



Scanned by CamScanner

The code of *grades.cpp* is as follows:

```
// This program prints "You Pass" if a student's average is 60 or higher and prints "You Fail" otherwise

#include <iostream>
#include<conio.h>

using namespace std;

int main() {
float average;    // holds the grade average

cout << "Input your average:" << endl;
cin >> average;

if (average > 60)
    cout << "You Pass" << endl;

if (average < 60)
    cout << "You Fail" << endl;

getch();
}
```

Assignment 4.3: Working with if, if..else and switch statements

What is the output for the following codes? If there is error in the code write down the reason

The Code	The Output

int x= 20;
if(!x)
x=10;
cout<<x;
	20

int x=20;
if(x>4);
x++;
else
x-=7;
cout<<x;
	error

int x=3;
if (x >= 0 && x<5)
x += 2;
else if (x==5)
x =7;
cout<<x;
	5

Scanned by CamScanner

int a = 5, b = 7, c=9;
if ((c = a + b) != 35)
cout << "The value of c is: " << c;
else
cout << "The value of c is: " << c;
	12

int n=8;
if(n>7)
cout<<n/2;
else
cout<< n*3;
cout<< n+2;
	4 10

int x=5, y=7;
if(x < y++ || y++ <=10)
cout<<x<<" "<<y<<endl;
else
cout<<"It is False"<<endl;
	5 8

int x = 0;
int y = 3;
if (x < 1) {
cout << "stmt 1 ";
x = y;
}
if (x > 1) {
cout << "stmt 2 ";
} else
cout << "stmt 3 ";
 	
cout << "stmt 4 ";
	stmt 1 stmt 2 stmt 4

int x=4,val=5,w=6;
int y=5;
if(x%7==4)
if(y%7==5)
x+=x%7;
val=val*2;
else
val=val*3;
w=val*x;
cout<<x<<"\t"<<val<<"\t"<<w;
	error

int x=2,y=10;
x+=(y++>10?'A':'B');
cout<<x;
	6 0

Scanned by CamScanner

int z, x = 11, y = 221;
if(y)
z = y < x ? x/y : y/x;
else
z = 0.0;
cout << "z = " << z << endl;
	z = 20

int a=6;
int b=7;
switch(a/--b) {
case 0:
cout<<"good"; break;
case 1:
cout<<"bad result"; break;
case 2:
cout<<"good result";
break;
}
	bad result

long a=5,b=3,c=0;
switch(a/b-1) {
case -1:
c+=3;
break;
case 0:
a-=3;
case 1:
c+=2;
break;
default:
c+=a;
}
cout<<a<<' , '<<b<<','<<c;
	2210844831 2

int val=64;
char b;
switch(b=val++) {
case 'A':
cout<<"65";
break;
case 'B':
cout<<val-2;
break;
case 'C':
cout<<66;	

Scanned by CamScanner

break; default: cout<<"val"; }	val

char d=97;
switch (2+d++) {
case 'a': cout<<d;
case 'b': cout<<(d-=3);
case 'c': cout<<d+3;
case 99: cout<<++d; break;
case 100: cout<<d*d; break;
default: cout<<(d+=3);
}
	error

Assignment 4.4: Write program: if..else statement

Step1: Create a new project named letter. After you've created the project, create a new source-code file named letter.cpp.

Step2: Write a program that take a single character from the user, and tells it's a Small Letter or it's a CAPITAL letter.

Write the code here:

```
{ char x;
cin>>x;
if(x<97)
cout<<"it's a capital letter";
else
cout<<"its a small letter";
}
```


Scanned by CamScanner

Assignment 4.5: Working with switch statement

Step1: Bring in the *grade_switch.cpp* program from Lab4 folder.

Step2: Compile and run the program more than one time, each time input different grade. Record the output.

Step3: Remove the break statements from each of the cases. What is the effect on the execution of the program (for the same previous grade inputs)?

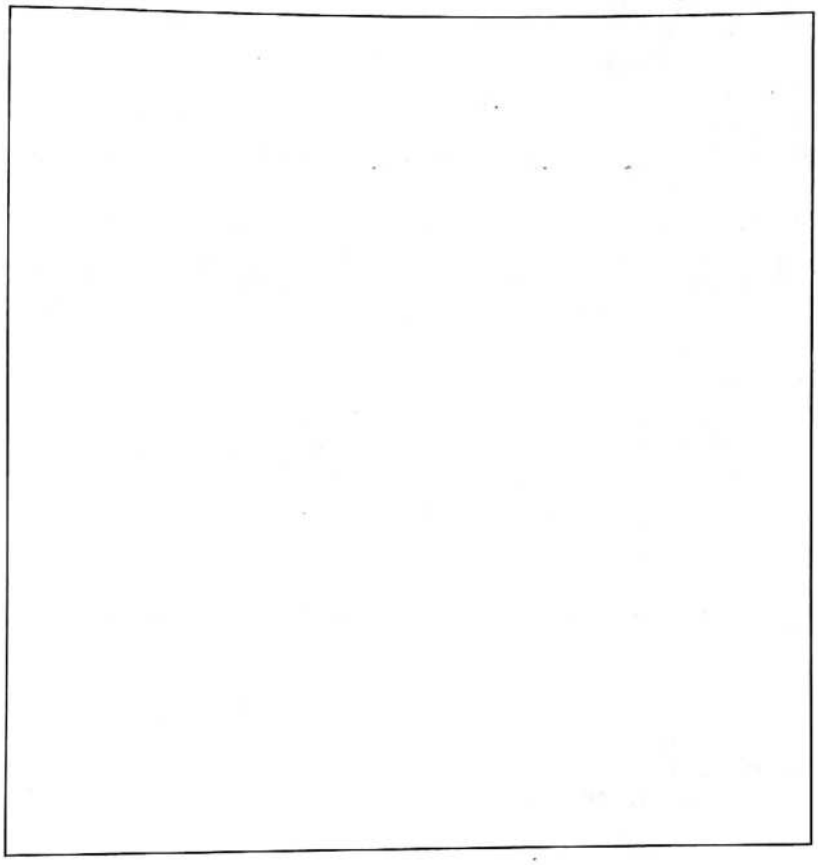
Step4: Add an additional switch statement that allows for a Passing option for a grade of D or better. Use the sample run given below to model your output.

Sample Run:

What grade did you earn in Computer Skills 2 ? A
YOU PASSED!
an A - excellent work!

Scanned by CamScanner

Step5: Rewrite the program *grade_switch.cpp* using if and else if statements rather than a switch statement.



The code of *grade_switch.cpp* is as follows:

Scanned by CamScanner

```
// This program illustrates the use of the switch statement.

#include <iostream>
#include <conio.h>
using namespace std;

int main() {
    char grade;
    cout << "What grade did you earn in Computer Skills 2 ? ";
    cin >> grade;
    switch( grade )              // This is where the switch statement begins
    {
        case 'A':
            cout << "an A - excellent work !" << endl;
            break;
        case 'B':
            cout << "you got a B - good job" << endl;
            break;
        case 'C':
            cout << "earning a C is satisfactory" << endl;
            break;
        case 'D':
            cout << "while  D is passing, there is a problem" << endl;
            break;
        case 'F':
            cout << "you failed - better luck next time" << endl;
            break;
        default:
            cout << "You did not enter an A, B, C, D, or F" << endl;
    }

    getch();

}
```


Scanned by CamScanner

LESSON 5

Control Structure:

Looping_Part1

Outline:

- *Overview*
  - ✓ *Looping*
  - ✓ *while statement*
  - ✓ *do..while statement*
  - ✓ *for statement*
- *PRE-LAB WRITING ASSIGNMENT*
- *Lesson 5Exercises*

Scanned by CamScanner

LESSON (5)
Control Structure: Looping_Part1

Overview

Looping

Conditionals execute certain statements if certain conditions are met; loops execute certain statements while certain conditions are met. It is also called a Repetitive control structure. C++ has three kinds of loops: while, do-while, and for.

while statement

The while loop has a form similar to the *if* conditional:

```
while(condition)
{
        statement 1
        statement2
        ...
}
```

As long as condition holds, the block of statements will be repeatedly executed. If there is only one statement, the curly braces may be omitted.

The flowchart in figure 5-1 illustrates how the while statement is work.

Scanned by CamScanner

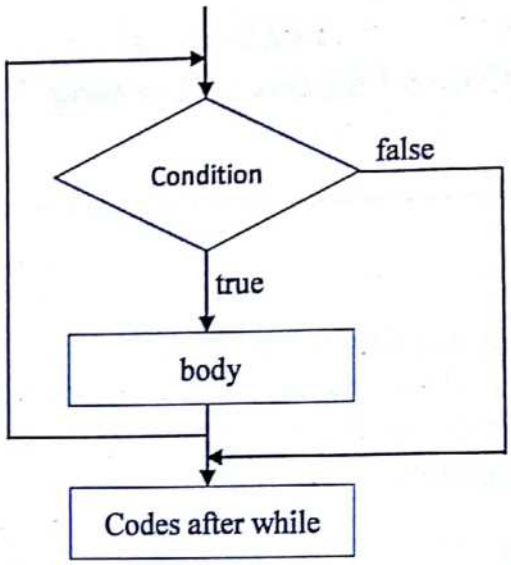


Figure 5-1: while statement Flowchart.

The flowchart indicates that a condition is first evaluated. If the condition is true, the loop body is executed and the condition is re-evaluated. Hence, the loop body is executed repeatedly as long as the condition remains true. As soon as the condition becomes false, it comes out of the loop and goes to the statement next to the 'while' loop (codes after while).

When the programmer wants to execute the statement *cout<<"***";* five times. There are two ways: the first one rewrite the statement 5 times as follows:

```
cout<<"***";
cout<<"***";
cout<<"***";
cout<<"***";
cout<<"***";
```

The second way is by using loop statement such as while statement to repeat the execution of the statement *cout<<"***";* 5 times, as follows:

```
int c = 1;
while (c<=5){
cout<<"***";
c++;
}
```


Scanned by CamScanner

Consider the following example:

```
int x = 0;
while(x < 10)
        x = x + 1;
cout << "x is " << x << "\n"; // output: x is 10
```

This program increment the variable x by 1 until its value greater than or equal to 10 and then prints the result to the screen. Note how the while loop controls the execution. Since x = 0 when the while loop is first encountered, the block of statements in the body of the loop is executed at least once. In fact, the block is executed 11 times because of the increment operator which forces the value of x to increase by one every time the block is executed. During the eleventh iteration of the loop x becomes 10, so the next time the expression is tested x < 10 is false and the loop is exited. Then the cout statement is executed.

If a programmer needs to find the average of five test scores. We must first input and add the five scores. This can be done with a counter-controlled loop as shown in bellow program. Notice how the variable named test works as a counter. Also notice the use of a constant for the number of tests. This is done so that the number of tests can easily be changed if we want a different number of tests to be averaged.

```
#include<iostream>
using namespace std;
const int N_OF_TESTS = 5;
int main(){
   int score ;      // the individual score read in
   float total = 0.0;      //  the total of the scores
   float average;          //  the average of the scores
   int test = 1;           //  counter that controls the loop
   while (test <= N_OF_TESTS) //  Note that test is 1 the first time
   {
      cout << "Enter your score on test " << test << ": " << endl;
      cin >> score;
      total = total + score;
      test++;
      }
   average = total / N_OF_TESTS;
   cout << "Your average is " << average << endl;
}
```


Scanned by CamScanner

What do you think will happen if we eliminated the increment operator test++ in the above code? The value of test is always 1. This means that the expression *test <= N_OF_TESTS* is always true! If we try to execute the modified program, the result is an infinite loop, i.e., a block of code that will repeat forever. One must be very cautious when using loops to ensure that the loop will terminate. Here is another example where the user may have trouble with termination.

```
char letter = 'a';
while (letter != 'x')
{
        cout << "Please enter a letter"  << endl;
        cin >> letter;
        cout << "The letter your entered is " << letter << endl;
}
```

This program requires input from the user during execution. Infinite loops can be avoided, but it would help if the user knew that the 'x' character terminates the execution. Without this knowledge the user could continually enter characters other than 'x' and never realize how to terminate the program.

do..while statement

The while loop is a pre-test or top test loop. Since we test the expression before entering the loop, if the test expression in the while loop is initially false, then no iterations of the loop will be executed. If the programmer wants the loop to be executed at least once, then a post-test or bottom test loop should be used. C++ provides the do-while loop for this purpose. A do-while loop is similar to a while loop except that the statements inside the loop body are executed before the expression is tested. The format for a single statement in the loop body is the following:

```
do
        statement;
while (expression);
```


Scanned by CamScanner

Note that the statement must be executed once even if the expression is false. To see the difference between these two loops consider the code

```
int num1 = 5;
int num2= 7;
while (num2< num 1) {
        num 1= num 1+ 1;
        num2= num2- 1;
}
```

Here the statements *num1 = num1 + 1* and *num2 = num2 - 1* are never executed since the test expression *num2 < num1* is initially false. However, we get a different result using a do-while loop:

```
int num 1= 5;
int num2= 7;
do {
        num 1= num 1+ 1;
        num2= num2- 1;
} while (num2< num 1);
```

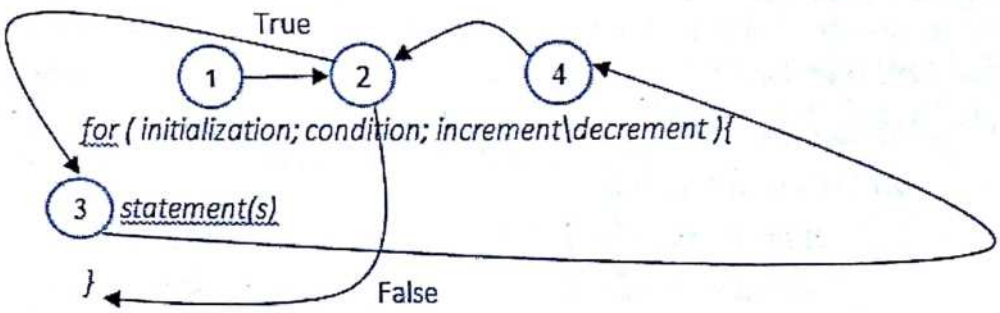
In this code the statements *num1 = num1 + 1* and *num2 = num2 - 1* are executed exactly once. At this point *num1 = 6* and *num2 = 6* so the expression *num2 < num1* is false. Consequently, the program exits the loop and moves to the next section of code. Also note that since we need a block of statements in the loop body, curly braces must be placed around the statements.

for statement

It is a count controlled loop in the sense that the program knows in advance how many times the loop is to be executed.

Scanned by CamScanner

The syntax for the *for* loop is:



Here is the flow of control in a *for* loop:

- The initialization step is executed first, and only once. This step allows you to declare and initialize any loop control variables. You are not required to put a statement here, as long as a semicolon appears.
- Next, the condition is evaluated. If it is true, the body of the loop is executed. If it is false, the body of the loop does not execute and flow of control jumps to the next statement just after the for loop. This statement can be left blank, in this case the condition is evaluated always true.
- After the body of the for loop executes, the flow of control jumps back up to the increment\decrement statement. This statement allows you to update any loop control variables. This statement can be left blank, as long as a semicolon appears after the condition.
- The condition is now evaluated again. If it is true, the loop executes and the process repeats itself (body of loop, then increment\decrement step, and then again condition). After the condition becomes false, the for loop terminates.

The program bellow will print out the values 0 through 9, each on its own line using the for statement.

```
for (int x = 0; x < 10; x = x + 1)
    cout << x << "\n";
```

If the counter variable is already defined, there is no need to define a new one in the initialization portion of the for loop. Therefore, it is valid to have the following:

Scanned by CamScanner

```
int x = 0;
for (; x < 10; x += 1)
   cout << x << "\n";
```

Note that the first semicolon inside the for loop's parentheses is still required.

And also you can omit the increment\decrement part as follows:

```
int x = 0;
for (; x < 10;){
        cout << x << "\n";
        x += 1;
}
```

A loop becomes infinite loop if a condition never becomes false. The for loop is traditionally used for this purpose. Since none of the three expressions that form the for loop are required, you can make an endless loop by leaving the conditional expression empty.

```
for ( ; ; )
        cout<<"This loop will run forever.\n";
```

When the conditional expression is absent, it is assumed to be true. You may have an initialization and increment expression, but C++ programmers more commonly use the *for(;;)* construct to signify an infinite loop.

In the initialization part, you can declare multiple names, as long as they can use the declaration specifier. Also in increment\decrement part, you can write multiple statements as follows:

```
for ( int x = 1, y = 4; x <= y ; x++, y-=2)
        cout<<"C++\n";
```

The output is:

```
C++
C++
```


Scanned by CamScanner

A for loop can be expressed as a while loop and vice-versa. Recalling that a for loop has the form

```
for (initialization; condition; increment\decrement)
{
        statement 1;
        statement2;
        ...
}
```

We can write an equivalent while loop as

```
initialization;
while(condition)
{
        statement 1;
        statement2;
        ...
        Increment\decrement;
}
```

The following code

```
for ( int x = 1, y = 4; x <= y ; x++, y-=2)
        cout<<"C++\n";
```

is converted to

```
int x = 1, y = 4;
while ( x <= y){
        cout<<"C++\n";
        x++;
        y-=2;
}
```


Scanned by CamScanner

PRE-LAB WRITING ASSIGNMENT

Fill in the blank

1. A block of code that repeats forever is called _____.
2. To write out the first 12 positive integers and their cubes, one should use a _____ loop.
3. The _____ loop is a pre-test or top test loop, Instead of _____ loop is a post-test or bottom test loop.
4. The _____ loop it is a count controlled loop in the sense that the program knows in advance how many times the loop is to be executed.
5. When the condition part of the for statement is left blank, in this case the condition is evaluated always _____.
6. Assume the following code:

```
for (int i= 1; ++i<=5; cout<<i<<"\t")
;
```

what is the output? _____

Scanned by CamScanner

Lesson 5 Exercises

Assignment 5.1: Working with the while loop statement_1

Step1: Logon to your eLearning system.

Step2: Bring in the *input_letter.cpp* program from Lab5 folder.

Step3: Run the program several times and record the output.

Step4: This program is not user friendly. Explain why.

Scanned by CamScanner

Step5: Add to the code so that the program is more user friendly.

Step6: How would this code affect the execution of the program if the while loop is replaced by a do-while loop? Try it and see.

The code of *input_letter.cpp* is as follows:

```
#include<iostream>
#include<conio.h>

using namespace std;

int main () {

    char letter = 'a';
    while (letter != 'x') {
        cout << "Please enter a letter" << endl;
        cin >> letter;
        cout << "The letter you entered is " << letter << endl;
    }

getch();
}
```


Scanned by CamScanner

Assignment 5.2: Working with the while loop statement_2

Step1: Bring in the average.cpp program from Lab5 folder.

Step2: Fill in the code as mentioned in the comments.

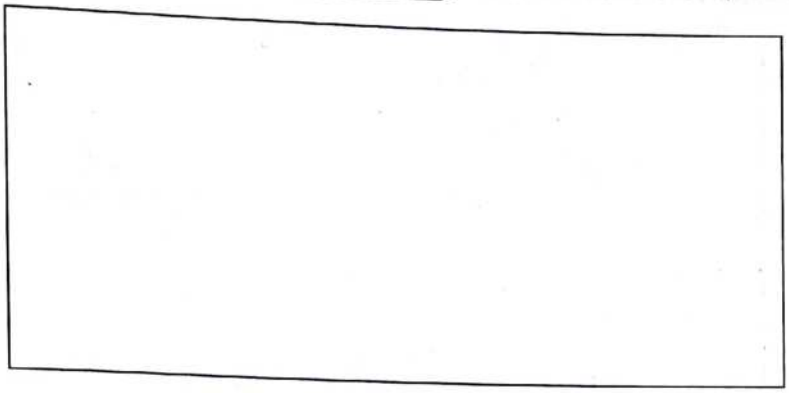
Step3: Compile and run the program several times with various input. Record your results. Are they correct?

Step4: What happens if you enter –1 first?

Step5: What is the purpose of the following code in the program above?

```
if (count == 1)

cout << "No mark has been entered" << endl;
```


Scanned by CamScanner



The code of *average.cpp* is as follows:

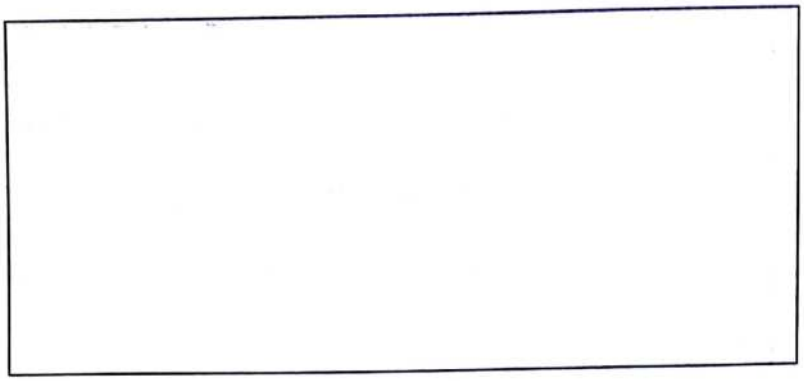
```
// This program is asked the user for enter the marks until a value of -1 is entered.
//Then the average is calculated and displayed.

#include <iostream>
#include<conio.h>
using namespace std;

int main() {

    // Fill in the code to define and initialize to 1 the variable count
    float total = 0, mark, average;

    cout << "Enter the mark " << count << endl;
    cout << "Enter -1 when you are finished" << endl;
                                        cin>>mark ;
    // Fill in the code to read in the value for the first mark
    // Fill in the code to start a while loop that iterates
    // while mark does not equal -1        while(mark!=1)
    {
        // Fill in the code to update total by adding it to mark total=total+mark;
        // Fill in the code to increment count by one count++;
        cout << "Enter the mark " << count << endl;
        cout << "Enter -1 when you are finished" << endl;
        // Fill in the code to read in the value for the mark cin>>mark;
    }
    if (count == 1)
        cout << "No mark has been entered" << endl;
    else{
        // Fill in the code to calculate the average average=total/count;
        cout << "The average for the " << count-1 << " markss is "<< average << endl;
    }

    getch();
}
```

Assignment 5.3: Working with the for loop statement

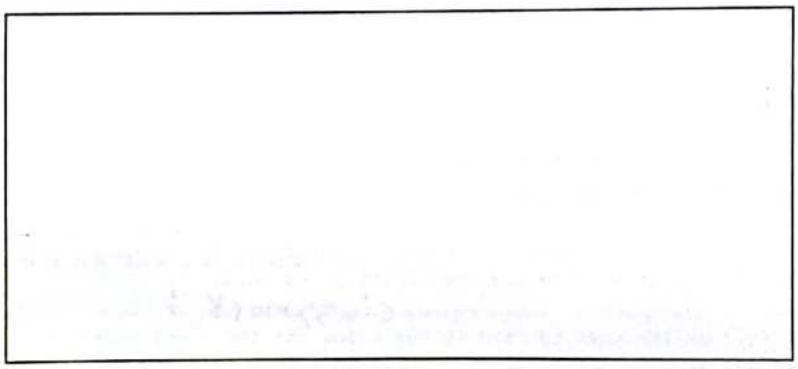
Step1: Bring in the avg_for.cpp program from Lab5 folder.

Step2: Compile and run the program several times with various input. Record your results.

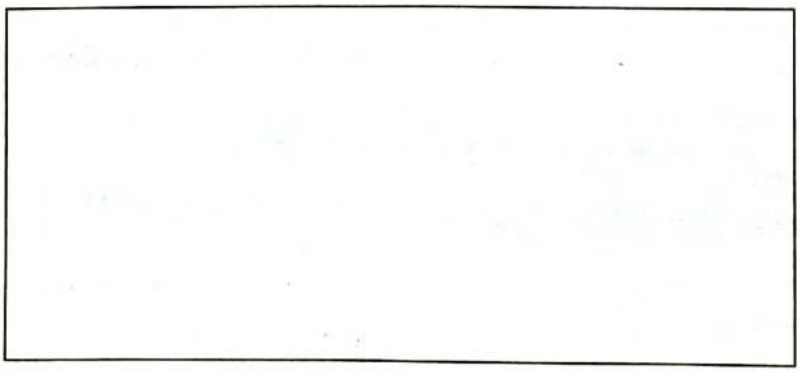
Scanned by CamScanner



Step3: Why is the typecast operator needed to compute the average in the statement *avg = float (total)/value;*? What do you think will happen if it is removed? Modify the code and try it. Make sure that you try both even and odd cases. Record what happens.

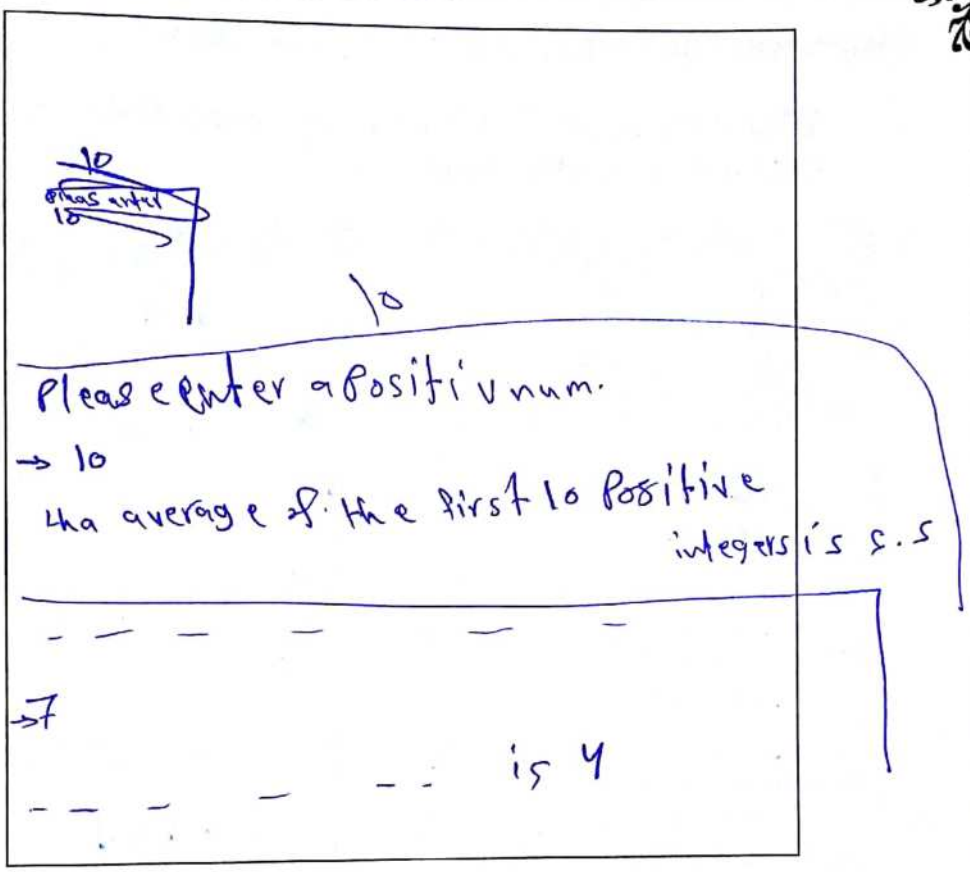


Step4: What happens if you enter a float such as 2.99 instead of an integer for value? Try it and record the results.



Step5: Modify the code so that it computes the average of the consecutive positive integers n, n+1, n+2, . . . , m, where the user chooses n and m. For example, if the user picks 3 and 9, then the program should find the average of 3, 4, 5, 6, 7, 8, and 9, which is 6.

Scanned by CamScanner



The code of *avg_for.cpp* is as follows:

```
// In this program the user input a number n and then finds the average of the first n positive integers

#include <iostream>
#include <conio.h>
using namespace std;

int main() {
    int value;        // value is some positive number n
    int total = 0;    // total holds the sum of the first n positive numbers
    int number;       // the amount of numbers
    float avg;       // the average of the first n positive numbers

    cout << "Please enter a positive integer" << endl;
    cin >> value;
    if (value > 0) {
        for (number = 1; number <= value; number++) {
            total = total + number;
        } // curly braces are optional since there is only one statement

        avg = float(total) / value; // note the use of the typecast operator here
        cout << "The average of the first " << value << " positive integers is " << avg << endl;
        }
    else
        cout << "Invalid input - integer must be positive" << endl;

    getch();
}
```


Scanned by CamScanner

Assignment 5.4: Working with the loop statements

What is the output for the following codes? If there is error in the code write down the reason

The Code	The Output
for (int x=10,x>5;i++) cout<<x;	error

for (int d= 1; d <=3; d+=2)
{
cout<<"*";
d++;
}
	*

for (int c = 8; c>=5 ;) cout<<'H';	H - - - - - ∞
for (char z='a' ; z < 'c' ; z++) cout<<z; cout<<z-1+1<<endl;	a a b b

int x=5;
while (x>=5)
cout<<x;
x++;	

int i, k;
k=10;
for (i=1;i<k; i++)
if (k%i ==0)
cout<< i << ", ";
	1, 2, 5,

int n = 12;
while (n <=15)
if (n%2)
n = n-1;
else
n = n+3;
cout<<n;	

for (int i = 1; i<=5 ; i+=3);
cout<<i-1;
if(i>=7)
cout<<i;
	error

Scanned by CamScanner

```
int n,d,c;
cout<<"Enter an integer: ";
cin>>n; //suppose user enter 815
c=n;
if(n>10){
        while(n>10)
        {
                d=(n%10);
                n=n/10;
                cout<< d;
        }
}
if(c==10)
        cout<<"01"<<endl;
else
        cout<<n<<endl;
```

Assignment 5.5: Write Program_1

Step1: Create a new project named power. After you've created the project, create a new source-code file named power.cpp.

Step2: Write a program that computes the power x^y. To calculate the power x^y where both x and y are integers we must repeat multiply x by itself y times. For example, if x=3 and y=4 so you must compute 3*3*3*3.

Write the code here:

```
#include<iostream
int x,y;
int result=1
for(int i=1;i<=y;i++)
    result*=x
cout<<result;
```

cin>>x
cin>>y

Scanned by CamScanner

Assignment 5.6: Write Program_2

Step1: Create a new project named armstrong. After you've created the project, create a new source-code file named armstrong.cpp.

Step2: Write a program that ask the user to input a number consists of 3 digits, then check if the number is armstrong or not. For example, the number 153 is an armstrong number, because $1^3 + 5^3 + 3^3 = 153$.

Write the code here:

Scanned by CamScanner

LESSON 6

Damat 9/11

Control Structure:

Looping_Part1

Outline:

- *Overview*
  - ✓ *Nested Loops*
  - ✓ *break statement*
  - ✓ *continue statement*
- *PRE-LAB WRITING ASSIGNMENT*
- *Lesson 6 Exercises*

Scanned by CamScanner

LESSON (6)
Control Structure: Looping_Part2

Overview

Nested Loops

A loop inside another loop is called a nested loop. The number of loops depend on the complexity of a problem. Suppose, a loop, outer loop, running *n* number of times consists of another loop inside it, inner loop, running *m* number of times. Then, for each execution of the outer loop from 1...n, the inner loop runs maximum of m times.

There can be many types of nested loops in C++ but the mostly used nested loops are:

- Nested while loop
- Nested do-while loop
- Nested for loop

Note: There can also be very variation of nested loops where a while loop can be inside a for loop, a for loop can be inside a do-while loop.

In the following example we need to print all the combinations of the digits from 0 to 5. i.e 00, 01 and so on until 55.

```
#include<iostream>
using namespace std;
int main(){
    int i, j;
    for(i=0; i<=5; i++){  // Outer loop
       for(j=0; j<=5; j++)  // Inner loop
          cout<<i<<j<<" \t";
       cout <<"\n";
    }
}
```


Scanned by CamScanner

The program output is

```
00  01  02  03  04  05
10  11  12  13  14  15
20  21  22  23  24  25
30  31  32  33  34  35
40  41  42  43  44  45
50  51  52  53  54  55
```

Let's take a look at a trace of two nested loops. In the outer for loop when the value of i becomes 0 then control enters into the inner loop where j will take value from 0 to 5 considering the same value of i = 0. The following table trace the nested loop.

Memory i	Memory j		Screen
0	0		00
0	1		00 01
0	2		00 01 02
0	3		00 01 02 03
0	4		00 01 03 04
0	5	(end inner loop)	00 01 02 03 04 05
1	0		00 01 02 03 04 05 10
....		....	....
1	5	(end inner loop)	00 01 02 03 04 05 10 11 12 13 14 15
....		....	....
5	5	(end inner and outer loop)	00 01 02 03 04 05 10 11 12 13 14 15 20 21 22 23 24 25 30 31 32 33 34 35 40 41 42 43 44 45 50 51 52 53 54 55

Scanned by CamScanner

Another example to understand working of nested loop, is to create a multiplication table in the following form:

```
     0   1   2   3   4   5   6   7   8   9
0 - 0   0   0   0   0   0   0   0   0   0
1 - 0   1   2   3   4   5   6   7   8   9
2 - 0   2   4   6   8   10  12  14  16  18
3 - 0   3   6   9   12  15  18  21  24  27
4 - 0   4   8   12  16  20  24  28  32  36
5 - 0   5   10  15  20  25  30  35  40  45
6 - 0   6   12  18  24  30  36  42  48  54
7 - 0   7   14  21  28  35  42  49  56  63
8 - 0   8   16  24  32  40  48  56  64  72
9 - 0   9   18  27  36  45  54  63  72  81
```

You can see that for row 0, the program has to iterate from column 0 through column 9. The program repeats the process for row 1, again for row 2, and so on right down to row 9. This implies the need for two loops: an inner loop to iterate over the columns and a second outer loop to iterate over the rows.

Each position in the table is simply the row number times the column number.

This is exactly how the following nested loops program works:

Scanned by CamScanner

```
#include<iostream>
using namespace std;
int main(){
    // display the column headings
    int nColumn = 0;
    cout<<endl;
    cout << "    ";
    while (nColumn < 10)
    {
       // display the column number
       cout << nColumn << "\t";
       // increment to the next column
       nColumn++;
    }
    cout << endl;

    // now go loop through the rows
    int nRow = 0;
    while (nRow < 10)
       {
          // start with the row value
          cout << nRow << " - ";
          // now for each row, start with column 0 and
          // go through column 9
          nColumn = 0;
          while(nColumn < 10)
          {
             // display the product of the column*row
             cout << nRow * nColumn << "\t";
             // go to next column
             nColumn++;
          }
          // go to next row
          nRow++;
          cout << endl;
    }
}
```


Scanned by CamScanner

The first section creates the column headings. This section initializes nColumn to 0. It then iterates through nColumn printing out its value separated by a horizontal tab until nColumn reaches 10. At this point, the program exits the first loop and then tacks a new line on the end to finish the row.

The second set of loops, the nested loops, starts at nRow equal to 0. The program prints out the row number followed by a dash before launching into a second loop that starts nColumn at 0 again and iterates it back up to 9. For each pass through this inner loop, the program displays nRow * nColumn followed by a horizontal tab. The program outputs a newline to move output to the next row each time it increments nRow.

In the following program, nested do-while loop is used to print the star pattern. The outer loop runs 5 times and for every loop, the inner loop runs *i* times which is 1 at first, meaning only one "*" is printed, then on the next loop it's 2 printing two stars and so on till 5 iterations of the loop executes, printing five stars.

```
#include<iostream>
using namespace std;
int main(){
    int i=1, j;
    do{
      j=1;
      do{
        cout<<"*";
        j++;
      }while(j<=i);
      i++;
      cout<<endl;
    }while(i<=5);
}
```


Scanned by CamScanner

The output is:

```
*
**
***
****
*****
```

break statement

In C++ break statement is used for the termination of the loop statement and switch case statement. In loops, the break statement leaves a loop, even if the condition for its end is not fulfilled. It can be used to end an infinite loop, or to force it to end before its natural end. For example, let's stop the countdown before its natural end:

```
#include<iostream>
using namespace std;
int main(){
    for (int n=10; n>0; n--)
    {
      cout << n << ", ";
      if (n==3)
      {
        cout << "countdown aborted!";
        break;
      }
    }
}
```

The program output is:

```
10, 9, 8, 7, 6, 5, 4, 3, countdown aborted!
```


Scanned by CamScanner

The following program is another example of using break statement in the loop.

```
#include<iostream>
using namespace std;
int main(){
   int count = 0;
   // do loop execution
   do
   {
      cout << "Count : " << count << endl;
      count++;
      if( count > 5)
      {
         // Terminate the loop
         break;
      }
   }while( count < 20 );
}
```

The program output is:

```
Count : 0
Count : 1
Count : 2
Count : 3
Count : 4
Count : 5
```

In the above example, while condition specified that, loop will iterate 20 times.

while(count < 20)

Now in the loop body we have written the simple condition which tells us to break the loop if the count value becomes greater than 5.

Scanned by CamScanner

```
if( count > 5)
{
 // Terminate the loop
 break;
}
```

If we used the nested loop and used break statement in the inner loop then break will terminate the inner loop only.

```
#include<iostream>
using namespace std;
int main(){
   int count=0;
   do
   {
      for(int i=0; i<5; i++) {
         cout << i;
         if(i == 1)
            break;
      }
      cout << "Count : " << count << endl;
      count++;
   }while( count < 5 );
}
```

In the above example, outer loop will be executed as usual, but the inner loop will be terminated as soon as the value of i becomes 1. The program output is:

```
01Count : 0
01Count : 1
01Count : 2
01Count : 3
01Count : 4
```

Example: the following program uses a nested for loop and break statement to find the prime numbers from 2 to 20:

Scanned by CamScanner

```
#include<conio.h>

#include<iostream>
using namespace std;
int main(){
    int i, j;
   for(i = 2; i<20; i++) {
      for(j = 2; j <= (i/j); j++)
         if(!(i%j)) break; // if factor found, not prime
      if(j > (i/j)) cout << i << " is prime\n";
   }
}
```

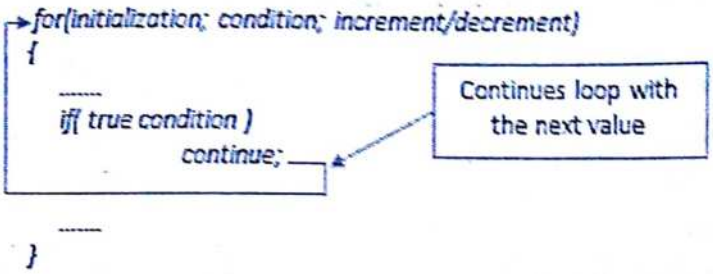
The output is:

```
2 is prime
3 is prime
5 is prime
7 is prime
11 is prime
13 is prime
17 is prime
19 is prime
```

continue statement

The continue statement forces the next iteration of the loop to take place, skipping remaining code in between.

Scanned by CamScanner



- In the case of the for loop as soon as after the execution of continue statement, increment/decrement statement of the loop gets executed. After the execution of increment statement, condition will be checked.
- In case of the while loop and do..while loop, continue statement will take control to the condition statement.

For example, let's skip number 5 in our countdown:

```
#include<iostream>
using namespace std;
int main(){
    for (int n=10; n>0; n--) {
        if (n==5) continue;
        cout << n << ", ";
    }
    cout << "liftoff!\n";
}
```

The output is:

```
10, 9, 8, 7, 6, 4, 3, 2, 1, liftoff!
```


Scanned by CamScanner

PRE-LAB WRITING ASSIGNMENT

Fill in the blank

1. A loop within a loop is called a _____.

2. In a nested loop the _____ loop goes through all of its iterations for each iteration of the _____ loop. (Choose inner or outer for each blank.)

3. In loops, the _____ statement leaves a loop, even if the condition for its end is not fulfilled.

4. If we are using the nested loop and we used break statement in the inner loop then break will terminate the _____ loop only.

5. The _____ statement forces the next iteration of the loop to take place, skipping remaining code in between.

Scanned by CamScanner

Lesson 6 Exercises

Assignment 6.1: working with nested loop

Step1: Logon to your eLearning system.

Step2: Bring in the *shape1.cpp* program from Lab6 folder.

Step3: Compile and run the program. Record the output.

Step4: Note that the outer loop of this program is always executed exactly five times. Modify the code so that the outer loop iterates n times, where n is a positive integer input by the user. In other words, let the user decide how many lines of stars to print

Scanned by CamScanner

Step5: Modify the revised step4 program by replacing the two do..while statements with a a two for statement. Compile and run the program again to test the results

Step6: rewrite the program to print line number instead of star as follows (assume the user input 5 for number of lines)

1
22
333
4444
55555

Scanned by CamScanner

Step6: rewrite the program to print column number instead of line number as follows (assume the user input 5 for number of lines)

1

12

123

1234

12345

```
int i=1, j;
 do {
j=1;
do{
 cout<<j;
  j++;
} while(j<=i);
i++;
cout<<endl; cout<<i<<
}while(i<=5);
```

The code of *shape1.cpp* is as follows:

```
// This program print the following star pattern
/*

*
**
***
****
*****

*/
#include<iostream>
#include<conio.h>
using namespace std;
```


Scanned by CamScanner

```
int main(){

    int i=1,j;
    do{
        j=1;
        do{
            cout<<"*";
            j++;
        }while(j<=i);
        i++;
        cout<<endl;
    }while(i<=5);

    getch();
}
```

Assignment 6.2: Working with nested loop and break/continue statements

What is the output for the following codes?

The Code	The Output

for (int i=0 ;; i+=2) {
if(i==4)
break;
cout<<'*';
}
	*

int n=5;
for(int a=2; a<n; a+=2)
for(char dd='a'; dd<'c'; dd++)
cout<<dd <<a<<"-";
	a2-b2-a4-b4-

for (char ch='a';ch<='m';ch++) {
if(ch % 3 == 0)
continue;
cout<<ch;
}
	q

Scanned by CamScanner

```
int i = 0;
while(i < 3){
        int j = 0;
        while(j < 2){
                cout << "i = " << i << " and
j = " << j << endl;
                j++;
        }
        i++;
}
```

i = 0 and j = 0
i = 0 and j = 1
i = 1 and j = 0
i = 1 and j = 1
i = 2 and j = 0
i = 2 and j = 1

h.w

Assignment 6.3: Write program: nested loop_1

Step1: Create a new project named equation. After you've created the project, create a new source-code file named equation.cpp.

Step2: Write a program that calculate and print the result of the following series.

$$z = \sum_{i=0}^{5} \sum_{j=0}^{10} i + 2j$$

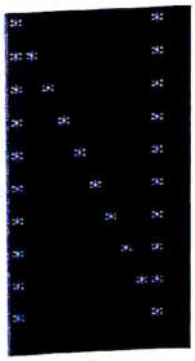
Scanned by CamScanner

Write the code here:

Assignment 6.4: Write program: nested loop_2

Step1: Create a new project named shape2. After you've created the project, create a new source-code file named shape2.cpp.

Step2: Write a program that print the following star pattern:



Scanned by CamScanner

Write the code here:

Scanned by CamScanner

LESSON 7

Functions_Part1

Outline:

- *Overview*
  - ✓ *Introduction*
  - ✓ *Predefined (Built-in) Functions*
  - ✓ *cmath Library*
- *PRE-LAB WRITING ASSIGNMENT*
- *Lesson 7Exercises*

Scanned by CamScanner

LESSON (7)
Functions_Part1

Overview

Introduction

A function is a block of code that performs some operation. Every C++ program has at least one function, which is main(), and all the most trivial programs can define additional functions. In other words, a function is just a separate part of a code that can be used at (almost) any time to evaluate something, to do something, or both. We identify a function by its name. The name of a function is subject to the same restrictions as a variable's name. Additionally, we can't have a variable and a function of the same name.

A function can optionally define input parameters that enable callers to pass arguments into the function. A function can optionally return a value as output.

In general, we can divide functions into two groups:

- Predefined functions: functions written by someone else (not you) which are made available by the environment, sometimes called Built-in functions or library functions.
- User-defined functions: functions written (created) by programmer.

Predefined (Built-in) Functions

Predefined functions are functions that are built into C++ Language to perform some standard operations. Programmer can use library function by invoking function directly; they don't need to write it themselves.

The functions that are stand-alone used for general purposes and that are not depended on any classes are stored in the Standard Function Library. The C++ Standard Library provides a rich collection of functions for performing common mathematical calculations, string manipulations, character manipulations, input/output, error checking and many other useful operations. This makes the programmer's job easier, because these functions provide many of the capabilities programmers need. The C++

Scanned by CamScanner

Standard Library functions are provided as part of the C++ programming environment.

List of some basic types of standard library functions:

- I/O Functions are in iostream header.
- String Functions are in cstring header.
- Mathematical Functions are in cmath header.

Header file names ending in .h are "old-style" header files that have been superseded by the C++ Standard Library header files. For example:

- <iostream>: Contains function prototypes for the C++ standard input and standard output functions. This header file replaces header file <iostream.h>
- <cmath>: Contains function prototypes for math library functions. This header file replaces header file <math.h>.
- <cstring>: Contains function prototypes for C-style string-processing functions. This header file replaces header file <string.h>.

cmath Library

Some of the important mathematical functions in header file <cmath> are:

1. abs(x)	Return the absolute value of integer number x
2. ceil(x)	Return the smallest integer that is greater or equal to x. (Round up value)
3. floor(x)	Returns the largest integer that is less than or equal to x. (Round down value)
4. exp(x)	Calculate the exponential function of x
5. log(x)	Calculate the natural logarithm of x
6. log10(x)	Calculate the logarithm of number x to the base 10
7. pow(x, y)	Calculate the numeric power (x raised to the power y)
8. sqrt(x)	Calculate the square root of x

Scanned by CamScanner

9. cos(x)	Calculate the cosine of an angle x (measured in radians)
10. sin(x)	Calculate the sine of an angle x (measured in radians)
11. tan(x)	Calculate the tangent of an angle x (measured in radians)

1. abs

int abs (int n); // long abs (long n);
// float abs (float n); // double abs (double n)

Return absolute value of the parameter n.

The absolute value corresponding to the parameter n is returned (|n|).

Parameters:

n integer value or floating-point value.

Return Value:

The absolute value of n.

Example:

```
/* abs example */
#include <iostream>
#include <cmath>
#include <conio.h>
using namespace std;

int main()
{
  int n,m;
  n=abs(23);
  m=abs(-11);
  cout << "n=" << n << "\n";
  cout << "m=" << m << "\n";

  getch();
}
```


Scanned by CamScanner

Output:

```
n=23
m=11
```

2. ceil

double ceil (double x); // float ceil (float x);

Round up a value.

Returns the smallest integer that is greater or equal to x.

Parameters:

x Floating-point value .

Return Value:

Ceiling of x.

Example:

```
/* ceil example */
#include <iostream>
#include <cmath>
#include <conio.h>
using namespace std;

int main() {

  cout << "ceil of 2.3 is " << ceil(2.3) << "\n";
  cout << "ceil of 3.8 is " << ceil(3.8) << "\n";
  cout << "ceil of -2.3 is " << ceil(-2.3) << "\n";
  cout << "ceil of -3.8 is " << ceil(-3.8) << "\n";

  getch();
}
```


Scanned by CamScanner

Output:

```
ceil of 2.3 is 3
ceil of 3.8 is 4
ceil of -2.3 is -2
ceil of -3.8 is -3
```

3. floor

double floor (double x); // float floor (float x);

Round down value.

Returns the largest integer that is less than or equal to x.

Parameters:

x Floating-point value.

Return Value:

Floor of x.

Example:

```
/* floor example */
#include <iostream>
#include <cmath>
#include <conio.h>
using namespace std;

int main() {

  cout << "floor of 2.3 is " << floor(2.3) << "\n";
  cout << "floor of 3.8 is " << floor(3.8) << "\n";
  cout << "floor of -2.3 is " << floor(-2.3) << "\n";
  cout << "floor of -3.8 is " << floor(-3.8) << "\n";

  getch();
}
```


Scanned by CamScanner

Output:

```
floor of 2.3 is 2
floor of 3.8 is 3
floor of -2.3 is -3
floor of -3.8 is -4
```

4. exp

double exp (double x); // float exp (float x);

Calculate exponential.

Returns the exponential value of parameter x.

Parameters:

x Floating-point value.

Return Value:

Exponential of x.

Example:

```
/* exp example */
#include <iostream>
#include <cmath>
#include <conio.h>
using namespace std;

int main(){

  double param, result;
  param = 5;
  result = exp(param);
  cout << "Exponential of " << param << " is " << result << "\n";

  getch();
}
```


Scanned by CamScanner

Output:

```
Exponential of 5 is 148.413
```

5. log

double log (double x); // float log (float x);

Calculate natural logarithm.

Returns the natural logarithm of parameter x.

Parameters:

x Floating-point value.

Return Value:

Logarithm of x.

Example:

```
/* log example */
#include <iostream>
#include <cmath>
#include <conio.h>
using namespace std;

int main(){

  double param, result;
  param = 5.5;
  result = log(param);
  cout << "ln(" << param << ") = " << result << "\n";

  getch();
}
```


Scanned by CamScanner

Output:

```
ln(5.5) = 1.70475
```

6. log10

double log10 (double x); // float log10 (float x);

Calculate logarithm base 10.

Returns the logarithm base 10 of parameter x: $\log_{10} x$

Parameters:

x Floating-point value.

Return Value:

Logarithm base 10 of x.

Example:

```
/* log10 example */
#include <iostream>
#include <cmath>
#include <conio.h>
using namespace std;

int main(){

  double param, result;
  param = 1000.0;
  result = log10(param);
  cout << "log10(" << param << ") = " << result << "\n";

  getch();
}
```


Scanned by CamScanner

Output:

```
log10(1000) = 3
```

7. pow

double pow (double x, double y);// double pow (double x, int y);
// float pow (float x, float y); // float pow (float x, int y);

Calculate numeric power.

Returns x raised to the power of y: x^y

Parameters:

x Base value.

y Exponent value.

Return Value:

x raised to the power of y.

Example:

```
/* pow example */
#include <iostream>
#include <cmath>
#include <conio.h>
using namespace std;

int main(){

  cout << "7 ^ 3 =  " << pow (7.0,3)<<endl;
  cout << "4.73 ^ 12 =  " << pow (4.73,12)<<endl;
  cout << "32.01 ^ 1.54 =  " << pow (32.01,1.54)<<endl;

  getch();
}
```


Scanned by CamScanner

Output:

```
7 ^ 3 = 343
4.73 ^ 12 = 1.2541e+008
32.01 ^ 1.54 = 208.037
```

8. sqrt

double sqrt (double x); // float sqrt (float x);

Calculate square root.

Returns the square root of parameter x.

Parameters:

x Non-negative floating-point value.

Return Value:

Square root of x.

Example:

```
/* sqrt example */
#include <iostream>
#include <cmath>
#include <conio.h>
using namespace std;

int main(){

  double param, result;
  param = 16.0;
  result = sqrt(param);
  cout << "sqrt(16.0) =  " << result << endl;

  getch();
}
```


Scanned by CamScanner

Output:

```
sqrt(16.0) = 4
```

9. cos

double cos (double x); // float cos (float x);

Calculate cosine.

Performs the trigonometric cosine operation on x returning a value between -1 and 1.

Parameters:

x Angle expressed in radians (180 degrees = PI radians).

Return Value:

Cosine of x.

Example:

```
/* cos example */
#include <iostream>
#include <cmath>
#include <conio.h>
using namespace std;

const double PI = 3.14159265;

int main()
{
  double param, result;
  param = 45;
  result = cos(param*PI/180);
  cout << "Cosine of " << param << " degrees is " << result;

  getch();
}
```


Scanned by CamScanner

Output:

```
Cosine of 45 degrees is 0.707107
```


Scanned by CamScanner

PRE-LAB WRITING ASSIGNMENT

Fill in the blank

1. Every C++ program has at least _____ function.
2. There are two types of functions: _____ functions and _____ functions.
3. The predefined functions are also called _____ or _____.
4. The _____ header contains function prototypes for the C++ standard input and standard output functions.
5. The _____ function is return the smallest integer that is greater or equal to x.
6. The ceil function round _____ value and the floor function round _____ value. (Choose up or down for each blank.)

Scanned by CamScanner

Lesson 7 Exercises

Assignment 7.1: Working with ***abs*** and ***pow*** predefined functions

Step1: Logon to your eLearning system.

Step2: Bring in the *math1.cpp* program from Lab7 folder.

Step3: Compile and run the program. Record the output.

```
Number          Absolute Value
*********       **************
-2              2
-1              1
0               0
1               1
2               2
```

تعديل

Step4: Modify the program by replacing the

cout<<i<<"\t\t\t"<<abs(i)<<endl;

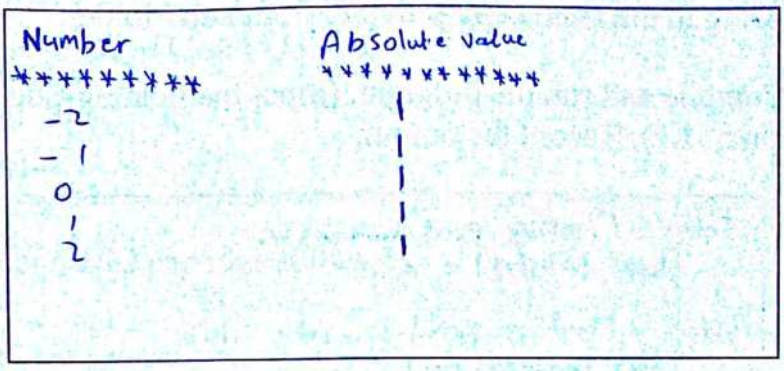
statement with a *cout<<i<<"\t\t\t"<<pow(abs(i), 3)<<endl;* statement.

Compile and run the program again to test the results. Record the output.

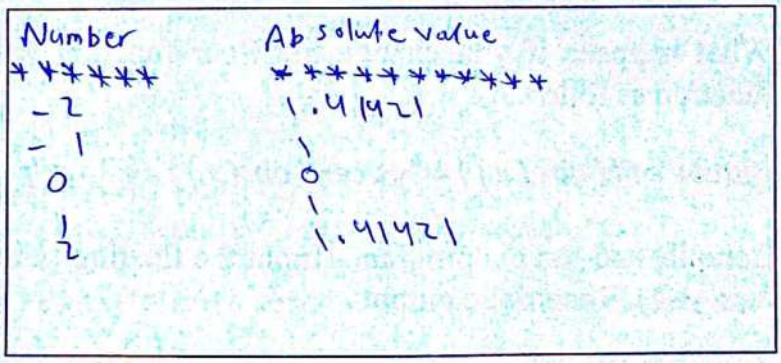
```
Number          Absolute Value
*********       **************
-2              8
-1              1
0               0
1               1
2               8
```


Scanned by CamScanner

Step5: What happens if you modify the exponent value of the pow function to **1/2** instead of **3?** Try it and record what happens. (Explain your answer)



Step6: Now modify the exponent value of the pow function **to 0.5** instead of **1/2**? Try it and record what happens. (Explain your answer)



The code of *math1.cpp* is as follows:

```
//This program print the absolute value for the numbers (-2, -1, 0, 1, 2)
#include<iostream>
#include<conio.h>
#include<cmath>

using namespace std;

int main(){

    cout<<"Number\t\t\tAbsolute Value\n";
    cout<<"*********\t\t***************\n";

    for(double i=-2; i<=2; i++)
        cout<<i<<"\t\t\t"<<abs(i)<<endl;

    getch();
}
```


Scanned by CamScanner

Assignment 7.2: Working with *abs, floor* and *ceil* predefined functions

Step1: Bring in the *math2.cpp* program from Lab7 folder.

Step2: Compile and run the program. (Input the floating-point value **2.5** then **-1.7**). Record the output.

```
Enter a Floating Point number:2.5
   floor (abs(x)) = 2

Enter a Floating Point number: -1.7
   floor (abs(x)) = 1
```

Step3: What happens if you change the floor function with the ceil function as follows?

```
cout<<"ceil ( abs ( x ) ) = "<< ceil ( abs( x ) ) <<"\n\n";
```

Compile and run the program. (Input the floating-point value **2.5** then **-1.7**). Record the output.

```
Enter a floating Point number :2.5
  Ceil (abs (x) ) = 3

Enter a floating Point number :- 1.7
  Ceil (abs (x)) = 2
```

Step4: Now modify the cout statement as follows?

```
cout<<"abs ( floor ( x ) ) = "<< abs ( floor( x ) ) <<"\n\n";
```

Compile and run the program. (Input the floating-point value **2.5** then **-1.7**). Record the output.

Scanned by CamScanner

```
Enter a floating point number:2.5
  abs(floor(X)) = 2

Enter a floating point number:-1.7
  abs(floor(X)) = 2
```

The code of *math2.cpp* is as follows:

```
//Nested Functions
#include<iostream>
#include<conio.h>
#include<cmath>

using namespace std;

int main(){

    float x;

    for(int i=1; i<=2; i++){
        cout<<"Enter a floating point number:";
        cin>>x;
        cout<<"floor ( abs ( x ) ) = "<< floor ( abs( x ) ) <<"\n\n";
    }

    getch();
}
```

Assignment 7.3: Working with *abs* and ***sqrt*** predefined functions

Step1: Bring in the *math3.cpp* program from Lab7 folder.

Step2: Compile and run the program. Record the output.

Scanned by CamScanner

20

Step3: modify the data type for the variable x from double to **bool** and modify the **cout** statement as follows:

cout<<sqrt((double) x) + abs(x);

Compile and run the program. Record the output. Explain what happens.

2

Step4: Modify the revised step3 program by replacing the variable **x** value to **-3**. Compile and run the program. Record the output.

2

Scanned by CamScanner

Step5: Modify the revised step4 program by changing the data type for the variable **x** from **bool** to **double**. Compile and run the program. Record the output. Explain what happens.

-1.#IND

The code of *math3.cpp* is as follows:

```
#include<iostream>
#include<conio.h>
#include<cmath>

using namespace std;

int main(){
    double x = 16;

    cout<<sqrt(x) + abs(x);

    getch();
}
```

20

Assignment 7.4: Write program

Step1: Create a new project named math4. After you've created the project, create a new source-code file named math4.cpp.

Step2: Write a program that print the result of the following mathematic expression:

$$y = \begin{cases} x^3 + \frac{2x}{3} & if\ x \leq 30 \\ \sqrt{x+10} & if\ x > 30 \end{cases}$$

Scanned by CamScanner

Write the code here:

```
int main( ) {
int x
~~cin~~
cin>>x;
if(x > ~~Pow(x,3) + ((2+x)/3)~~ 30)
cout<<"y="<<Pow(x,3) + ((2+x)/3)
else
if(x<30)
cout<<"y="<< sqrt(x+16)
```


Scanned by CamScanner

LESSON 8

Functions_Part2

Outline:

- *Overview*
  - ✓ *User-defined Functions*
  - ✓ *Value-returning functions*
  - ✓ *Void functions*
  - ✓ *Function prototype (declaration)*
  - ✓ *Reference Variable*
  - ✓ *Parameter Passed by Value and by Reference*
  - ✓ *Scope and lifetime of variables*
- *PRE-LAB WRITING ASSIGNMENT*
- *Lesson 8 Exercises*

Scanned by CamScanner

LESSON (8)
Functions_Part2

Overview

User-defined Functions

C++ allows programmers to define their own functions. These functions are known as user-defined functions. Every C++ program must contain at least one user-defined function, which must be named *main*.

Each function has its own name. When that name is encountered in a program, the execution of the program branches to the body of that function. When the function is finished, execution returns to the area of the program code from which it was called, and the program continues on to the next line of code. In simple programs most functions are called by the *main* function.

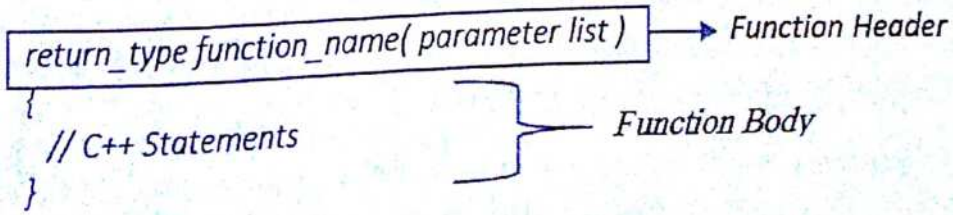
There are several motivations for using user-defined functions:

1. A programmer can focus on just that part of the program and construct it, debug it, and perfect it
2. Different people can work on different functions simultaneously, which can be assembled at a later time.
3. Can be re-used (even in different programs)
4. Enhance program readability

User-defined functions can be categorized as:

1. Value-returning functions: have a data type, return only one value to caller.
2. Void functions: no return type, do not return a value.

The general form of a C++ function definition is as follows:



Scanned by CamScanner

A C++ function definition consists of a function header and a function body. Here are all the parts of a function:

- Return Type: A function may return a value. The return type is the data type of the value the function returns. Some functions perform the desired operations without returning a value. In this case, the return type is the keyword *void*.
- Function Name: is the identifier by which it will be possible to call the function. The function name and the parameter list together constitute the function signature.
- Parameters: A parameter is like a placeholder. When a function is invoked, you pass a value to the parameter. Each parameter consists of a data type specifier followed by an identifier, like any regular variable declaration (for example: float f) and which acts within the function as a regular local variable. The different parameters are separated by commas. Parameters are optional; that is, a function may contain no parameters.
- Function Body: The function body contains a collection of statements that define what the function does.

Value-returning functions

One of the most useful forms of function is one that returns a value. In this case the type given to the function is that of the value to be returned. A return value, can be any of the data types we have studied so far. This means that a function can return a char, a short, an int, a long, a float, a double or a bool. In C++, if you declare a function without a return type, by default, the compiler considers that such a *function should return an integer.*

If you declare a function that is returning anything, the compiler will need to know what value the function returns. The value is set with the *return* keyword. The return value can be a literal, a variable that represents the result or an expression. A function can have any number of return statements. If the return statement omitted in the value-returning function, then the compile raise a *syntax error*.

Sometimes the calling function supplies some values to the called function. These are known as parameters. Some functions have needs parameters and some do not need parameters. The parameters of a

Scanned by CamScanner

function are provided between parentheses. If a function does not have a need of a parameter, leave its parentheses empty. The variables (or literals or expressions) which supply the values to a calling function called actual parameters. The variable which receive the value from called statement are termed formal parameters. The numbers of actual parameters and formals parameters should be the same.

When you call a function returns a value, then you can:

- Store the returned value in a variable using the assignment statement. For example: *x = function_name(....);*
- Print the returned value using output statement. For example: *cout<<function_name(...);*
- Using the return value in C++ expression. For example: *if(function_name(...)>=5)* ....
- Ignore the returned value. For example: *function_name(....);*

Here is an example to add two integers. To perform this task, a user-defined function addNumbers() is defined.

```
/* Value-returning functions with parameters */
#include <iostream>
#include <cmath>
#include <conio.h>
using namespace std;

int addNumbers (int n1, int n2) {
    int result;
    result=n1+n2;
    return result;
}

int main () {
    int sum;
    sum = addNumbers (6,5);
    cout << "The sum is " << sum;

    getch();
}
```

Formal Parameters

Actual Parameters

Scanned by CamScanner

This function has two input parameters (formal parameters), integer values n1 and n2, and returns the summations of the two parameter values. In the function a local variable result is used to temporarily hold the calculated value inside the function.

In order to examine this code, a C++ program always begins its execution by the main function. So we will begin there. We can see how the main function begins by declaring the variable sum of type int. Right after that, we see a call to a function called addNumbers. Notice the similarity between the structure of the call to the function and the declaration of the function itself.

```
int addNumbers (int n1, int n2)

sum = addNumbers (6,5);
```

Within the main function we called to addNumbers passing two values: 6 and 5, that correspond to the int n1 and int n2 parameters declared for function addNumbers. At the point at which the function is called from within main, the control is lost by main and passed to function addNumbers. The value of both arguments passed in the call (6 and 5) are copied to the local variables int n1 and int n2 within the function.

Function addNumbers declares another local variable (int result), and by means of the expression *result=n1+n2*, it assigns to result variable the result of n1 plus n2. The result is 11.

The following line of code:

```
return result;
```

The return statement terminates the execution of the addNumbers function and returns a value to the calling function (in this case, main). At this moment the program follows it regular execution from the same point at which it was interrupted by the call to addNumbers.

Scanned by CamScanner

```
return result;  //the returned value data type is int (int addNumbers (int n1, int n2))
```

11

```
sum = addNumbers (6,5);
```

So being the value returned by a function the value given to the function call itself when it is evaluated, the variable sum will be set to the value returned by *addNumbers (6, 5)*, that is 11. To explain it another way, you can imagine that the call to a function (*addNumbers (6, 5)*) is literally replaced by the value it returns (11).

Now, we using some different calling methods for the addNumbers function, so that you see other ways or moments when a function can be called.

1.

```
int a=7, b=-5;
cout<< "The sum is " <<addNumbers(a, b); // output: The sum is 12
```

2.

```
int a=5, sum;
sum=addNumbers(a+1, 9)/4;
cout<< "The sum is " <<sum; // output: The sum is 3
```

3.

```
addNumbers(10, 9); //Ignore the return value
```

The compiler generate a syntax error, if the programmer call the addNumbers function in one way of the following:

1. Calling without passing parameters. Ex: *sum=addNumbers();*
2. with passing actual parameters number less than the formal parameters. Ex: *sum=addNumbers(4);*
3. with passing actual parameters more than the formal parameters. Ex: *sum=addNumbers(4, 6, 9);*

Scanned by CamScanner

void Functions

The function declaration begins with a type, which is the type of the function itself (i.e., the type of the datum that will be returned by the function with the return statement). But what if we want to return no value?

Imagine that we want to make a function just to show a message on the screen. We do not need it to return any value. In this case we should use the void type specifier for the function.

Note that the function-type has been given as *void*, this tells the compiler that this function does not return any value.

Since this function does not return a value it cannot be used in an expression, and it cannot be displayed on the same line with the cout extractor and it cannot be assigned to a variable (since it does not return a value). Therefore, a void function can only be treating it as a statement as follows:

```
functionName(....);
```

void can also be used in the function's parameter list to explicitly specify that we want the function to take no actual parameters when it is called. For example, function displaymsg could have been declared as:

```
void displaymsg (void) {
        cout << "I'm a function!";
}
```

Although it is optional to specify void in the parameter list. In C++, a parameter list can simply be left blank if we want a function with no parameters.

What you must always remember is that the format for calling a function includes specifying its name and enclosing its parameters between parentheses. The non-existence of parameters does not permit to omitting the parentheses. For that reason the call to displaymsg function is:

Scanned by CamScanner

displaymsg ();

The parentheses clearly indicate that this is a call to a function and not the name of a variable or some other C++ statement. The following call would have been incorrect:

displaymsg;

Here is an example to print stars. To perform this task, a void function printStars() is defined.

```
#include <iostream>
#include <conio.h>
using namespace std;

void printStars (int count) {
    for(int i=1; i<=count; i++)
        cout<<"*";
}
int main () {

    printStars(5);
    cout<<" Welcome to C++ ";
    printStars(5);
    cout<<"\n\n\n";
    for (int row=1; row<=4; row++){
        printStars(row);
        cout<<endl;
    }
    getch();
}
```

The output of the program is:

```
***** Welcome to C++ *****


*
**
***
****
```


Scanned by CamScanner

In void function, you may or may not use the return statement, as there is no return value. Even without the return statement, control will return to the caller automatically at the end of the function.

The *return* statement stops execution and returns to the calling function. When a return statement is executed, the function is terminated immediately at that point, regardless of whether it's in the middle of a loop, etc. A void function may optionally contain one or more return statements.

In the following program, the return statement in the test function inforce the compiler to exit from the test function even if the test function not ended.

```
#include <iostream>
#include <conio.h>
using namespace std;

void test() {
    int n;
    cout<<"Input an integer number:";
    cin>>n;
    while(true){
        if(n%2==1){
            cout<<"you input odd number, we must terminate the program";
            return;
        }
        cout<<"The even number is:"<<n<<endl;
        cout<<"Input an integer number:";
        cin>>n;
    }
}

int main () {
    test();
    getch();
}
```

Exit from the test function

Scanned by CamScanner

The output of the program, when the user input 4, 10 then 5 is:

```
Input an integer number:4
The even number is:4
Input an integer number:10
The even number is:10
Input an integer number:5
You input odd number, we must terminate the program
```

Function prototype (declaration)

User-defined functions can be placed either before or after the function main. Until now, we have defined all of the functions before the function main. Although execution always starts with main, compilation proceeds from top to bottom, so if a user-defined function is placed after the function main, compiler will generate an error. The function's prototype (function declaration) must be placed before the function main. Compiler can then correctly translate a function call. The function prototype (function declaration) is a function heading without the body of the function.

In C++, function prototype is a declaration of function without its body to give compiler information about user-defined function. Function prototype for addNumbers function in the first example of this lesson is:

```
int addNumbers (int, int);
```

You can see that, there is no body of function in prototype. Also, there are only the data type of parameters but not their name (the prototype can have the name for its formal parameters). You can also declare function prototype as below:

```
int addNumbers (int n1, int n2);
```

Note: It is not necessary to define prototype if user-defined function exists before main() function.

Scanned by CamScanner

Here is an equivalent code to the first program (using function prototype):

```
#include <iostream>
#include <conio.h>
using namespace std;

int addNumbers(int , int ); ——→ Function prototype

int main () {
    int sum;
    sum=addNumbers(6,5);
    cout<<"The sum is: "<<sum;

    getch();
}

int addNumbers(int n1, int n2) { ——→ Function definition
    int result;
    result=n1+n2;
    return result;
}
```

Reference Variable

Think of a variable name as a label attached to the variable's location in memory. You can then think of a reference as a second label attached to that memory location. Therefore, you can access the contents of the variable through either the original variable name or the reference. The references allow you to create a second name for the variable that you can use to read or modify the original data stored in that variable.

For example, suppose we have the following example:

```
int   i = 17;
```

We can declare reference variables for i as follows.

```
int&   r = i;
```


Scanned by CamScanner

Read the & in these declarations as reference. Following example makes use of references on int and double:

```
#include <iostream>
#include <conio.h>
using namespace std;

int main () {
   // declare simple variables
   int    i;
   double d;

   // declare reference variables
   int&    r = i;
   double& s = d;

   i = 5;
   cout << "Value of i : " << i << endl;
   cout << "Value of i reference : " << r  << endl;

   d = 11.7;
   cout << "Value of d : " << d << endl;
   cout << "Value of d reference : " << s  << endl;

   getch();
}
```

When the above code is compiled and executed, it produces the following result:

```
Value of i : 5
Value of i reference : 5
Value of d : 11.7
Value of d reference : 11.7
```

References are usually used for function parameter lists.

Scanned by CamScanner

Parameter Passed by Value and by Reference

The formal parameters behave like other local variables inside the function and are created upon entry into the function and destroyed upon exit.

There are two types of function parameters:

- Value Parameter: formal parameter receives copy of content of corresponding actual parameter.
- Reference Parameter: formal parameter receives location (memory address) of corresponding actual parameter.

Value Parameters:

- Value of corresponding actual parameter copied into formal parameter.
- Value parameter has own copy of data.
- During program execution, value parameter manipulates data stored in own memory space. (Actual parameter's (Original variables) contents DO NOT change).
- Value parameters will accept expressions, constants, or variables from actual parameters (i.e., in function call)

Reference Parameters:

- Formal parameter receives and stores address of corresponding actual parameter.
- During program execution, address stored in reference parameter can modify data of corresponding actual parameter, by sharing same memory space.
- Reference parameters useful in two situations:
  - Changing value of actual parameter
  - When passing address would save memory space and time
- Reference parameter limitations: reference parameters will not accept expressions or constants from actual parameters (i.e., in function call), ONLY variables.

Scanned by CamScanner

- To pass the parameters by references, add ampersand (&) before parameter name in header and prototype.

By default, C++ uses call by value to pass parameters. In general, this means that code within a function cannot alter the actual parameters used to call the function.

For example, consider the following function definition:

```
void myFun(int& x, int y){
        x+=5;
        y++;
        cout<<x<< "\t"<<y<<endl;
}
```

The myFun function has two parameters, x is a reference parameter as indicated by the ampersand (&), and y is a value parameter. This ampersand must be in both the prototype and the function header.

Assume the main function contains the following code:

```
int a=10, b =20;
myFun (a, b); // passing a by reference and passing b by value
cout<<a<<"\t"<<b<<endl;
```

The output of this code is:

```
15      21
15      20
```

Notice that, the variable *a* is passed by reference, we are not passing a copy of its value, but we are somehow passing the variable itself (a variable reference) to the function and any modification that we do to the local variable *x* will have an effect in the variable a.

Scanned by CamScanner

A variable *y* without the ampersand sign (&), would have not passed the variable *b* by reference, but a copy of their value instead, and therefore, the value of variable *b* is not modified.

The following calling statements for myFun function are invalid:

- *myFun(10, b); // cannot passing literal to reference variable*
- *myFun(a+1, b); // cannot passing expression to reference variable*
- *const int c = 30;*
  myFun(c, b); // cannot passing constant to reference variable

Scope and lifetime of variables

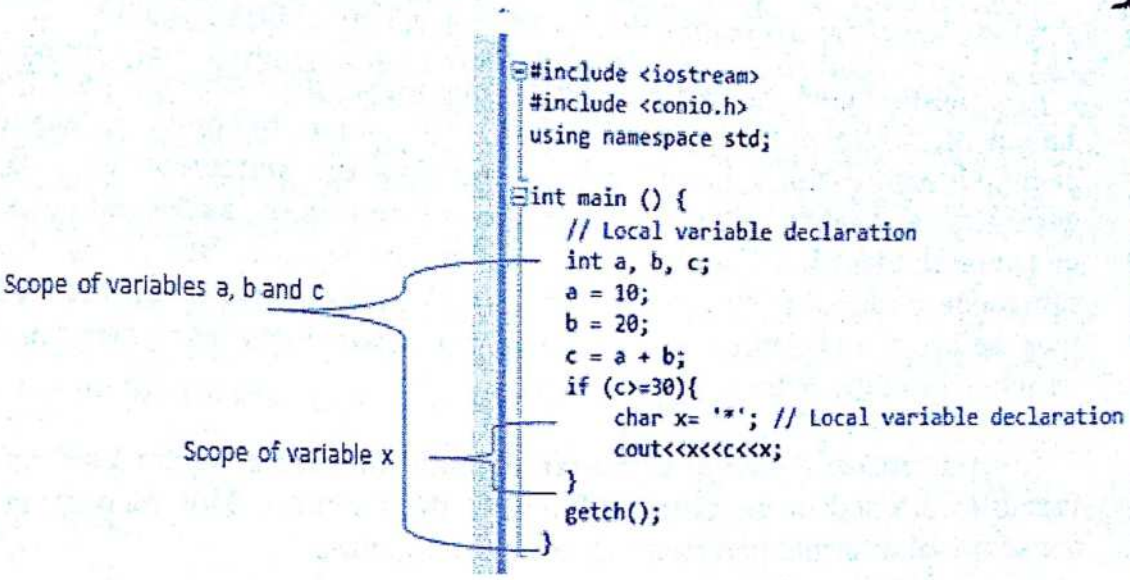
The scope of an identifier (variable, constant, etc.) is an indication of where it can be accessed in a program. There are two types of scopes:

1. Local scope.
2. Global scope

Local scope refers to identifiers defined within a block. They are active only within the bounds of that particular block. In C++ a block begins with a left brace { and ends with a right brace }. Since all functions (including main) begin and end with a pair of braces, the body of a function is a block. Variables defined within functions are called local variables. Local variables can normally be accessed anywhere within the function from the point where they are defined. However, blocks can be defined within other blocks, and the scope of an identifier defined in such an inner block would be limited to that inner block.

The following example using local variables:

Scanned by CamScanner

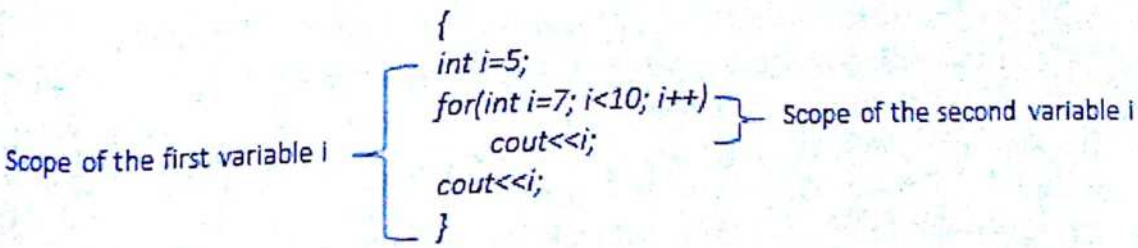


The output is

```
*30*
```

If the statement *cout<<x;* added after the if block (before *getch();* statement), in this case the variable x is used outside the scope and leads to compiler time (syntax) error, the error message is *'x' : undeclared identifier*.

The scope of variables declared in a for statement, begins with the for statement (where they are defined) and ends at the end of the for statement. Consider the following example:



The output is

```
7895
```


Scanned by CamScanner

The first cout statement using the inner i, where declared in the for statement, while the second cout statement using the outer i, where declared before the for statement. Removing the statement *int i=5;* generates a syntax error, because the second cout statement used undeclared identifier (the variable i is out of scope). But if the for statement modified as follows: *for(i=7; i<10; i++)* by removing the data type before the variable i, then the two cout statements using the same variable I and the output will be: *78910*.

A function's formal parameters have the same scope as local variables defined in the outmost block of the function. This means that the scope of a formal parameter is the entire function.

Global variables are defined outside of all the functions, usually on top of the program. The global variables will hold their value throughout the life-time of your program. A global variable can be accessed by any function. That is, a global variable is available for use throughout your entire program after its declaration. Following is the example using global and local variables:

```
#include <iostream>
#include <conio.h>
using namespace std;

int g; // Global variable declaration

int main () {
    int a, b; // Local variable declaration
    a = 10;
    b = 20;
    g = a + b;
    cout << g << endl;      } Using global variable g
    int g = 5;
    cout << g;              } Using local variable g
    getch();
}
```

The output is:

```
30
5
```


Scanned by CamScanner

It is possible to declare local and global variables of the same name. C++ provides the unary scope resolution operator (::) to access a global variable when a local variable of the same name is in scope. A global variable can be accessed directly without the unary scope resolution operator if the name of the global variable is not the same as that of a local variable in scope.

For example, in the previous code, if the second cout statement is modified as follows: *cout<<::g;* then the variable used in the statement is the global variable g. in this case the output is:

```
30
30
```

Note that the global variables are initialized automatically by the system when you define them as follows:

Data Type	Initializer
Numeric type (int, long, float, ...)	0
Character type (char)	'\0'
Boolean type (bool)	0

It is a good programming practice to initialize variables properly, otherwise sometimes program would produce unexpected result.

Lifetime is similar but not exactly the same as scope. It refers to the time during a program that an identifier has storage assigned to it. The local variable lifetime is begin when declaring the variable and ends when execution leaves the block where the variable declared. The global variable lifetime begins when storage allocated to variable (variable declaration) and the storage remains allocated as long as program executes.

Static variables means the storage is allocated statically and its lifetime is throughout the program execution time. The scope of static variable is the same as local variable.

Scanned by CamScanner

One of the biggest advantages of a function is the fact that it can be called multiple times to perform a job. This saves programming time and memory space. The values of local variables do not remain between multiple function calls. What this means is that the value assigned to a local variable of a function is lost once the function is finished executing. Local variables start "fresh," in terms of their value, each time the function is called. There may be times when a function needs to retain the value of a variable between calls. This can be done by defining the variable to be static, which means it is initialized at most once and its storage space is retained even after the function in which it is defined has finished executing. Thus the lifetime of a static variable is different than a normal local variable. Static variables are defined by placing the word static before the data type and name of the variable as shown below.

sta c int count = 0;

Consider the following example:

```
#include<iostream>
using namespace std;
void counter() {
    static int count=0;
    cout << count++;
}
int main() {
    for(int i=0;i<5;i++)
        counter();
}
```

The output is:

```
01234
```


Scanned by CamScanner

Let's see the same program without using static keyword:

```
#include<iostream>
using namespace std;
void counter() {
    int count=0;
    cout << count++;
}
int main() {
    for(int i=0;i<5;i++)
        counter();
}
```

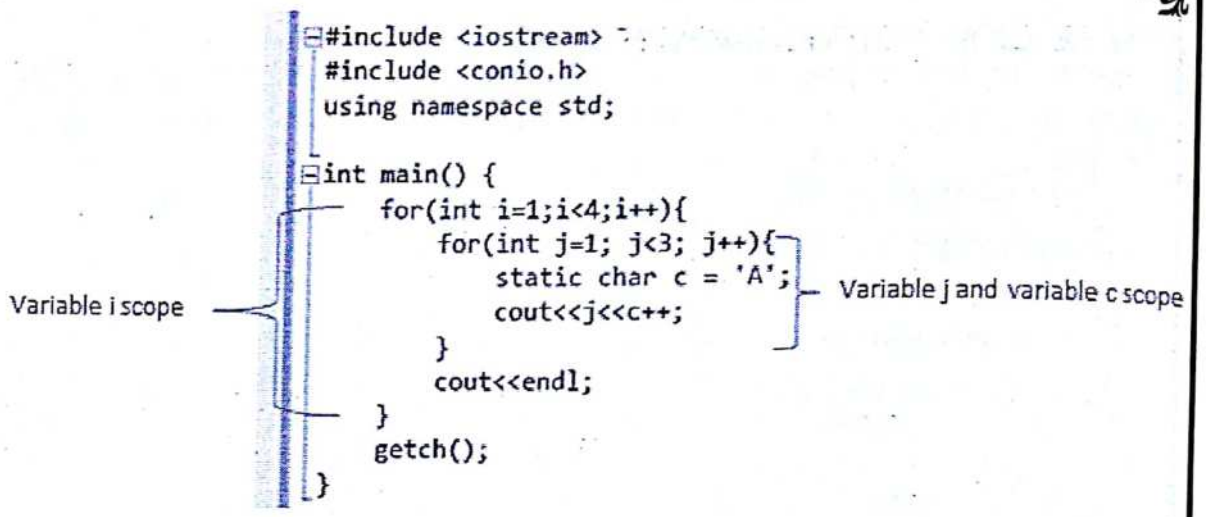
The output is:

```
00000
```

If we do not use static keyword, the variable count, is reinitialized every time when counter() function is called, and gets destroyed each time when counter() functions ends. But, if we make it static, once initialized count will have a lifetime till the end of main() function and it will carry its value through function calls too. If you don't initialize a static variable, they are by default initialized as a global variable.

Consider the following example:

Scanned by CamScanner



The output is:

```
1A2B
1C2D
1E2F
```

Notice that, the variable j and the variable c have the same scope, local scope, but different lifetime.

Scanned by CamScanner

PRE-LAB WRITING ASSIGNMENT

Fill in the blank

1. The word _____ precedes the name of every function prototype and heading that does not return a value back to the calling function.

2. Pass by _____ indicates that a copy of the actual parameter is placed in the memory location of its corresponding formal parameter.

3. _____ parameters are found in the call to a function.

4. A prototype must give the _____ of its formal parameters and may give their _____.

5. A after a data type in the function heading and in the prototype _____ indicates that the parameter will be passed by reference.

6. Pass by _____ indicates that the location of an actual parameter, rather than just a copy of its value, is passed to the called function.

7. Every function that begins with a data type in the heading, rather than the word void, must have a(n) _____ statement somewhere, usually at the end, in its body of instructions.

8. A _____ scope refers to identifiers defined within a block.

9. A _____ variables are defined outside of all the functions, usually on top of the program and can be accessed by any function.

10. The unary operator _____ is used to access a global variable when a local variable of the same name is in scope.

Scanned by CamScanner

Lesson 8 Exercises

Assignment 8.1: Working with void function.

Step1: Logon to your eLearning system.

Step2: Bring in the *msg.cpp* program from Lab8 folder.

Step3: Fill in the code so that the program will print out the message listed in the comments at the beginning of the program. The message will be printed by the function which is called by the main function. Compile and run the program. Record the output.

Write your code here:

The program output:

Scanned by CamScanner

Step4: Modify the program to allow the user to choose which programming language they want it printed, by input a number from keyboard, so that an input of 1 from the user will print "I love C++ programming language", a 2 will print "I love java programming language" and any other number will be invalid so that the user will need to enter a new choice. The user's input number will be passed to the writeMsg function.

Write your code here:

Scanned by CamScanner

The code of *msg.cpp* is as follows:

```
// This program prints the message:.
// "I love C++ programming language"
// in a function called writeMsg that is called by the main function

#include <iostream>
#include <conio.h>
using namespace std;

void writeMsg();  //This is the prototype for the writeMsg function

int main() {
// Fill in the code to call the writeMsg function

    getch();
}

//  ************************************************************************
//                                writeMsg
//
//   task: This function prints a message
//   Input: none
//
//    ************************************************************************
// Fill in the function heading and the body of the function that will print
// to the screen the message listed in the comments at the beginning of the program
```

Assignment 8.2: Working with void and value-returning function with reference parameter.

Step1: Bring in the *mul.cpp* program from Lab8 folder.

Step2: Compile and run the program. Input the values 3 and 4. Record the output.

Scanned by CamScanner

Step3: What happens if you modify the passing type of the third parameter in the mulFun function from passing by reference to passing by value? (Note: remove the ampersand (&) from function prototype and function definition). Compile and run the program. Input the values 3 and 4. Record the output and explain the result.

Step4: modify the program so that the function mulFun return the multiplication result by using return statement instead of using reference parameter. The function prototype must be as follows:

```
int mulFun (int, int);
```

Compile and run the program. Input the values 3 and 4. Record the output.

Scanned by CamScanner

Write your code here:

The program output:

Scanned by CamScanner

The code of *mul.cpp* is as follows:

```
#include <iostream>
#include <conio.h>
using namespace std;

void mulFun(int, int, int&);
int main(){
    int a, b, c;
    cout<<"Enter the first integer number: ";
    cin>>a;
    cout<<endl<<"Enter the second integer number: ";
    cin>>b;
    mulFun(a,b,c);
    cout<<endl<<"The result is: "<< c;

    getch();
}

void mulFun (int a, int b, int& res){
    res = a * b;
}
```

Assignment 8.3: Working with void and value-returning function.

In the code fragments below, the programmer has almost certainly made an error(s). Find the errors and state the reason.

The Code	Errors

void printTime (int hour, minute) {
cout << hour;
cout << ":";
cout << minute;
}
int main() {
int hour = 11,minute = 59;
printTime (int hour, int minute);
}	

int main() {
int hour = 11,minute = 59;
printTime (int hour, int minute);
}	

Scanned by CamScanner

void printTime (int hour, minute) {
cout << hour;
cout << ":";
cout << minute;
}	

void myFun(int c){
cout<<c;
return c+1;
}
int main() {
int x=10, y;
y=myFun(x);
cout<<y;
}	

Assignment 8.4: General examples in function.

What is the output of the following codes?

The Code	The Output

float F1( );
int F2(float);
 	
int main(){
cout<<F2(F1());
getch();
}
float F1( ){
return 4;
}
int F2(float a){
return --a/-2;
}
	-1

int sequence (int n) {
while (n != 1) {
if (n%2 == 0)
n = n / 2;
else
n = n*3 + 1;
}
return 3.9;
}
	3.9

Scanned by CamScanner

```
int main() {
        cout<<sequence(2);
        getch();
}
```

```
int mystery(int x, int y, int z);

int main() {
  cout << mystery(7, 8, 3) << endl;
  cout << mystery(10, 5, 30) << endl;
  cout << mystery(9, 12, 11) << endl;
  cout << mystery(5, 5, 8) << endl;
  cout << mystery(10, 10, 10) << endl;
  getch();
}

int mystery(int x, int y, int z) {
        if (x <= y && x <= z)
                return (y + z - x);

        if (y <= z && y <= x)
                return (z + x - y);

        return (x + y - z);
}
```

12
35
8
2
10

```
void X(int &A, int &B){
        A = A + B;
        B = A - B;
        A = A - B;
}
void main(){
        int a = 4, b =18;
        X(a,b);
        cout<<a<<","<<b;
        getch();
}
```

0, 0

```
int global = 10;
void func(int &x, int y){
        x = x - y;
        y = x * 10;
        cout << x << y << '\n';
}
```


Scanned by CamScanner

```
void main(){
 int global = 7;
 func (::global, global);
 cout << global <<","
     << ::global <<'\n';
 func(global,::global);
 cout<< global << ","
    <<::global<<'\n';
 getch();
}
```

```
int i=100;
void abc(){
        sta  c int i=8;
        cout<< "first=" <<i++<<endl;
}
int main(){
 int i = 2;
 abc();
 cout << "second =" << i << endl;
 abc();
 getch();
}
```

first = 8
second = 2
first = 9

Assignment 8.5: Write program_1

Step1: Create a new project named character. After you've created the project, create a new source-code file named character.cpp.

Step2: Write a function called isCapital that takes as input a char value and return true if the character is uppercase; otherwise, it returns false. Then write a main function that calls the isCapital function to test its correctness.

Scanned by CamScanner

Write the code here:

Assignment 8.6: Write program_2

Step1: Create a new project named exchange. After you've created the project, create a new source-code file named exchange.cpp.

Step2: Write a function called swap that takes as input a two integer values, the function code swap the two values. Then write a main function that calls the swap function to test its correctness.

Scanned by CamScanner

Write the code here:

Assignment 8.7: Write program_3

Step1: Create a new project named countInteger. After you've created the project, create a new source-code file named countInteger.cpp.

Step2: Write a function called check that takes as input a number and return true if the number is integer; otherwise, it return false. Then write a main function that accepts as input from user until the user input 0, then print number of integer values inputted by user (using check function).

Scanned by CamScanner

Write the code here:

Scanned by CamScanner

LESSON 9

Arrays- one dimensional arrays

Outline:

- *Overview*
  - ✓ *Introduction*
  - ✓ *Declaring an Array*
  - ✓ *Initializing Arrays*
- *PRE-LAB WRITING ASSIGNMENT*
- *Lesson 9 Exercises*

Scanned by CamScanner

LESSON (9)
Arrays- one dimensional arrays

Overview

Introduction

Arrays are used to store a collection of variables all of the same type. For instance you can have arrays that store numbers, words etc.

Instead of declaring individual variables, such as number0, number1, ..., and number99, you declare one array variable such as numbers and use numbers[0], numbers[1], and ..., numbers[99] to represent individual variables. A specific element in an array is accessed by an index. This *index* number is used to access the value of that element.

Array Dimension

- In a one-dimensional array, you can think of the elements as being arranged in a list.
- In a two-dimensional array, you can think of the elements as being arranged in a table with rows and columns.

Declaring an Array

To declare an array in C++, the programmer specifies the type of the elements and the number of elements required by an array as the following syntax:

```
type arrayName [ arraySize ];
```

This is called a one-dimension array. The *arraySize* must be an integer literal or integer constant greater than zero and type can be any valid C++ data type.

For example:

```
int number[12];
```


Scanned by CamScanner

This array is called numbers and will store 12 integers.

```
const int size =12;
float myArray[size];
```

This array is called myArray and will store 12 floating-point numbers.

```
char name [10];
```

This array is called names and will store 9 characters plus \0.

Initializing Arrays

After declaring the array, many ways could be used to initialize it. To explain them suppose we have a one dimensional array as follow:

```
int example[10];
```

This array called example with 10 elements but not initialized yet.

You can assign values to each element just as with other variables

```
example [0] = 10;
example [3] = 3;
```

These two lines will set the first member of example equal to 10 and the fourth equal to 3.

Arrays can be initialized during declaration; values are placed between curly braces.

The array's size is determined by the number of initial values in the braces, see the following example:

```
double array [5] = {2.25, 2.50, 6.98, 21, 25.68};
```

The number of values between braces { } cannot be larger than the number of elements that we declare for the array between square brackets [ ].

Scanned by CamScanner

If you omit the size of the array, an array just big enough to hold the initialization is created. Therefore, if you write:

```
double balance[] = {10.0, 2.2, 3.8, 7.0, 30.0};
```

The number of values here is 5 so the size of array is 5.

- partial Initialization of Arrays During Declaration
  - The statement: *int array[10] = {0};* declares an array of 10 elements and initializes all of them to zero.
  - The statement: *int array[10] = {8, 5, 12};* declares an array of 10 elements and initializes array[0] to 8, array[1] to 5, array[2] to 12. All other elements are initialized to 0.

- Alternatively to all of previous methods you could input the values of array elements from the keyboard, as follow :

```
int array[5];
for(i=0; i<5; i++)
{
        cout << "enter value of member " << i << endl;
        cin >> array[i];
}
```

A single element also can be entered from the keyboard:

```
cin >> array[2]; //take input from the user
                 //and insert in third element
cin >> mark[i]; // take input from the user
                 //and insert in (i+1)th element
```

Note: If you try to access array elements outside of its bound, the compiler may not show any error. However, this may cause unexpected output (undefined behavior).

Scanned by CamScanner

Basic Operation on One Dimensional Array

Here is a program that will find the sum of an array.

```
#include<iostream>
using namespace std;
int main() {
    const int SIZE=5;
    int arr[SIZE];
    int i;
    int sum;
    sum=0; //We must initialize sum to zero
    for(i=0; i<SIZE; i++) {
        cout<< "Enter a number" <<endl;
        cin>>arr[i];
    }
    for(i=0; i<SIZE; i++) {
        sum=sum+arr[i];
    }
    cout<< "The sum of the five numbers is " << sum <<endl;
}
```

The program output (if we enter 1 2 3 4 5 as array elements) will be:

```
Enter a number
1
Enter a number
2
Enter a number
3
Enter a number
4
Enter a number
5
The sum of the five numbers is 15
```


Scanned by CamScanner

To search for an element from an array, we can use linear search as follow:

```
#include<iostream>
using namespace std;
int main() {
    int A[5]={4,5,6,2,1},element;
    bool flag=0;
    cout<<" Enter an element to search :";
    cin>>element;
    for(int i = 0; i < 5; i++) {
        if(A[i] == element) {
            cout << "\n Element Found at : " << i;
            flag=1;
        }
    }
    if (flag==0)
        cout << "\n Element Not Found in the array" << endl;
}
```

The output (if we enter 2) will be:

```
Enter an element to search :2
Element Found at : 3
```

If we enter (7) for example, the output will be:

```
Enter an element to search :7
Element Not Found in the array
```


Scanned by CamScanner

No Aggregate Operations on Arrays

C++ does not allow aggregate operations on arrays, such as assignment, input, output, etc. that treats the entire array as a single unit. For example:

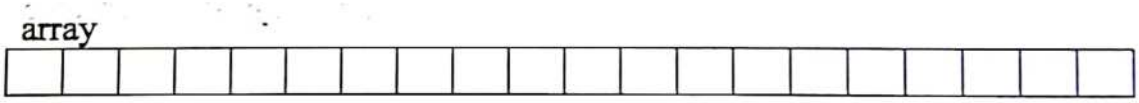
```
int array1[4]={3,4,2,1};
int array2[4];
array2=array1; // illegal operation
```

Character Arrays

Character Array is actually a one-dimensional array of characters which is terminated by a *null* character '\0'. Thus a *null*-terminated string contains the characters that comprise the string followed by a null.

For example, the following array:

char array [20]; is an array that can store up to 20 elements of type char. It can be represented as:



Therefore, in this array, in theory, we can store sequences of characters up to 20 characters long. But we can also store shorter sequences. For example, array could store at some point in a program either the sequence "Hello" or the sequence "Hello Every Body", since both are shorter than 20 characters.

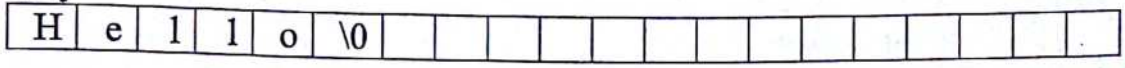
Therefore, since the array of characters can store shorter sequences than its total length, a special character is used to signal the end of the valid sequence: the null character, whose literal constant can be written as '\0' (backslash zero).

Our array of 20 elements of type char, called array, can be initialized and represented storing the characters sequences "Hello" and "Hello Every Body", as:

Scanned by CamScanner

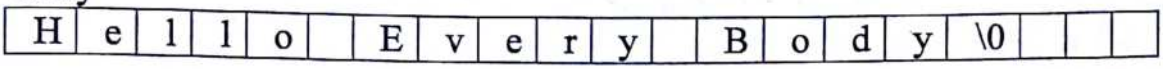
```
char array[]={'H','e','l','l','o','\0'};
```

array




```
char array[]={'H','e','l','l','o',' ','E','v','e','r','y',' ','B','o','d','y','\0'};
```

array



Strings of characters have an additional way to initialize their values: using constant strings, which are specified enclosed between double quotes ("), for example:

"the result is: " is a constant string .

Unlike single quotes (') which specify single character constants, double quotes (") are constants that specify a succession of characters. Strings enclosed between double quotes always have a null character ('\0') automatically appended at the end.

Therefore we could initialize the string *myarray* with values by either of these two ways:

```
char myarray [] = { 'H', 'e', 'l', 'l', 'o', '\0' };
char myarray [] = "Hello";
```

In both cases the array or string of characters *myarray* is declared with a size of 6 characters (elements of type char): the 5 characters that compose *Hello* plus a final null character ('\0') which specifies the end of the string and that, in the second case, when using double quotes (") it is automatically appended.

Before going further, notice that the assignation of multiple constants like double-quoted constants (") to arrays are only valid when initializing the array, that is, at the moment when declared. The Following expressions within the code are not valid for arrays:

```
myarray = "Hello";
myarray [] = "Hello";
myarray = { 'H', 'e', 'l', 'l', 'o', '\0' };.
```


Scanned by CamScanner

Reading strings with/without embedded blanks

To read a string without blanks cin can be used

cin>>str;

To read a string with blanks *cin.getline()* or *gets()* can be used.

cin.getline(str,80); or gets(str);

Printing strings

cout and *puts()* can be used to print a string.

cout<<str; or puts(str);

Note: For gets( ) and puts(), the header file <cstdio> (formally stdio.h) has to be included. puts() can be used to display only strings. It takes a line feed after printing the string.

The following example shows how to insert the array elements from keyboard and print it:

```
#include <iostream>
using namespace std;
int main (){
    char question[] = "Please, enter your first name: ";
    char greeting[] = "Hello, ";
    char yourname [80];
    cout << question;
    cin >> yourname;
    cout << greeting << yourname << "!";
}
```

The output of this program, if the user enter Nawal as first name:

```
Please, enter your first name: Nawal
Hello, Nawal!
```


Scanned by CamScanner

The next code is a function to count the number of words in a string:

```
void count(char S[])
{
        int words=0;
        for(int i=0;S[i]!='\0';i++) {
            if (S[i]==' ')
                words++;    //Checking for spaces
        }
}
```

Another function to change the case of string S to uppercase:

```
void Upper(char S[]){
    for(int i=0;S[i]!='\0';i++)
        S[i] = (S[i]>='a' && S[i]<='z')?(S[i]-32):S[i];
}
```

Strings Functions

The individual characters in a string can be manipulated either directly by the programmer as shown above or by using special functions provided by the C/C++ libraries such as cstring.

Cstring library supports a wide range of functions that manipulate null-terminated strings:

S.N.	Function & Purpose
1	strcpy(s1, s2); // Copies string s2 into string s1.
2	strcat(s1, s2); //Concatenates string s2 onto the end of string s1.
3	strlen(s1); //Returns the length of string s1.

4	strcmp(s1, s2); //Returns 0 if s1 and s2 are the same;
	//-1 if s1<s2; 1 if s1>s2.

Scanned by CamScanner

Strcpy: Copy string

*char * strcpy (char * destination, const char * source);*

Copies the C string pointed by source into the array pointed by destination, including the terminating null character (and stopping at that point).

To avoid overflows, the size of the array pointed by destination shall be long enough to contain the same C string as source (including the terminating null character), and should not overlap in memory with source.

Parameters

Destination: Pointer to the destination array where the content is to be copied.

Source: C string to be copied.

Return Value: destination is returned

strcmp : Compare two strings

*int strcmp (const char * str1, const char * str2);*

Compares the C string str1 to the C string str2. This function starts comparing the first character of each string. If they are equal to each other, it continues with the following pairs until the characters differ or until a terminating null-character is reached. This function performs a binary comparison of the characters.

Parameters

str1 :C string to be compared.

str2: C string to be compared.

Return Value: Returns an integral value indicating the relationship between the strings:

Scanned by CamScanner

Return value	Indicates
-1	the first character that does not match has a lower value in str1 than in str2
0	the contents of both strings are equal
1	the first character that does not match has a greater value in str1 than in str2

Strcat: Concatenate strings

*char * strcat (char * destination, const char * source);*

Appends a copy of the source string to the destination string. The terminating null character in destination is overwritten by the first character of source, and a null-character is included at the end of the new string formed by the concatenation of both in destination.

destination and source shall not overlap.

Parameters

Destination: Pointer to the destination array, which should contain a C string, and be large enough to contain the concatenated resulting string.

Source: C string to be appended. This should not overlap destination.

Return value: destination is returned.

strlen: Get string length

*size_t strlen (const char * str);*

Returns the length of the C string str. The length of a C string is determined by the terminating null-character: A C string is as long as the number of characters between the beginning of the string and the terminating null character (without including the terminating null character itself).

Scanned by CamScanner

This should not be confused with the size of the array that holds the string. For example:

char mystr[100]="test string";

defines an array of characters with a size of 100 chars, but the C string with which mystr has been initialized has a length of only 11 characters. Therefore, while sizeof(mystr) evaluates to 100, strlen(mystr) returns 11.

Parameters

Str :C string.

Return Value: The length of string.

Following example makes use of few of the above-mentioned functions:

```
#include <iostream>
#include <cstring>
using namespace std;
int main () {
   char str1[10] = "Hello";
   char str2[10] = "World";
   char str3[10];
   int  len ;
   // compares str1 with str2
   cout << "strcmp( str3, str1) : " << strcmp( str3, str1) << endl;
   // copy str1 into str3
   strcpy( str3, str1);
   cout << "strcpy( str3, str1) : " << str3 << endl;
   // concatenates str1 and str2
   strcat( str1, str2);
   cout << "strcat( str1, str2): " << str1 << endl;
   // total lenghth of str1 after concatenation
   len = strlen(str1);
   cout << "strlen(str1) : " << len << endl;
}
```


Scanned by CamScanner

When the above code is compiled and executed, it produces result something as follows:

```
strcmp( str3, str1) : 1
strcpy( str3, str1) : Hello
strcat( str1, str2): HelloWorld
strlen(str1) : 10
```


Scanned by CamScanner

PRE-LAB WRITING ASSIGNMENT

Fill in the blank

1. The arraySize in array declaration must be an ---------- constant greater than zero.

2. C++ does not allow aggregate operations on arrays, such as assignment, input, output, etc. that treats the entire array as a single unit. This statement is true or false ----------.

3. example [0] = 10; this statement stores 10 in the ---------- array element .

4. Character Array is actually a one-dimensional array of characters which is terminated by a ----------.

5. ---------- library supports a wide range of functions that manipulate null-terminated arrays, i.e. strcpy function.

6. The string function that concatenates a string onto the end of another string called ----------.

Scanned by CamScanner

Lesson 9 Exercises

Assignment 9.1: Working with 1D Array and Functions.

Step1: Logon to your eLearning system.

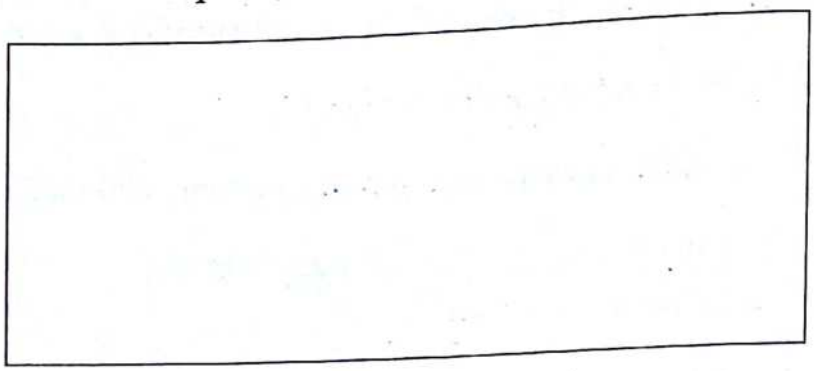
Step2: Bring in the *scoreArray.cpp* program from Lab9 folder.

Step3: Fill in the code as mentioned in the comments.

Write your code here:

Scanned by CamScanner

Step4: Run the program with the following data: 90 45 73 62 -99 and record the output.



The code of *scoreArray.cpp* is as follows:

```
// This program will read in a group of exam scores (positive integers from 1 to 100)
// from the keyboard and then calculate and output the average score
// as well as the highest and lowest score. There will be a maximum of 100 scores.

#include <iostream>
#include <conio.h>
using namespace std;

int grades[100]; // an integer array of 100 elements

float findAverage (int);  // finds average of all grades
int   findHighest (int);  // finds highest of all grades
int   findLowest  (int);  // finds lowest of all grades

int main() {

int numberOfGrades; // the number of grades read.
int pos; // index to the array.
float avgOfGrades; // contains the average of the grades.
int highestGrade; // contains the highest grade.
int lowestGrade; // contains the lowest grade.

// Read in the values into the array
pos = 0;
cout << "Please input a grade from 1 to 100, (or -99 to stop)" << endl;
cin  >> grades[pos];
while (grades[pos] != -99) {
// Fill in the code to read the grades
}
numberOfGrades = _________;  // Fill blank with appropriate identifier
// call to the function to find average
avgOfGrades = findAverage(numberOfGrades);
cout << endl << "The average of all the grades is " << avgOfGrades << endl;
// Fill in the call to the function that calculates highest grade
cout << endl << "The highest grade is " << highestGrade << endl;
// Fill in the call to the function that calculates lowest grade
// Fill in code to write the lowest to the screen

getch();
}
```


Scanned by CamScanner

```
//*************************************************************************
//findAverage Function Definition
//*************************************************************************
float findAverage (int size)
{
float sum = 0;   // holds the sum of all the numbers
for (int pos = 0; pos < size; pos++)
sum = sum + grades[pos];
return (sum / size);  //returns the average
}

//*************************************************************************
//findHighest Function Definition
//*************************************************************************
int   findHighest (int size)
{
// Fill in the code for this function
}
//*************************************************************************
//findLowest Function Definition
//*************************************************************************
int   findLowest  (int size)
{
// Fill in the code for this function
}
```

Assignment 9.2: Working with 1D Array, String and Functions.

Step1: Bring in the *student.cpp* program from Lab9 folder.

Step2: Fill in the code as mentioned in the comments.

Scanned by CamScanner

Write your code here:

Step3: Compile and run the program with 3 grades per student using the sample data below.

Samer	100 90 90
Osama	95 92 87
Ali	60 58 71

Scanned by CamScanner

Record the output:

The code of student.*cpp* is as follows:

```
// This program will input an undetermined number of student names and a number of grades for
// each student. The number of grades is given by the user. The grades are stored in an array.
// Two functions are called for each student. One function will give the numeric
// average of their grades. The other function will give a letter grade to that average.
// Grades are assigned on a 10 point spread. 90-100 A   80-89 B  70-79 C   60-69 D   Below 60 F
#include <iostream>
#include <conio.h>
using namespace std;
const  int MAXGRADE = 25;           // maximum number of grades per student
const  int MAXCHAR = 30;            // maximum characters used in a name
char stdname[MAXCHAR + 1];          // character array for names having 30 characters or less.
float grades[MAXGRADE];             // one dimensional grades array
float findGradeAvg(int);            // finds grade average by taking number of grades
char  findLetterGrade(float);       // finds letter grade from average given to it as a parameter
int main() {
    int numOfGrades;                 // holds the number of grades
    float average;                   // holds the average of a student's grade
    char moreInput;                  // determines if there is more input
    // Input the number of grades for each student
    cout << "Please input the number of grades each student will receive." << endl
        << "This must be a number between 1 and " << MAXGRADE << " inclusive" << endl;
    cin >> numOfGrades;
    while (numOfGrades > MAXGRADE || numOfGrades < 1) {
        cout << "Please input the number of grades for each student." << endl
            << "This must be a number between 1 and " << MAXGRADE << " inclusive\n";
        cin >> numOfGrades;
    }
    // Input names and grades for each student
    do{
        cout << "Please input the name of the student" << endl;
        cin >> stdname;
        for (int count = 0; count < numOfGrades; count++) {
            cout << endl << "Please input a grade" << endl;
            // Fill in the input statement to place grade in the array
        }
        cout << stdname << " has an average of ";
        // Fill in code to get and print average of student to screen
        // Fill in call to get and print letter grade of student to screen
        cout << endl << endl << endl; cout << "Please input a y if you want to input more students"
            << " any other character will stop the input" << endl;
        cin >> moreInput;
    }while (moreInput == 'y' || moreInput == 'Y');
    getch();
}
```


Scanned by CamScanner

```
//*************************************************************************
//findGradeAvg Function Definition
//*************************************************************************
float findGradeAvg(int numGrades)
{
// Fill in the code for this function
}
//*************************************************************************
//findLetterGrade Function Definition
//*************************************************************************
char  findLetterGrade(float avgGrade)
{
// Fill in the code for this function
}
```

Assignment 9.3: Working with arrays.

What is the output of the following codes?

The Code	The Output

int rg[5] = {9,8,7};
int Ind = 2;
int X = rg[Ind + 2];
int Y = rg[Ind] + 2;
cout << X << "\t" << Y;
	0 9

const int Length=5; int Arr[Length-2]; Arr[0]=2; Arr[1]=Arr[0]*3; Arr[2]=Arr[Arr[0]-1]; cout<<Arr[0]<<'\t'<<Arr[1]<<'\t'<<Arr[2];	6 6

double mylist[4];
mylist[0]=2.5;
for(int i=1;i<4;i++) {
mylist[i]=i*mylist[i-1];
if(i>2)
mylist[i]=mylist[i]/2;
}
 	
for(int i=0;i<4;i++)
cout<<mylist[i]<<'\t';
	2.5 2.5 5 7.5
	2.5 2.5 5 7.5

Scanned by CamScanner

```
void printString(char a[]){
        int i = 0 ;
        while ( a[i]!='\0' ){
                cout<<a[i];
                i++;
        }
}
int main()
{
  char ch1[50] = "Welcome to";
  char ch2[] = " C++";
  printString(ch1);
  cout<<endl;
  printString(ch2);
  cout<<endl;
  strcat(ch1, ch2);
  printString(ch1);
  cout<<endl;
  printString(ch2);
  cout<<endl;
  cout<<strcmp(ch2,"C++")<<endl;
  strcpy(ch2,"**");
  printString(ch2);
  cout<<endl;
  cout<<strlen(ch2)<<"\t"<<ch2[3];
}
```

```
void copy(char S1[ ], char S2[ ]){
int i;
for(i=0; S2[i]!='\0'; i++)
  S1[i]=S2[i];
S1[i]='\0';
}

int main(){
char x[]="AB C";
char y[30]="############";
copy(y, x);
cout<<x<<endl<<y<<endl<<y[5];

        getch();
}
```


Scanned by CamScanner

Assignment 9.4: Write program_1

Step1: Create a new project named search. After you've created the project, create a new source-code file named search.cpp.

Step2: Write a program that firstly asks the user to enter an integer number num. Then declare an integer array of size=10. Ask the user to enter their elements. Your program should:

a. Search on the item inside the array and determine if the item is exist or not

b. If the item is exist, print the indexes at which the item is stored.

Write the code here:

Scanned by CamScanner

Assignment 9.5: Write program_2

Step1: Create a new project named countWords. After you've created the project, create a new source-code file named countWords.cpp.

Step2: Write a function called count that takes as input a string and return the number of words in the string. Then write a main function that calls the count function to test its correctness. (Note: call the function count several times with different string)

Write the code here:

Scanned by CamScanner

LESSON 10

Arrays- two dimensional arrays

Outline:

- *Overview*
  - ✓ *Introduction*
  - ✓ *Declaration of Two-Dimensional Array*
  - ✓ *Initialization of Two-Dimensional Array*
  - ✓ *Referring to Array Elements*
  - ✓ *Basic Operation on Two Dimensional Arrays*
- *PRE-LAB WRITING ASSIGNMENT*
- *Lesson 10 Exercises*

Scanned by CamScanner

LESSON (10)
Arrays- two dimensional arrays

Overview

Introduction

It is a collection of data elements of same data type arranged in rows and columns (that is, in two dimensions), Sometimes called matrices or tables.

Declaration of Two-Dimensional Array

datatype arrayName [numberOfRows][numberOfColumn];

Where *numberOfRows* and *numberOfColumn* are expressions yielding positive integer values, and specify the number of rows and the number of columns, respectively, in the array.

The array will have *numberOfRows* x *numberOfColumn* elements of the same type and can be thought of as an array of arrays. The first index indicates which of *numberOfRows* subarrays to access, and then the second index accesses one of *numberOfColumn* elements within that subarray. Initialization and access thus work similarly to the one-dimensional case:

For example: *int A [3][4];*

Creates a matrix with three rows and four columns.

A[0][0]	A[0][1]	A[0][2]	A[0][3]
A[1][0]	A[1][1]	A[1][2]	A[1][3]
A[2][0]	A[2][1]	A[2][2]	A[2][3]

Multidimensional arrays are merely an abstraction for programmers, as all of the elements in the array are sequential in memory. Declaring *int arr[2][4];* is the same thing as declaring *int arr[8];*.

Scanned by CamScanner

Initialization of Two-Dimensional Array

Two-dimensional arrays can be initialized by many methods:

- when they are declared, elements of each row are enclosed within braces and separated by commas

```
int Array[2][4] = { { 6, 0, 9, 6 }, { 2, 0, 1, 1 } };
```

- All rows are enclosed within braces .

```
int Array[2][4] = { 6, 0, 9, 6, 2, 0, 1, 1 };
```

- Declaring an array by specifying the number of columns only. The number of rows are determined implicitly by the initializer.

```
int x[][3]={1,2,3,4,5,6,7};
```

1	2	3
4	5	6
7	0	0

```
int y[][2]={{1,2},{3,4},{5}};
```

1	2
3	4
5	0

```
int y[][2]; // cause an error
```

Note: For number arrays, if all components of a row aren't specified, unspecified ones are set to 0.

- To initialize a certain row or column elements we need to loop through it. For example to initialize row number 1 (i.e., second row) to 0 :

Scanned by CamScanner

```
int A[5][2],cols=2;
    for ( int j = 0; j < cols; j++ ) {
        A[1][j]=0;
```

Using Loop to input in Two-Dimensional Array from user

```
int mat[3][5], row, col ;
for (row = 0; row < 3; row++)
        for (col = 0; col < 5; col++)
                cin >> mat[row][col];
```

Referring to Array Elements

To access the elements of a two-dimensional array, we need a pair of indices: one for the row position and one for the column position (also called subscripts). The format is as simple as

name[rowIndex][columnIndex].

Examples:

- *int val = A[2][3];* this statement will take 4th element from the 3rd row of the array. You can verify it in the following diagram:

A[0][0]: **1**	A[0][1]: **2**	A[0][2]: **3**	A[0][3]: **4**
A[1][0]: **5**	A[1][1]: **6**	A[1][2]: **7**	A[1][3]: **8**
A[2][0]: **9**	A[2][1]: **10**	A[2][2]: **11**	A[2][3]: **12**

- *cout << A[1][2]; //print an array element A[1][2] = 7;*
- *A[1][0]=20; // assign value to an array element.*
- *cin >> A[1][2]; //input element*

Scanned by CamScanner

In addition to access one element, we can process the entire array, or process a particular row of the array (called row processing), or process a particular column of the array, (called column processing). Each row and each column of a two-dimensional array is a one-dimensional array.

Basic Operation on Two Dimensional Arrays

Rather than initialize a certain row or column elements and input matrix values from user, there are many operation can be done. We will show subset of them:

- To *print* all matrix elements:

```
#include <iostream>
using namespace std;
int main () {
  // an array with 5 rows and 2 columns.
  int a[5][2] = { {0,0}, {1,2}, {2,4}, {3,6},{4,8}};
  // output each array element's value
  for ( int i = 0; i < 5; i++ )
    for ( int j = 0; j < 2; j++ ) {
      cout << "a[" << i << "][" << j << "]: ";
      cout << a[i][j]<< endl;
    }
}
```


Scanned by CamScanner

The program output will be:

```
a[0][0]: 0
a[0][1]: 0
a[1][0]: 1
a[1][1]: 2
a[2][0]: 2
a[2][1]: 4
a[3][0]: 3
a[3][1]: 6
a[4][0]: 4
a[4][1]: 8
```

- To *initialize* the entire matrix to 0:

```
int arr[3][3];
  for ( int i = 0; i < 3; i++ )
    for ( int j = 0; j < 3; j++ ) {
      arr[i][j]=0;
    }
```

- To find the *sum of each individual column*:

```
#include <iostream>
using namespace std;
int main () {
   int arr[3][3]={1,2,2,4,5,6,7,8,2},sum=0;
   for ( int col = 0; col < 3; col++ ){
      sum=0;
      for ( int row = 0; row < 3 ; row++ )
         sum=sum+arr[row][col];
      cout<<"Sum of column "<<col+1<<"="<<sum<<endl;
   }
}
```


Scanned by CamScanner

The output:

```
Sum of column 1=12
Sum of column 2=15
Sum of column 3=10
```

- To find the *sum of row number 3* of matrix marks:

```
#include <iostream>
using namespace std;
int main () {
    int marks[3][3]={{70,88,67},{90,65,87},{97,65,83}}, sum=0;
    for ( int col = 0; col < 3; col++ ){
       sum=sum+marks[2][col];
    }
    cout<<"Sum of row three ="<<sum<<endl;
}
```

The output:

```
Sum of row three =245
```

Note that dimensions must always be provided when initializing multidimensional arrays, as it is otherwise impossible for the compiler to determine what the intended element partitioning is. For the same reason, when multidimensional arrays are specified as arguments to functions, all dimensions except the first must be provided (the first dimension is optional), as in the following:

int aFunction(int arr[][4]) { ... }

Scanned by CamScanner

Example:

```
#include <iostream>
using namespace std;
void print(int A[][3], int N, int M) {
    for (int R = 0; R < N; R++)
        for (int C = 0; C < M; C++)
            cout << A[R][C];
}
int main () {
    int arr[4][3] ={{12, 29, 11}, {25, 25, 13}, {24, 64, 67}, {11, 18, 14}};
    print(arr,4,3);
}
```

The output:

```
229112525132464671111814
```

Another example on arrays passed through function, the following code is a function to read the array A

```
void Read(int A[][20], int N, int M){
        for(int R = 0; R < N; R++)
                for(int C = 0; C < M; C++){
                        cout << "(R<<','<<")?";
                        cin >> A[R][C];
                }
}
```


Scanned by CamScanner

PRE-LAB WRITING ASSIGNMENT

Fill in the blank

1. When 2D arrays are declared, elements of each row are enclosed within ---------- and separated by ----------.

2. To access the elements of a two-dimensional array, we need a pair of indices: one for the ---------- position and one for the ---------- position.

3. To create an integer matrix with three rows and four columns called x : ----------

4. For number arrays, if not all components of a row are specified, unspecified ones are set to ----------.

5. When processing a particular row of the array this operation called ---------- .

6. The matrix indices have another name which is----------.

7. For matrix x[3][4] , the following element is available x[3][5] (is this statement true or false) ----------

Scanned by CamScanner

Lesson 10 Exercises

Assignment 10.1: Working with 2D Array.

Step1: Bring in the *price.cpp* program from Lab9 folder.

Step2: Fill in the code as mentioned in the comments.

Write your code here:

Scanned by CamScanner

Step3: Compile and run the program. Record the output.

Step4: Why does getPrices have the parameters numOfRows and numOfCols passed by reference whereas printPrices has those parameters passed by value?

Step5: add a new function "highestPrice" that returns the highest price in the array price. Call the function in the program to print out the highest value.

Scanned by CamScanner

Write your code here:

Scanned by CamScanner

The code of price.*cpp* is as follows:

```
// This program will read in prices and store them into a two-dimensional array.
// It will print those prices in a table form.

#include <iostream>
#include <conio.h>
using namespace std;

const int MAXROWS = 10;
const int MAXCOLS = 10;
void   getPrices(float[MAXROWS][MAXCOLS], int&, int&); // gets the prices into the array
void   printPrices(float[MAXROWS][MAXCOLS], int, int); // prints data as a table

int main() {
    int rowsUsed;                                  // holds the number of rows used
    int colsUsed;                                  // holds the number of columns used
    float price[MAXROWS][MAXCOLS];                 // declare a 2D array of floats
    getPrices(price, rowsUsed, colsUsed);          // calls getPrices to fill the array
    printPrices(price, rowsUsed, colsUsed);        // calls printPrices to display array
    getch();
}
//*************************************************************************
// getPrices Function Definition
//*************************************************************************
void   getPrices(float pricetable[MAXROWS][MAXCOLS], int& numOfRows, int& numOfCols) {
    cout << "Please input the number of rows from 1 to "<< MAXROWS << endl;
    cin >> numOfRows;
    cout << "Please input the number of columns from 1 to "<< MAXCOLS << endl;
    cin >> numOfCols;
    for (int row = 0;  row < numOfRows;  row++){
        for (int col = 0; col < numOfCols;  col++)
            // Fill in the code to read and store the next value in the array
    }
}
//*************************************************************************
// printPrices Function Definition
//*************************************************************************
void   printPrices(float pricetable[MAXROWS][MAXCOLS], int numOfRows, int numOfCols) {

    for (int row = 0;  row < numOfRows; row++) {
        for (int col = 0;  col < numOfCols; col++)
            // Fill in the code to print the table
    }
}
```


Scanned by CamScanner

Assignment 10.2: Working with arrays.

What is the output of the following codes?

2	6	0
3	8	4

The Code	The Output

int ind=2;
int Arr[][3] = {{2, 6}, {3, 8, 4}};
cout<<Arr[0][ind]<<"\t"
<<Arr[1][ind-1]<<endl;
Arr[0][1] = ++Arr[ind-2][ind];
Arr[1][ind] = Arr[0][ind] * Arr[1][0];
 	
for (int row = 0; row<2; row++) {
for (int col = 0; col<3; col++)
cout <<Arr[row][col]
<<"\t"; cout <<"\n" ;
}
	2,6,0 3,8,4

int Arr[3][3]={1,2,3,4,5,6,7,8,9};
for(int R=0;R<3;R++) {
int SumR=0;
for(int C=0;C<3;C++)
SumR+=Arr[R][C];
cout<<"Row("<<R<<")="<<SumR<<endl;
}	

int Arr[3][3]={1,2,3,4,5,6,7,8,9};
int Max;
for(int R=0;R<3;R++) {
int Max=Arr[R][0];
for(int C=0;C<3;C++)
if(Arr[R][C]>Max)
Max=Arr[C][R];
cout<<"Max of the Row ("<<R<<")="
<<Max<<endl;
}	

Scanned by CamScanner

Assignment 10.3: Write program

Step1: Create a new project named exchange. After you've created the project, create a new source-code file named exchange.cpp.

Step2: Write a program that stores the following data for students in a 2D array.

St#	Mark	Grade
901	64	1
800	88	3
895	95	2
702	59	5
650	81	6

a. Find the summation of Marks.

b. How many Student get Marks >=80.

c. Find the minimum Mark.

d. Print the mark and grade for the student whose number is entered by the user.

Scanned by CamScanner

Write the code here:

```
int data[5][5]={{90,64,13},{500,88,3},
                {895,45,24},
                {702,59,56},
                {650,81,65}};
int sum;
sum = data[0][1] + data[1][1] + data[2][1]
      data[3][1] + data[4][1];
cout<<"sum="<<sum;
int i,j;
j=0;
For(i=0,i<=4;i++)
  {if(data[i][1]>=80]
    j++;}
cout<<j
```

min mark =

Scanned by CamScanner